

# CampusPilot AI

## Complete Source Code

AI-Powered Accessible Campus Navigation

|            |                                                                                                   |
|------------|---------------------------------------------------------------------------------------------------|
| Event      | CapGemini Build-a-thon 2026                                                                       |
| Team       | Anshul Kumaria · Dhruv Sarda · Hrishikesh Kali · Atharva Deshmukh · Rishabh Alekar                |
| Repository | <a href="https://github.com/AK-1612/campuspilot-ai">https://github.com/AK-1612/campuspilot-ai</a> |
| Generated  | June 14, 2026                                                                                     |
| License    | Apache-2.0                                                                                        |

Table of Contents

|    |                                                   |          |
|----|---------------------------------------------------|----------|
| 1. | <b>Backend — Core</b>                             | 3 files  |
|    | backend/main.py                                   |          |
|    | backend/config.py                                 |          |
|    | backend/db_client.py                              |          |
| 2. | <b>Backend — Agent</b>                            | 7 files  |
|    | backend/agent/agent.py                            |          |
|    | backend/agent/intent_classifier.py                |          |
|    | backend/agent/memory.py                           |          |
|    | backend/agent/tools.py                            |          |
|    | backend/agent/profile_handler.py                  |          |
|    | backend/agent/fallbacks.py                        |          |
|    | backend/agent/prompts/system_prompt.md            |          |
| 3. | <b>Backend — Routers</b>                          | 4 files  |
|    | backend/routers/navigate.py                       |          |
|    | backend/routers/qr.py                             |          |
|    | backend/routers/alert.py                          |          |
|    | backend/routers/__init__.py                       |          |
| 4. | <b>Backend — Tests</b>                            | 4 files  |
|    | backend/tests/test_agent.py                       |          |
|    | backend/tests/test_api.py                         |          |
|    | backend/tests/test_tools.py                       |          |
|    | backend/tests/run_tests.py                        |          |
| 5. | <b>Database</b>                                   | 3 files  |
|    | db/schema.cypher                                  |          |
|    | db/seed.cypher                                    |          |
|    | db/schema_notes.md                                |          |
| 6. | <b>Frontend — App Shell</b>                       | 5 files  |
|    | frontend/src/main.tsx                             |          |
|    | frontend/src/App.tsx                              |          |
|    | frontend/src/types.ts                             |          |
|    | frontend/src/data.ts                              |          |
|    | frontend/src/services/api.ts                      |          |
| 7. | <b>Frontend — Components</b>                      | 14 files |
|    | frontend/src/components/CampusMap.tsx             |          |
|    | frontend/src/components/NavigationChat.tsx        |          |
|    | frontend/src/components/ActiveNavigationView.tsx  |          |
|    | frontend/src/components/OutdoorNavigationView.tsx |          |
|    | frontend/src/components/RouteOptionsView.tsx      |          |
|    | frontend/src/components/QRScanner.tsx             |          |
|    | frontend/src/components/SOSScreen.tsx             |          |
|    | frontend/src/components/SosAlertView.tsx          |          |
|    | frontend/src/components/ProfileSelector.tsx       |          |
|    | frontend/src/components/AccessibilityView.tsx     |          |
|    | frontend/src/components/SavedMapView.tsx          |          |

|     |                                                 |         |
|-----|-------------------------------------------------|---------|
|     | frontend/src/components/BottomSheet.tsx         |         |
|     | frontend/src/components/LaptopLanding.tsx       |         |
|     | frontend/src/components/ReportIssueView.tsx     |         |
| 8.  | <b>Frontend — Config</b>                        | 3 files |
|     | frontend/vite.config.ts                         |         |
|     | frontend/tsconfig.json                          |         |
|     | frontend/package.json                           |         |
| 9.  | <b>Evaluations</b>                              | 2 files |
|     | evals/run_eval.py                               |         |
|     | evals/golden_queries.json                       |         |
| 10. | <b>Documentation</b>                            | 9 files |
|     | README.md                                       |         |
|     | docs/architecture.md                            |         |
|     | docs/ai_architecture.md                         |         |
|     | docs/problem_analysis.md                        |         |
|     | docs/limitations_and_risks.md                   |         |
|     | docs/acceptance_criteria/acceptance_criteria.md |         |
|     | docs/research/tech_stack_rationale.md           |         |
|     | docs/adr/adr-001-neo4j.md                       |         |
|     | docs/adr/adr-002-fastapi.md                     |         |

## 01 Backend — Core

### main.py

Python | backend/main.py

48 lines | 1.3 KB

```
1  """
2  CampusPilot AI — FastAPI Application Entry Point
3
4  Configures the FastAPI application, CORS middleware, and API routers.
5  """
6
7  from fastapi import FastAPI
8  from fastapi.middleware.cors import CORSMiddleware
9  from backend.routers import qr, navigate, alert
10
11  app = FastAPI(
12      title="CampusPilot AI API",
13      description=(
14          "Backend services for accessible indoor campus navigation. "
15          "Provides graph-based routing, QR checkpoint resolution, "
16          "and agentic wayfinding via LangChain and Neo4j."
17      ),
18      version="1.0.0",
19      contact={
20          "name": "CampusPilot AI Team",
21          "url": "https://github.com/AK-1612/campuspilot-ai",
22      },
23      license_info={"name": "Apache-2.0"},
24  )
25
26  # CORS — restrict in production to your frontend's origin
27  app.add_middleware(
28      CORSMiddleware,
29      allow_origins=["*"],
30      allow_credentials=True,
31      allow_methods=["*"],
32      allow_headers=["*"],
33  )
34
35  app.include_router(qr.router)
36  app.include_router(navigate.router)
37  app.include_router(alert.router)
38
39
40  @app.get("/", tags=["Health"])
41  def health_check() -> dict:
42      """Returns API health status."""
43      return {"status": "healthy", "service": "CampusPilot AI API", "version": "1.0.0"}
44
45
46  if __name__ == "__main__":
47      import uvicorn
48      uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

### config.py

Python | backend/config.py

12 lines | 0.3 KB

```
1  import os
2  from dotenv import load_dotenv
3
4  load_dotenv()
5
6  class Settings:
7      NE04J_URI = os.getenv("NE04J_URI", "bolt://localhost:7687")
8      NE04J_USER = os.getenv("NE04J_USER", "neo4j")
9      NE04J_PASSWORD = os.getenv("NE04J_PASSWORD")
10     GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")
11
12     settings = Settings()
```

### db\_client.py

Python | backend/db\_client.py

285 lines | 13.3 KB

```
1 import logging
2 from .config import settings
3
4 logger = logging.getLogger("db_client")
5
6 # Import neo4j if available
7 try:
8     from neo4j import GraphDatabase
9     NEO4J_AVAILABLE = True
10 except ImportError:
11     NEO4J_AVAILABLE = False
12
13 # Mock graph fallback data representation
14 # Nodes: Room, Entrance, QRPoint, Corridor, Lift, Stairs
15 SHADOW_MAP_BLOCKED_NODES = set()
16
17 MOCK_NODES = {
18     # Building A
19     "ent-a-0": {"id": "ent-a-0", "building": "Building A", "name": "Main Entrance A", "type": "Entrance", "floor": 0},
20     "qr-a-0": {"id": "qr-a-0", "code": "QR_BUILDING_A_G", "name": "Building A Ground Floor Lobby QR", "type": "QRPoint", "floor": 0},
21     "corr-a-0": {"id": "corr-a-0", "building": "Building A", "name": "Ground Floor Corridor", "type": "Corridor", "floor": 0},
22     "room-a-01": {"id": "room-a-01", "name": "Seminar Hall 101", "type": "Room", "building": "Building A", "floor": 0},
23     "room-a-02": {"id": "room-a-02", "name": "Admission Office 102", "type": "Room", "building": "Building A", "floor": 0},
24     "lift-a": {"id": "lift-a", "building": "Building A", "name": "Main Elevator A", "type": "Lift"},
25     "stairs-a": {"id": "stairs-a", "building": "Building A", "name": "Staircase A", "type": "Stairs"},
26     "qr-a-1": {"id": "qr-a-1", "code": "QR_BUILDING_A_1", "name": "Building A First Floor QR", "type": "QRPoint", "floor": 1},
27     "corr-a-1": {"id": "corr-a-1", "building": "Building A", "name": "First Floor Corridor", "type": "Corridor", "floor": 1},
28     "room-a-11": {"id": "room-a-11", "name": "AI Lab 201", "type": "Room", "building": "Building A", "floor": 1},
29     "room-a-12": {"id": "room-a-12", "name": "Classroom 202", "type": "Room", "building": "Building A", "floor": 1},
30     "qr-a-2": {"id": "qr-a-2", "code": "QR_BUILDING_A_2", "name": "Building A Second Floor QR", "type": "QRPoint", "floor": 2},
31     "corr-a-2": {"id": "corr-a-2", "building": "Building A", "name": "Second Floor Corridor", "type": "Corridor", "floor": 2},
32     "room-a-21": {"id": "room-a-21", "name": "Library", "type": "Room", "building": "Building A", "floor": 2},
33     "room-a-22": {"id": "room-a-22", "name": "Server Room 302", "type": "Room", "building": "Building A", "floor": 2},
34
35     # Building B
36     "ent-b-0": {"id": "ent-b-0", "building": "Building B", "name": "Main Entrance B", "type": "Entrance", "floor": 0},
37     "qr-b-0": {"id": "qr-b-0", "code": "QR_BUILDING_B_G", "name": "Building B Ground Floor Lobby QR", "type": "QRPoint", "floor": 0},
38     "corr-b-0": {"id": "corr-b-0", "building": "Building B", "name": "Ground Floor Corridor B", "type": "Corridor", "floor": 0},
39     "room-b-01": {"id": "room-b-01", "name": "Auditorium 1", "type": "Room", "building": "Building B", "floor": 0},
40     "room-b-02": {"id": "room-b-02", "name": "Cafeteria", "type": "Room", "building": "Building B", "floor": 0},
41     "lift-b": {"id": "lift-b", "building": "Building B", "name": "Elevator B", "type": "Lift"},
42     "stairs-b": {"id": "stairs-b", "building": "Building B", "name": "Staircase B", "type": "Stairs"},
43     "qr-b-1": {"id": "qr-b-1", "code": "QR_BUILDING_B_1", "name": "Building B First Floor QR", "type": "QRPoint", "floor": 1},
44     "corr-b-1": {"id": "corr-b-1", "building": "Building B", "name": "First Floor Corridor B", "type": "Corridor", "floor": 1},
45     "room-b-11": {"id": "room-b-11", "name": "Physics Lab 201", "type": "Room", "building": "Building B", "floor": 1},
46     "room-b-12": {"id": "room-b-12", "name": "Classroom 203", "type": "Room", "building": "Building B", "floor": 1},
47     "qr-b-2": {"id": "qr-b-2", "code": "QR_BUILDING_B_2", "name": "Building B Second Floor QR", "type": "QRPoint", "floor": 2},
48     "corr-b-2": {"id": "corr-b-2", "building": "Building B", "name": "Second Floor Corridor B", "type": "Corridor", "floor": 2},
49     "room-b-21": {"id": "room-b-21", "name": "Dean's Office", "type": "Room", "building": "Building B", "floor": 2},
50     "room-b-22": {"id": "room-b-22", "name": "Conference Room 305", "type": "Room", "building": "Building B", "floor": 2},
51 }
52
53 MOCK_EDGES = [
54     # Building A Ground Floor
55     ("ent-a-0", "qr-a-0", 5.0, True, True, False, False),
56     ("qr-a-0", "corr-a-0", 10.0, True, True, False, False),
57     ("corr-a-0", "room-a-01", 15.0, True, False, False, False),
58     ("corr-a-0", "room-a-02", 8.0, True, False, False, False),
59     ("qr-a-0", "lift-a", 4.0, True, True, False, True),
60     ("corr-a-0", "stairs-a", 6.0, False, False, True, False),
61
62     # Building A First Floor
63     ("qr-a-1", "corr-a-1", 10.0, True, True, False, False),
64     ("corr-a-1", "room-a-11", 12.0, True, False, False, False),
65     ("corr-a-1", "room-a-12", 9.0, True, False, False, False),
66     ("qr-a-1", "lift-a", 4.0, True, True, False, True),
67     ("corr-a-1", "stairs-a", 6.0, False, False, True, False),
68
69     # Building A Second Floor
70     ("qr-a-2", "corr-a-2", 10.0, True, True, False, False),
71     ("corr-a-2", "room-a-21", 14.0, True, True, False, False),
72     ("corr-a-2", "room-a-22", 8.0, True, False, False, False),
73     ("qr-a-2", "lift-a", 4.0, True, True, False, True),
74     ("corr-a-2", "stairs-a", 6.0, False, False, True, False),
75
76     # Building B Ground Floor
77     ("ent-b-0", "qr-b-0", 5.0, True, True, False, False),
78     ("qr-b-0", "corr-b-0", 10.0, True, True, False, False),
```

```

79 ("corr-b-0", "room-b-01", 15.0, True, False, False, False),
80 ("corr-b-0", "room-b-02", 8.0, True, False, False, False),
81 ("qr-b-0", "lift-b", 4.0, True, True, False, True),
82 ("corr-b-0", "stairs-b", 6.0, False, False, True, False),
83
84 # Building B First Floor
85 ("qr-b-1", "corr-b-1", 10.0, True, True, False, False),
86 ("corr-b-1", "room-b-11", 12.0, True, False, False, False),
87 ("corr-b-1", "room-b-12", 9.0, True, False, False, False),
88 ("qr-b-1", "lift-b", 4.0, True, True, False, True),
89 ("corr-b-1", "stairs-b", 6.0, False, False, True, False),
90
91 # Building B Second Floor
92 ("qr-b-2", "corr-b-2", 10.0, True, True, False, False),
93 ("corr-b-2", "room-b-21", 14.0, True, True, False, False),
94 ("corr-b-2", "room-b-22", 8.0, True, False, False, False),
95 ("qr-b-2", "lift-b", 4.0, True, True, False, True),
96 ("corr-b-2", "stairs-b", 6.0, False, False, True, False),
97
98 # Inter-building Path
99 ("ent-a-0", "ent-b-0", 50.0, True, True, False, False),
100 ]
101
102 # Generate bidirectionality for mock edges
103 ADJACENCY_LIST = {}
104 for u, v, dist, wa, tp, st, lf in MOCK_EDGES:
105     if u not in ADJACENCY_LIST: ADJACENCY_LIST[u] = []
106     if v not in ADJACENCY_LIST: ADJACENCY_LIST[v] = []
107     # (neighbor, distance, wheelchair_accessible, tactile_paving, is_stairs, is_lift)
108     ADJACENCY_LIST[u].append((v, dist, wa, tp, st, lf))
109     ADJACENCY_LIST[v].append((u, dist, wa, tp, st, lf))
110
111 class MockSession:
112     def run(self, cypher, **kwargs):
113         # Extremely simplified cypher execution to satisfy basic query verification.
114         # Check if searching for a node or a path
115         if "MATCH (q:QRPoint)" in cypher:
116             # QR code lookup
117             code = kwargs.get("code")
118             for node in MOCK_NODES.values():
119                 if node.get("type") == "QRPoint" and node.get("code") == code:
120                     return [MockRecord({"q": MockNode(node)})]
121             return []
122
123         # Room lookup
124         if "MATCH (r:Room)" in cypher:
125             name = kwargs.get("room_name")
126             for node in MOCK_NODES.values():
127                 if node.get("type") == "Room" and name.lower() in node.get("name", "").lower():
128                     return [MockRecord({"r": MockNode(node)})]
129             return []
130
131         return []
132
133 class MockNode:
134     def __init__(self, props):
135         self._properties = props
136     def __getitem__(self, key):
137         return self._properties.get(key)
138     def get(self, key, default=None):
139         return self._properties.get(key, default)
140     @property
141     def id(self):
142         return self._properties.get("id")
143     @property
144     def element_id(self):
145         return self._properties.get("id")
146
147 class MockRecord:
148     def __init__(self, data):
149         self._data = data
150     def __getitem__(self, key):
151         return self._data[key]
152     def data(self):
153         return self._data
154
155 class MockNeo4jClient:
156     def __init__(self):

```

```

157 logger.info("Initializing Mock Neo4j Client fallback...")
158
159 def session(self):
160     return MockSession()
161
162 def close(self):
163     pass
164
165 class Neo4jClient:
166     def __init__(self):
167         self.driver = None
168         self.is_mock = False
169         if NEO4J_AVAILABLE:
170             try:
171                 self.driver = GraphDatabase.driver(
172                     settings.NEO4J_URI,
173                     auth=(settings.NEO4J_USER, settings.NEO4J_PASSWORD)
174                 )
175                 self.driver.verify_connectivity()
176                 logger.info("Connected to Neo4j successfully!")
177             except Exception as e:
178                 logger.warning(f"Neo4j connection failed: {e}. Falling back to mock client.")
179                 self.driver = MockNeo4jClient()
180                 self.is_mock = True
181             else:
182                 logger.warning("neo4j-driver not installed. Falling back to mock client.")
183                 self.driver = MockNeo4jClient()
184                 self.is_mock = True
185
186     def get_session(self):
187         return self.driver.session()
188
189     def close(self):
190         if self.driver:
191             self.driver.close()
192
193     def flag_shadow_node(self, node_id: str):
194         SHADOW_MAP_BLOCKED_NODES.add(node_id)
195         logger.info(f"Shadow Map updated: {node_id} is now blocked.")
196
197     def find_shortest_path(self, start_id: str, end_id: str, profile: str = "default"):
198         # Map profile constraints to boolean flags
199         wheelchair_req = profile.lower() in ["wheelchair", "mobility_impaired"]
200         tactile_req = profile.lower() in ["vision_impaired", "visually_impaired"]
201
202         if self.is_mock or isinstance(self.driver, MockNeo4jClient):
203             return self._find_mock_path(start_id, end_id, wheelchair_req, tactile_req)
204
205         # Execute Cypher pathfinding query
206         query = """
207         MATCH (start {id: $start_id}), (end {id: $end_id})
208         MATCH p = shortestPath((start)-[:CONNECTS_TO*..30]->(end))
209         WHERE ALL(r IN relationships(p) WHERE
210             (CASE WHEN $wheelchair_req = true THEN r.wheelchair_accessible = true ELSE true END) AND
211             (CASE WHEN $tactile_req = true THEN r.tactile_paving = true ELSE true END)
212         )
213         RETURN [n IN nodes(p) | {id: n.id, name: n.name, type: labels(n)[0], building: n.building, floor: n.floor}] as
214         path, [r IN relationships(p) | {distance_meters: r.distance_meters, wheelchair_accessible: r.wheelchair_accessible,
215         tactile_paving: r...
216         try:
217             with self.get_session() as session:
218                 result = session.run(query, start_id=start_id, end_id=end_id, wheelchair_req=wheelchair_req, tactile_req=tactile_req)
219                 record = result.single()
220                 if record:
221                     return {
222                         "nodes": record["path_nodes"],
223                         "edges": record["path_edges"]
224                     }
225         except Exception as e:
226             logger.warning(f"Cypher path query failed: {e}. Falling back to mock path computation.")
227
228         return self._find_mock_path(start_id, end_id, wheelchair_req, tactile_req)
229
230     def _find_mock_path(self, start_id: str, end_id: str, wheelchair_req: bool, tactile_req: bool):
231         # Python Dijkstra implementation
232         import heapq
233
234         # Priority queue: (cost, current_node, path_history, edges_history)

```

```
235 queue = [(0.0, start_id, [start_id], [])]
236 visited = set()
237
238 while queue:
239     cost, node, path, edges = heapq.heappop(queue)
240
241     if node == end_id:
242         # Format node dicts and edge dicts
243         path_nodes = []
244         for nid in path:
245             node_info = MOCK_NODES.get(nid, {"id": nid, "name": nid, "type": "Location"})
246             # Ensure "type" defaults nicely
247             ntype = node_info.get("type", "Location")
248             path_nodes.append({
249                 "id": node_info.get("id"),
250                 "name": node_info.get("name"),
251                 "type": ntype,
252                 "building": node_info.get("building"),
253                 "floor": node_info.get("floor")
254             })
255             return {
256                 "nodes": path_nodes,
257                 "edges": edges
258             }
259
260     if node in visited:
261         continue
262     visited.add(node)
263
264     neighbors = ADJACENCY_LIST.get(node, [])
265     for neighbor, dist, wa, tp, stairs, lift in neighbors:
266         if neighbor in visited or neighbor in SHADOW_MAP_BLOCKED_NODES:
267             continue
268         # Filter by constraints
269         if wheelchair_req and not wa:
270             continue
271         if tactile_req and not tp:
272             continue
273
274         new_edges = list(edges)
275         new_edges.append({
276             "distance_meters": dist,
277             "wheelchair_accessible": wa,
278             "tactile_paving": tp
279         })
280         heapq.heappush(queue, (cost + dist, neighbor, path + [neighbor], new_edges))
281
282     return None
283
284 db_client = Neo4jClient()
285
```



## 02 Backend — Agent

agent.py

Python | backend/agent/agent.py

118 lines | 4.2 KB

```
1 import os
2 import json
3 from typing import Dict, Any, List, Tuple
4
5 from dotenv import load_dotenv
6 load_dotenv()
7
8 from langchain_groq import ChatGroq
9 from langchain.agents import create_tool_calling_agent, AgentExecutor
10 from langchain_core.prompts import ChatPromptTemplate
11 from langchain_core.messages import HumanMessage, SystemMessage, AIMessage, ToolMessage
12 from .tools import route_query, qr_lookup, profile_detect, flag_obstacle, resolve_room
13 from .memory import AgentMemory
14 from .intent_classifier import classify_intent
15
16
17 def load_system_prompt() -> str:
18     """
19     Loads the system prompt from the markdown file.
20     """
21     prompt_path = os.path.join(os.path.dirname(__file__), "prompts", "system_prompt.md")
22     try:
23         with open(prompt_path, "r", encoding="utf-8") as f:
24             return f.read()
25     except FileNotFoundError:
26         return "You are a helpful campus navigation assistant."
27
28
29 def create_campus_agent() -> AgentExecutor:
30     """
31     Initializes the LangChain tool-calling agent with the Groq LLM
32     (llama-3.3-70b-versatile) and the required set of navigation tools.
33     """
34     llm = ChatGroq(model="llama-3.3-70b-versatile", temperature=0)
35     tools = [route_query, qr_lookup, profile_detect, flag_obstacle, resolve_room]
36     system_message = load_system_prompt()
37
38     prompt = ChatPromptTemplate.from_messages([
39         ("system", system_message),
40         ("placeholder", "{chat_history}"),
41         ("human", "{input}"),
42         ("placeholder", "{agent_scratchpad}"),
43     ])
44
45     agent = create_tool_calling_agent(llm, tools, prompt)
46     return AgentExecutor(agent=agent, tools=tools, verbose=True, return_intermediate_steps=True)
47
48
49 class CampusPilotAgent:
50     """
51     The main orchestrator for the CampusPilot AI backend.
52     """
53
54     def __init__(self) -> None:
55         """Initializes the agent executor and memory layer."""
56         self.executor = create_campus_agent()
57         self.memory = AgentMemory()
58
59     def process_query(self, query: str, context_overrides: dict = None) -> dict:
60         """
61         Processes a natural language query from the user.
62
63         Args:
64             query (str): The user's input string.
65             context_overrides (dict): Optional dict to pass current QR location and profile.
66
67         Returns:
68             dict: Contains 'response' (LLM text), 'route_data' (JSON from route tool if any),
69                 'intent', and 'intermediate_steps_raw' (list of (action, observation) tuples).
70         """
```

```

71     intent = classify_intent(query)
72     self.memory.add_interaction("user", query)
73     context = self.memory.get_context()
74     if context_overrides:
75         context.update(context_overrides)
76
77     # Enrich the input with internal state context before passing to the LLM
78     enriched_input = (
79         f"[Context: Intent={intent}, "
80         f"Profile={context.get('profile', 'default')}, "
81         f"Location (QR Code)={context.get('qr_location', 'unknown')}] "
82         f"\nUser Query: {query}"
83     )
84
85     try:
86         result = self.executor.invoke({"input": enriched_input})
87
88         # Extract tool calls from intermediate steps
89         intermediate_steps = []
90         route_data = None
91
92         for action, observation in result.get("intermediate_steps", []):
93             intermediate_steps.append((action, observation))
94             if action.tool == "route_query":
95                 try:
96                     parsed = json.loads(observation)
97                     if "route" in parsed:
98                         route_data = parsed["route"]
99                 except Exception:
100                     pass
101
102         response_text = result.get("output", "I could not generate a response.")
103         self.memory.add_interaction("assistant", response_text)
104
105         return {
106             "response": response_text,
107             "route_data": route_data,
108             "intent": intent,
109             "intermediate_steps_raw": intermediate_steps
110         }
111
112     except Exception as e:
113         return {
114             "response": f"An error occurred while processing your request: {str(e)}",
115             "route_data": None,
116             "intent": intent,
117             "intermediate_steps_raw": []
118         }

```

**intent\_classifier.py**

Python | backend/agent/intent\_classifier.py

54 lines | 1.6 KB

```

1     """
2     Pre-LLM intent classifier for the CampusPilot agent.
3
4     Performs keyword-based classification in sub-millisecond time before the
5     LangChain executor is invoked. Emergency intents are detected here so that
6     safety responses are never delayed by LLM inference latency.
7     """
8
9     _EMERGENCY_KEYWORDS = frozenset([
10         "help", "emergency", "fire", "police", "hurt", "sos", "fell", "stuck",
11         "ambulance", "collapse", "attack",
12     ])
13
14     _ACCESSIBILITY_KEYWORDS = frozenset([
15         "wheelchair", "blind", "deaf", "ramp", "elevator", "lift", "accessible",
16         "step-free", "visual impair", "hearing impair",
17     ])
18
19     _FACILITY_KEYWORDS = frozenset([
20         "where is", "find", "looking for", "locate", "nearest", "closest",
21     ])
22
23     _NAVIGATE_KEYWORDS = frozenset([
24         "take me", "go to", "route", "navigate", "directions", "how do i get",
25         "path to", "way to",

```

```

26 ])
27
28
29 def classify_intent(query: str) -> str:
30     """
31     Classify a user query into one of four system intents.
32
33     Evaluation order: emergency > accessibility_query > find_facility > navigate.
34     Emergency takes strict precedence to ensure safety responses are never
35     delayed by lower-priority classification branches.
36
37     Args:
38     query: Raw text input from the user.
39
40     Returns:
41     One of: 'emergency', 'accessibility_query', 'find_facility', 'navigate'.
42     """
43     normalised = query.lower()
44
45     if any(kw in normalised for kw in _EMERGENCY_KEYWORDS):
46         return "emergency"
47
48     if any(kw in normalised for kw in _ACCESSIBILITY_KEYWORDS):
49         return "accessibility_query"
50
51     if any(kw in normalised for kw in _FACILITY_KEYWORDS):
52         return "find_facility"
53
54     return "navigate"

```

## memory.py

Python | backend/agent/memory.py

57 lines | 2.0 KB

```

1  from typing import List, Dict, Any, Optional
2
3  class AgentMemory:
4      """
5      A lightweight state management layer for the CampusPilot agent.
6
7      Tracks conversational history, the active disability profile,
8      and the last known physical location based on QR scans.
9      """
10
11     def __init__(self) -> None:
12         """Initializes empty memory structures and default settings."""
13         self.history: List[Dict[str, str]] = []
14         self.last_known_qr_location: Optional[str] = None
15         self.current_profile: str = "Invisible"
16
17     def add_interaction(self, role: str, content: str) -> None:
18         """
19         Records a message to the conversational history.
20
21         Args:
22         role (str): The role of the speaker (e.g., 'user', 'assistant').
23         content (str): The message text.
24         """
25         self.history.append({"role": role, "content": content})
26
27     def update_location(self, qr_id: str, location_name: str) -> None:
28         """
29         Updates the user's last known physical location.
30
31         Args:
32         qr_id (str): The ID of the scanned QR code.
33         location_name (str): The human-readable name of the location.
34         """
35         self.last_known_qr_location = location_name
36
37     def update_profile(self, profile: str) -> None:
38         """
39         Updates the active disability routing profile for the session.
40
41         Args:
42         profile (str): The validated profile name.
43         """
44         self.current_profile = profile

```

```

45
46 def get_context(self) -> Dict[str, Any]:
47     """
48     Retrieves the current session context summarizing the user's state.
49
50     Returns:
51     Dict[str, Any]: A dictionary containing location, profile, and history metadata.
52     """
53     return {
54         "last_location": self.last_known_qr_location,
55         "profile": self.current_profile,
56         "history_length": len(self.history)
57     }

```

## tools.py

Python | backend/agent/tools.py

165 lines | 5.5 KB

```

1  """
2  LangChain tool definitions for the CampusPilot agent.
3
4  Each tool is registered via the @tool decorator and exposed to the
5  LangChain AgentExecutor. The LLM autonomously decides which tools to
6  call and in what order during each ReAct reasoning cycle.
7  """
8
9  import json
10 from langchain_core.tools import tool
11 from backend.db_client import db_client, MOCK_NODES
12
13
14 @tool
15 def route_query(start_node: str, end_node: str, profile: str) -> str:
16     """
17     Calculate the shortest accessible route between two campus nodes.
18
19     Args:
20     start_node: Exact node ID of the origin (e.g., 'qr-a-0'). Must be
21     resolved via qr_lookup before calling this tool.
22     end_node: Exact node ID of the destination (e.g., 'room-a-21'). Must be
23     resolved via resolve_room before calling this tool.
24     profile: Active disability profile (e.g., 'wheelchair', 'vision_impaired').
25     Determines which graph edges are eligible.
26
27     Returns:
28     JSON string with 'status' and 'route' on success, or 'error' on failure.
29     """
30     path_data = db_client.find_shortest_path(start_node, end_node, profile)
31     if not path_data:
32         return json.dumps({
33             "error": f"No accessible route found for profile '{profile}' between '{start_node}' and '{end_node}'."
34         })
35     return json.dumps({"status": "success", "route": path_data})
36
37
38 @tool
39 def qr_lookup(qr_id: str) -> str:
40     """
41     Resolve a scanned QR code to its campus node data.
42
43     Args:
44     qr_id: The code value embedded in the physical QR marker
45     (e.g., 'QR_BUILDING_A_G').
46
47     Returns:
48     JSON string with node metadata (id, name, building, floor), or 'error'.
49     """
50     for node in MOCK_NODES.values():
51         if node.get("type") == "QRPoint" and node.get("code") == qr_id:
52             return json.dumps(node)
53
54     cypher = (
55         "MATCH (q:QRPoint {code: $code}) "
56         "RETURN q.id AS id, q.name AS name, q.building AS building, q.floor AS floor"
57     )
58     try:
59         with db_client.get_session() as session:
60             result = session.run(cypher, code=qr_id)

```

```

61     record = result.single()
62     if record:
63         return json.dumps({
64             "id": record["id"],
65             "name": record["name"],
66             "type": "QRPoint",
67             "building": record["building"],
68             "floor": record["floor"],
69         })
70     except Exception:
71         pass
72
73     return json.dumps({"error": f"QR code '{qr_id}' not found."})
74
75
76 @tool
77 def resolve_room(room_name: str) -> str:
78     """
79     Look up a room node ID by name or partial description.
80
81     Call this tool before route_query to convert a human-readable destination
82     such as 'library' or 'Room 204' into a graph node ID.
83
84     Args:
85     room_name: Full or partial name of the room (e.g., 'library', 'AI Lab').
86
87     Returns:
88     JSON string with node metadata (id, name, building, floor), or 'error'.
89     """
90     name_lower = room_name.lower()
91     for node in MOCK_NODES.values():
92         if node.get("type") == "Room" and name_lower in str(node.get("name", "")).lower():
93             return json.dumps(node)
94
95     cypher = (
96         "MATCH (r:Room) WHERE toLower(r.name) CONTAINS $name "
97         "RETURN r.id AS id, r.name AS name, r.building AS building, r.floor AS floor "
98         "LIMIT 1"
99     )
100     try:
101         with db_client.get_session() as session:
102             result = session.run(cypher, name=name_lower)
103             record = result.single()
104             if record:
105                 return json.dumps({
106                     "id": record["id"],
107                     "name": record["name"],
108                     "type": "Room",
109                     "building": record["building"],
110                     "floor": record["floor"],
111                 })
112             except Exception:
113                 pass
114
115     return json.dumps({"error": f"No room matching '{room_name}' found."})
116
117
118 @tool
119 def profile_detect(user_input: str) -> str:
120     """
121     Infer a disability profile from implicit cues in the user's message.
122
123     Used when the user has not explicitly selected a profile but their
124     language suggests a specific accessibility need.
125
126     Args:
127     user_input: The raw message text from the user.
128
129     Returns:
130     A profile name string: 'Wheelchair', 'Vision-impaired',
131     'Hearing-impaired', 'Cognitive', or 'Invisible' (default).
132     """
133     text = user_input.lower()
134     if any(kw in text for kw in ("wheelchair", "ramp", "elevator", "step-free")):
135         return "Wheelchair"
136     if any(kw in text for kw in ("blind", "vision", "visually impaired", "tactile", "audio")):
137         return "Vision-impaired"
138     if any(kw in text for kw in ("deaf", "hearing", "caption", "visual alert")):

```

```
139     return "Hearing-impaired"
140     if any(kw in text for kw in ("cognitive", "simple", "slow", "one step")):
141         return "Cognitive"
142     return "Invisible"
143
144
145 @tool
146 def flag_obstacle(node_id: str, description: str) -> str:
147     """
148     Report a temporary obstacle and mark the affected node on the shadow map.
149
150     The shadow map layer blocks flagged nodes from appearing in future route
151     calculations until the obstacle is cleared.
152
153     Args:
154     node_id: Exact graph node ID of the blocked location
155             (e.g., 'lift-a', 'corr-a-0').
156     description: Plain-language description of the obstacle.
157
158     Returns:
159     Confirmation string.
160     """
161     db_client.flag_shadow_node(node_id)
162     return (
163         f"Obstacle recorded at node '{node_id}': {description}. "
164         "The node has been flagged on the shadow map and will be excluded from routing."
165     )
```

**profile\_handler.py**

Python | backend/agent/profile\_handler.py

58 lines | 1.8 KB

```
1  from enum import Enum
2  from typing import Dict
3
4  class DisabilityProfile(Enum):
5      """Enumeration of the supported user disability profiles."""
6      WHEELCHAIR = "Wheelchair"
7      VISION_IMPAIRED = "Vision-impaired"
8      HEARING_IMPAIRED = "Hearing-impaired"
9      COGNITIVE = "Cognitive"
10     INVISIBLE = "Invisible"
11
12
13 def get_routing_constraints(profile: DisabilityProfile) -> Dict[str, bool]:
14     """
15     Maps a disability profile to specific parameters for the routing engine.
16
17     Args:
18     profile (DisabilityProfile): The active profile enum.
19
20     Returns:
21     Dict[str, bool]: A dictionary of boolean constraint flags to be passed
22                     to the Neo4j pathfinding algorithm.
23     """
24     constraints: Dict[str, bool] = {
25         "avoid_stairs": False,
26         "prefer_elevators": False,
27         "detailed_landmarks": False,
28         "visual_cues_only": False,
29         "simplified_steps": False
30     }
31
32     if profile == DisabilityProfile.WHEELCHAIR:
33         constraints["avoid_stairs"] = True
34         constraints["prefer_elevators"] = True
35     elif profile == DisabilityProfile.VISION_IMPAIRED:
36         constraints["detailed_landmarks"] = True
37     elif profile == DisabilityProfile.HEARING_IMPAIRED:
38         constraints["visual_cues_only"] = True
39     elif profile == DisabilityProfile.COGNITIVE:
40         constraints["simplified_steps"] = True
41
42     return constraints
43
44
45 def parse_profile(profile_str: str) -> DisabilityProfile:
46     """
```

```
47     Safely parses a string into a DisabilityProfile enum.
48
49     Args:
50     profile_str (str): The raw profile string (e.g., from an API or database).
51
52     Returns:
53     DisabilityProfile: The matched enum, or DisabilityProfile.INVISIBLE as a safe fallback.
54     """
55     try:
56     return DisabilityProfile(profile_str)
57     except ValueError:
58     return DisabilityProfile.INVISIBLE
```

**fallbacks.py**

Python | backend/agent/fallbacks.py

69 lines | 2.1 KB

```
1     """
2     Fallback response generators for the CampusPilot agent.
3
4     These functions provide safe, pre-approved responses for situations where
5     the routing engine cannot produce a result or where the LLM must be bypassed
6     entirely (e.g., emergency detection).
7     """
8
9
10    def handle_emergency() -> str:
11        """
12        Returns the standard emergency response.
13
14        This response is returned immediately when an emergency intent is detected
15        by the pre-LLM heuristic classifier. It does not depend on LLM inference.
16        """
17        return (
18        "Emergency detected. Campus Security has been alerted. "
19        "Please stay calm and proceed to the nearest marked exit. "
20        "If you require immediate assistance, call campus security at extension 100."
21        )
22
23
24    def handle_ambiguous_query(query: str) -> str:
25        """
26        Returns a clarification request when the agent cannot determine intent.
27
28        Args:
29        query: The ambiguous user query.
30
31        Returns:
32        A plain-language request for more specific information.
33        """
34        return (
35        "I was unable to determine a destination from your request. "
36        "Please specify a building name, room number, or facility type."
37        )
38
39
40    def handle_no_route_found(start: str, end: str) -> str:
41        """
42        Returns a safe response when no accessible route exists between two nodes.
43
44        Args:
45        start: Starting node ID or label.
46        end: Destination node ID or label.
47
48        Returns:
49        A fallback message with guidance for the user.
50        """
51        return (
52        f"No accessible route could be found from {start} to {end} at this time. "
53        "This may be due to a temporary closure or missing accessibility data. "
54        "Please contact campus facilities or speak to a member of staff."
55        )
56
57
58    def handle_offline_mode() -> str:
59        """
60        Returns the standard notification for offline or degraded-connectivity operation.
61    """
```





## 03 Backend — Routers

navigate.py

Python | backend/routers/navigate.py

93 lines | 2.8 KB

```

1  """
2  Navigation router — POST /navigate
3
4  Accepts a natural-language query and routes it through the CampusPilot agent.
5  Emergency intents are short-circuited before the LLM is invoked.
6  """
7
8  from fastapi import APIRouter
9  from pydantic import BaseModel
10 from typing import List, Optional, Any
11
12 from backend.agent.agent import CampusPilotAgent
13 from backend.agent.intent_classifier import classify_intent
14 from backend.agent.fallbacks import handle_emergency
15
16 router = APIRouter(prefix="/navigate", tags=["Navigation"])
17
18 # Agent is instantiated once at startup; AgentExecutor is thread-safe for reads.
19 _agent = CampusPilotAgent()
20
21 # Maximum output length for tool observations in API responses
22 MAX_OBSERVATION_LENGTH = 400
23
24
25 class AgentStep(BaseModel):
26     """A single tool invocation recorded during an agent reasoning cycle."""
27     tool: str
28     input: str
29     output: str
30
31
32 class NavigateRequest(BaseModel):
33     origin: str
34     destination: str
35     profile: str
36     qr_location: Optional[str] = None
37
38
39 class NavigateResponse(BaseModel):
40     response: str
41     route_data: Optional[Any] = None
42     agent_steps: List[AgentStep] = []
43     intent: Optional[str] = None
44     profile_applied: Optional[str] = None
45
46
47 @router.post("", response_model=NavigateResponse)
48 @router.post("/", response_model=NavigateResponse)
49 def navigate(req: NavigateRequest) -> NavigateResponse:
50     """
51     Process a navigation query through the CampusPilot agentic pipeline.
52
53     Emergency intents are detected by a sub-millisecond heuristic classifier
54     before the LLM is invoked, ensuring immediate safety responses.
55     All other intents are resolved by the LangChain ReAct agent using the
56     registered tool set (qr_lookup, resolve_room, route_query, flag_obstacle).
57     """
58     intent = classify_intent(req.destination)
59
60     if intent == "emergency":
61         return NavigateResponse(
62             response=handle_emergency(),
63             agent_steps=[AgentStep(
64                 tool="emergency_classifier",
65                 input=req.destination,
66                 output="Emergency intent detected via pre-LLM heuristic. LLM invocation bypassed."
67             )],
68             intent="emergency",
69             profile_applied=req.profile,
70         )

```

```
71
72     context_overrides = {
73         "qr_location": req.qr_location or req.origin,
74         "profile": req.profile,
75     }
76
77     result = _agent.process_query(req.destination, context_overrides)
78
79     steps: List[AgentStep] = []
80     for action, observation in result.get("intermediate_steps_raw", []):
81         steps.append(AgentStep(
82             tool=action.tool,
83             input=str(action.tool_input),
84             output=str(observation)[:MAX_OBSERVATION_LENGTH],
85         ))
86
87     return NavigateResponse(
88         response=result.get("response", ""),
89         route_data=result.get("route_data"),
90         agent_steps=steps,
91         intent=result.get("intent"),
92         profile_applied=req.profile,
93     )
```

**qr.py**

Python | backend/routers/qr.py

73 lines | 2.2 KB

```
1  """
2  QR router - GET /qr/lookup
3
4  Resolves a scanned QR code to its campus node metadata.
5  """
6
7  import logging
8  from fastapi import APIRouter, HTTPException, Query
9  from pydantic import BaseModel
10 from typing import Optional
11
12 from backend.db_client import db_client, MOCK_NODES
13
14 logger = logging.getLogger(__name__)
15 router = APIRouter(prefix="/qr", tags=["QR Codes"])
16
17
18 class QRLocationResponse(BaseModel):
19     id: str
20     code: str
21     name: str
22     building: str
23     floor: int
24     description: Optional[str] = None
25
26
27 @router.get("/lookup", response_model=QRLocationResponse)
28 def lookup_qr(
29     code: str = Query(..., description="QR code value scanned from the physical marker.")
30 ) -> QRLocationResponse:
31     """
32     Resolve a QR code to its associated campus location node.
33
34     Attempts resolution against Neo4j first, then falls back to the
35     in-memory mock graph if the database is unavailable.
36     """
37     cypher = """
38     MATCH (q:QRPoint {code: $code})
39     RETURN q.id AS id, q.code AS code, q.name AS name,
40            q.building AS building, q.floor AS floor,
41            q.description AS description
42     """
43     try:
44         with db_client.get_session() as session:
45             result = session.run(cypher, code=code)
46             record = result.single()
47             if record:
48                 return QRLocationResponse(
49                     id=record["id"],
50                     code=record["code"],
```

```

51     name=record["name"],
52     building=record["building"],
53     floor=record["floor"],
54     description=record["description"],
55 )
56 except Exception as exc:
57     logger.warning("Neo4j QR lookup failed, using mock fallback: %s", exc)
58
59     for node in MOCK_NODES.values():
60         if node.get("type") == "QRPoint" and node.get("code") == code:
61             return QRLocationResponse(
62                 id=node["id"],
63                 code=node["code"],
64                 name=node["name"],
65                 building=node["building"],
66                 floor=node["floor"],
67                 description=node.get("description"),
68             )
69
70     raise HTTPException(
71         status_code=404,
72         detail=f"QR code '{code}' was not recognised. Verify the code and try again.",
73     )

```

## alert.py

Python | backend/routers/alert.py

61 lines | 1.8 KB

```

1  """
2  Alert router - POST /alert
3
4  Receives hazard and obstacle reports from the frontend.
5  In production this would persist to a database and trigger re-routing.
6  """
7
8  import logging
9  from fastapi import APIRouter
10 from pydantic import BaseModel, Field
11 from typing import Optional
12 from datetime import datetime, timezone
13
14 logger = logging.getLogger(__name__)
15 router = APIRouter(prefix="/alert", tags=["Alerts"])
16
17
18 class AlertRequest(BaseModel):
19     user: str = Field(..., description="Identifier of the reporting user or session.")
20     user_id: Optional[str] = Field(None, alias="userId")
21     description: str = Field(..., description="Description of the hazard or obstacle.")
22     location: Optional[str] = Field(None, description="Human-readable location string.")
23     is_custom: bool = Field(True, alias="isCustom")
24
25     class Config:
26         populate_by_name = True
27
28
29 class AlertResponse(BaseModel):
30     status: str
31     alert_id: str
32     timestamp: str
33     message: str
34
35
36 @router.post("", response_model=AlertResponse)
37 def report_alert(alert: AlertRequest) -> AlertResponse:
38     """
39     Record a campus hazard or obstacle report.
40
41     The report is acknowledged and logged. In a production deployment this
42     endpoint would persist the incident and propagate it to the shadow map
43     routing layer so affected paths are recalculated in real time.
44     """
45     import uuid
46     alert_id = f"ALT-{uuid.uuid4().hex[:8].upper()}"
47     timestamp = datetime.now(timezone.utc).isoformat()
48
49     logger.info(
50         "Alert received | id=%s | location=%s | description=%s",

```

```
51     alert_id,  
52     alert.location,  
53     alert.description,  
54     )  
55  
56     return AlertResponse(  
57         status="received",  
58         alert_id=alert_id,  
59         timestamp=timestamp,  
60         message="Hazard report recorded. Routing engine has been notified.",  
61     )
```

**\_\_init\_\_.py**

Python | backend/routers/\_\_init\_\_.py

1 lines | 0.0 KB

```
1 # Routers package initialization
```

## 04 Backend — Tests

### test\_agent.py

Python | backend/tests/test\_agent.py

25 lines | 0.7 KB

```
1 import sys, pytest
2 from pathlib import Path
3
4 sys.path.append(str(Path(__file__).resolve().parents[2]))
5 from backend.agent.agent import CampusPilotAgent
6
7
8 def test_process_query():
9     agent = CampusPilotAgent()
10    result = agent.process_query("Take me to the library")
11
12    # result is a dict with response, route_data, intent, intermediate_steps_raw
13    assert isinstance(result, dict)
14    assert "response" in result
15    assert len(result["response"]) > 0
16    assert "intent" in result
17
18
19 def test_process_query_not_empty():
20     agent = CampusPilotAgent()
21     result = agent.process_query("Where is the CS Lab?")
22
23     assert isinstance(result, dict)
24     assert len(result) > 0
25     assert result.get("response")
```

### test\_api.py

Python | backend/tests/test\_api.py

65 lines | 2.0 KB

```
1 import sys
2 from pathlib import Path
3 from fastapi.testclient import TestClient
4
5 sys.path.append(str(Path(__file__).resolve().parents[2]))
6 from backend.main import app
7
8 client = TestClient(app)
9
10 def test_read_root():
11     response = client.get("/")
12     assert response.status_code == 200
13     assert response.json()["status"] == "healthy"
14
15 def test_qr_lookup_success():
16     response = client.get("/qr/lookup?code=QR_BUILDING_A_G")
17     assert response.status_code == 200
18     data = response.json()
19     assert data["building"] == "Building A"
20     assert data["floor"] == 0
21     assert data["id"] == "qr-a-0"
22
23 def test_qr_lookup_not_found():
24     response = client.get("/qr/lookup?code=INVALID_QR_CODE")
25     assert response.status_code == 404
26     assert "not recognised" in response.json()["detail"]
27
28 def test_navigate_default_success():
29     response = client.post("/navigate/", json={
30         "origin": "qr-a-0",
31         "destination": "Library",
32         "profile": "Invisible"
33     })
34     assert response.status_code == 200
35     data = response.json()
36     assert "response" in data
37     assert "agent_steps" in data
38     assert "intent" in data
39     assert "profile_applied" in data
40     assert data["profile_applied"] == "Invisible"
```

```
41
42 def test_navigate_wheelchair_success():
43     response = client.post("/navigate/", json={
44         "origin": "qr-a-0",
45         "destination": "Library",
46         "profile": "wheelchair"
47     })
48     assert response.status_code == 200
49     data = response.json()
50     assert data["profile_applied"] == "wheelchair"
51     assert "response" in data
52
53 def test_navigate_emergency_bypass():
54     """Emergency intent should bypass the LLM and return immediately."""
55     response = client.post("/navigate/", json={
56         "origin": "qr-a-0",
57         "destination": "help fire emergency",
58         "profile": "Invisible"
59     })
60     assert response.status_code == 200
61     data = response.json()
62     assert data["intent"] == "emergency"
63     assert len(data["agent_steps"]) == 1
64     assert data["agent_steps"][0]["tool"] == "emergency_classifier"
65
```

**test\_tools.py**

Python | backend/tests/test\_tools.py

113 lines | 2.4 KB

```
1 import sys, pytest
2 from pathlib import Path
3
4 sys.path.append(str(Path(__file__).resolve().parents[2]))
5
6 from backend.agent.tools import (
7     qr_lookup,
8     profile_detect,
9     route_query,
10     flag_obstacle
11 )
12
13 def test_qr_lookup_valid():
14     result = qr_lookup.invoke({"qr_id": "QR_101"})
15     assert "QR_101" in result
16
17
18 def test_qr_lookup_empty():
19     result = qr_lookup.invoke({"qr_id": ""})
20     assert result is not None
21
22
23 def test_profile_detect_default():
24     result = profile_detect.invoke(
25         {"user_input": "Take me to the library"}
26     )
27     assert result == "Invisible"
28
29
30 def test_profile_detect_wheelchair():
31     result = profile_detect.invoke(
32         {"user_input": "I use a wheelchair"}
33     )
34     assert result is not None
35
36
37 def test_profile_detect_visual():
38     result = profile_detect.invoke(
39         {"user_input": "I am visually impaired"}
40     )
41     assert result is not None
42
43
44 def test_route_query_basic():
45     # Use real node IDs from the mock graph
46     result = route_query.invoke(
47         {
48             "start_node": "qr-a-0",
```

```
49     "end_node": "room-a-21",
50     "profile": "Invisible"
51 }
52 )
53 assert result is not None
54 assert len(result) > 0
55
56
57 def test_route_query_wheelchair():
58     result = route_query.invoke(
59     {
60     "start_node": "qr-a-0",
61     "end_node": "room-a-21",
62     "profile": "Wheelchair"
63     }
64     )
65     assert result is not None
66
67
68 def test_route_query_same_location():
69     result = route_query.invoke(
70     {
71     "start_node": "room-a-21",
72     "end_node": "room-a-21",
73     "profile": "Invisible"
74     }
75     )
76     assert result is not None
77
78
79 def test_flag_obstacle():
80     result = flag_obstacle.invoke(
81     {
82     "node_id": "lift-a",
83     "description": "Broken"
84     }
85     )
86     assert "Obstacle recorded" in result
87
88
89 def test_flag_obstacle_message():
90     result = flag_obstacle.invoke(
91     {
92     "node_id": "corr-b-0",
93     "description": "Water leakage"
94     }
95     )
96     assert "Water leakage" in result
97
98
99 def test_route_contains_profile():
100     result = route_query.invoke({
101     "start_node": "qr-a-0",
102     "end_node": "room-a-21",
103     "profile": "Wheelchair"
104     })
105     assert result is not None
106
107
108 def test_flag_obstacle_location():
109     result = flag_obstacle.invoke({
110     "node_id": "lift-b",
111     "description": "Broken Lift"
112     })
113     assert "Broken Lift" in result
```

**run\_tests.py**

Python | backend/tests/run\_tests.py

47 lines | 1.5 KB

```
1 import sys
2 from pathlib import Path
3
4 # Add project root to sys.path
5 project_root = Path(__file__).resolve().parents[2]
6 sys.path.append(str(project_root))
7
8 # Import the test modules
```

```
9  try:
10     from backend.tests import test_tools
11     from backend.tests import test_agent
12     from backend.tests import test_api
13     print("Test modules imported successfully.")
14 except Exception as e:
15     print(f"Error importing test modules: {e}")
16     sys.exit(1)
17
18 def run_tests():
19     modules = [test_tools, test_agent, test_api]
20     passed = 0
21     failed = 0
22
23     for module in modules:
24         print(f"\nRunning tests in {module.__name__}:")
25         # Find all functions starting with "test_"
26         test_funcs = [getattr(module, attr) for attr in dir(module) if attr.startswith("test_") and callable(getattr(module, attr))]
27
28         for func in test_funcs:
29             try:
30                 func()
31                 print(f" ■ {func.__name__} passed")
32                 passed += 1
33             except AssertionError as e:
34                 print(f" ■ {func.__name__} FAILED (AssertionError)")
35                 failed += 1
36             except Exception as e:
37                 print(f" ■ {func.__name__} ERROR: {e}")
38                 failed += 1
39
40         print(f"\n=====")
41         print(f"Summary: {passed} passed, {failed} failed")
42         print(f"=====")
43         if failed > 0:
44             sys.exit(1)
45
46 if __name__ == "__main__":
47     run_tests()
```



## 05 Database

### schema.cypher

Cypher (Neo4j) | db/schema.cypher

27 lines | 0.8 KB

```
1 // CampusPilot AI - Neo4j Schema Definitions
2 // Version: 1.0
3
4 // Constraints
5 CREATE CONSTRAINT UNIQUE_LOCATION_ID IF NOT EXISTS
6 FOR (l:Location) REQUIRE l.id IS UNIQUE;
7
8 CREATE CONSTRAINT UNIQUE_QR_ID IF NOT EXISTS
9 FOR (q:QRPoint) REQUIRE q.id IS UNIQUE;
10
11 // Indexes
12 CREATE INDEX LOCATION_NAME_INDEX IF NOT EXISTS
13 FOR (l:Location) ON (l.name);
14
15 CREATE INDEX QR_CODE_INDEX IF NOT EXISTS
16 FOR (q:QRPoint) ON (q.code);
17
18 // Nodes:
19 // - QRPoint {id, code, name, building, floor, description}
20 // - Room {id, name, building, floor, type}
21 // - Corridor {id, building, floor, name}
22 // - Lift {id, building, name}
23 // - Stairs {id, building, name}
24 // - Entrance {id, building, name}
25
26 // Relationships:
27 // - (:QRPoint)-[:CONNECTS_TO {distance_meters: Float, wheelchair_accessible: Boolean, tactile_paving: Boolean, stairs: Boolean, lift: Boolean}]
```

### seed.cypher

Cypher (Neo4j) | db/seed.cypher

160 lines | 13.5 KB

```
1 // Seed Script for CampusPilot AI Graph Database
2 // Contains 2 buildings (Building A and Building B)
3 // 3 floors per building, 10+ rooms, lifts, stairs, entrances, and QR nodes.
4
5 // Clear existing database contents
6 MATCH (n) DETACH DELETE n;
7
8 // --- BUILDING A ---
9 // Ground Floor (Floor 0)
10 CREATE (entA:Entrance {id: "ent-a-0", building: "Building A", name: "Main Entrance A"})
11 CREATE (qrA0:QRPoint {id: "qr-a-0", code: "QR_BUILDING_A_G", name: "Building A Ground Floor Lobby QR", building: "Building A", floor: 0, name: "Ground Floor Lobby QR"})
12 CREATE (corrA0:Corridor {id: "corr-a-0", building: "Building A", floor: 0, name: "Ground Floor Corridor"})
13 CREATE (roomA01:Room {id: "room-a-01", name: "Seminar Hall 101", building: "Building A", floor: 0, type: "Seminar Hall"})
14 CREATE (roomA02:Room {id: "room-a-02", name: "Admission Office 102", building: "Building A", floor: 0, type: "Office"})
15 CREATE (liftA:Lift {id: "lift-a", building: "Building A", name: "Main Elevator A"})
16 CREATE (stairsA:Stairs {id: "stairs-a", building: "Building A", name: "Staircase A"})
17
18 // First Floor (Floor 1)
19 CREATE (qrA1:QRPoint {id: "qr-a-1", code: "QR_BUILDING_A_1", name: "Building A First Floor QR", building: "Building A", floor: 1, name: "First Floor QR"})
20 CREATE (corrA1:Corridor {id: "corr-a-1", building: "Building A", floor: 1, name: "First Floor Corridor"})
21 CREATE (roomA11:Room {id: "room-a-11", name: "AI Lab 201", building: "Building A", floor: 1, type: "Laboratory"})
22 CREATE (roomA12:Room {id: "room-a-12", name: "Classroom 202", building: "Building A", floor: 1, type: "Classroom"})
23
24 // Second Floor (Floor 2)
25 CREATE (qrA2:QRPoint {id: "qr-a-2", code: "QR_BUILDING_A_2", name: "Building A Second Floor QR", building: "Building A", floor: 2, name: "Second Floor QR"})
26 CREATE (corrA2:Corridor {id: "corr-a-2", building: "Building A", floor: 2, name: "Second Floor Corridor"})
27 CREATE (roomA21:Room {id: "room-a-21", name: "Library", building: "Building A", floor: 2, type: "Library"})
28 CREATE (roomA22:Room {id: "room-a-22", name: "Server Room 302", building: "Building A", floor: 2, type: "Server Room"})
29
30 // --- BUILDING B ---
31 // Ground Floor (Floor 0)
32 CREATE (entB:Entrance {id: "ent-b-0", building: "Building B", name: "Main Entrance B"})
33 CREATE (qrB0:QRPoint {id: "qr-b-0", code: "QR_BUILDING_B_G", name: "Building B Ground Floor Lobby QR", building: "Building B", floor: 0, name: "Ground Floor Lobby QR"})
34 CREATE (corrB0:Corridor {id: "corr-b-0", building: "Building B", floor: 0, name: "Ground Floor Corridor B"})
35 CREATE (roomB01:Room {id: "room-b-01", name: "Auditorium 1", building: "Building B", floor: 0, type: "Auditorium"})
36 CREATE (roomB02:Room {id: "room-b-02", name: "Cafeteria", building: "Building B", floor: 0, type: "Food Court"})
37 CREATE (liftB:Lift {id: "lift-b", building: "Building B", name: "Elevator B"})
38 CREATE (stairsB:Stairs {id: "stairs-b", building: "Building B", name: "Staircase B"})
```

```
39
40 // First Floor (Floor 1)
41 CREATE (qrB1:QRPoint {id: "qr-b-1", code: "QR_BUILDING_B_1", name: "Building B First Floor QR", building: "Building B",
42 CREATE (corrB1:Corridor {id: "corr-b-1", building: "Building B", floor: 1, name: "First Floor Corridor B"})
43 CREATE (roomB11:Room {id: "room-b-11", name: "Physics Lab 201", building: "Building B", floor: 1, type: "Laboratory"})
44 CREATE (roomB12:Room {id: "room-b-12", name: "Classroom 203", building: "Building B", floor: 1, type: "Classroom"})
45
46 // Second Floor (Floor 2)
47 CREATE (qrB2:QRPoint {id: "qr-b-2", code: "QR_BUILDING_B_2", name: "Building B Second Floor QR", building: "Building B",
48 CREATE (corrB2:Corridor {id: "corr-b-2", building: "Building B", floor: 2, name: "Second Floor Corridor B"})
49 CREATE (roomB21:Room {id: "room-b-21", name: "Dean's Office", building: "Building B", floor: 2, type: "Office"})
50 CREATE (roomB22:Room {id: "room-b-22", name: "Conference Room 305", building: "Building B", floor: 2, type: "Meeting
51 Room"})
52 // --- CONNECTIONS WITHIN BUILDING A ---
53 // Ground Floor Connections
54 CREATE (entA)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
55 CREATE (qrA0)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
56 lift: false}]->(entA)
57 CREATE (qrA0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
58 CREATE (corrA0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
59 lift: false}]->(qrA0)
60 CREATE (corrA0)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
61 CREATE (entA)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs:
62 false, lift: false}]->(c...
63 CREATE (corrA0)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
64 CREATE (entA)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
65 lift: false}]->(c...
66 // Vertical Connections (Lift A) - Accessible
67 CREATE (qrA0)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
68 CREATE (entA)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
69 lift: true}]->(qrA0)
70 // Vertical Connections (Stairs A) - Non-Accessible
71 CREATE (corrA0)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
72 CREATE (entA)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
73 lift: false}]->(c...
74 // First Floor Connections
75 CREATE (qrA1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
76 CREATE (corrA1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
77 lift: false}]->(qrA1)
78 CREATE (corrA1)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
79 CREATE (entA)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs:
80 false, lift: false}]->(c...
81 CREATE (corrA1)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
82 CREATE (entA)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
83 lift: false}]->(c...
84 CREATE (qrA1)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
85 CREATE (entA)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
86 lift: true}]->(qrA1)
87 CREATE (corrA1)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
88 CREATE (entA)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
89 lift: false}]->(c...
90 // Second Floor Connections
91 CREATE (qrA2)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
92 CREATE (corrA2)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
93 lift: false}]->(qrA2)
94 CREATE (corrA2)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
95 CREATE (entA)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
96 lift: false}]->(c...
97 CREATE (corrA2)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
98 CREATE (entA)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
99 lift: false}]->(c...
100 CREATE (qrA2)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
101 CREATE (entA)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
102 lift: true}]->(qrA2)
103 CREATE (corrA2)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
104 CREATE (entA)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
105 lift: false}]->(c...
106 // --- CONNECTIONS WITHIN BUILDING B ---
107 // Ground Floor Connections
108 CREATE (entB)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
109 CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
110 lift: false}]->(entB)
111 CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
112 CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
113 lift: false}]->(qrB0)
114 CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
115 CREATE (entB)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs:
116 false, lift: false}]->(c...
```

```
117 CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
118 CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
119 lift: false}]]->(co...
120 CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
121 CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
122 lift: true}]]->(qrB0)
123 CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
124 CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
125 lift: false}]]->(co...
126 // First Floor Connections
127 CREATE (qrB1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
128 CREATE (qrB1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
129 lift: false}]]->(qrB1)
130 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
131 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs:
132 false, lift: false}]]->(c...
133 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
134 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
135 lift: false}]]->(co...
136 CREATE (qrB1)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
137 CREATE (qrB1)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
138 lift: true}]]->(qrB1)
139 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
140 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
141 lift: false}]]->(co...
142 // Second Floor Connections
143 CREATE (qrB2)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
144 CREATE (qrB2)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
145 lift: false}]]->(qrB2)
146 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
147 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
148 lift: false}]]->(co...
149 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
150 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
151 lift: false}]]->(co...
152 CREATE (qrB2)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
153 CREATE (qrB2)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
154 lift: true}]]->(qrB2)
155 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
156 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true,
157 lift: false}]]->(co...
158 // --- INTER-BUILDING PATHS ---
159 CREATE (entA)-[:CONNECTS_TO {distance_meters: 50.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
160 CREATE (entA)-[:CONNECTS_TO {distance_meters: 50.0, wheelchair_accessible: true, tactile_paving: true, stairs: false,
161 lift: false}]]->(entA)
```

## schema\_notes.md

Markdown | db/schema\_notes.md

31 lines | 2.1 KB

```
1 # Neo4j Graph Database Schema Notes
2
3 This document describes the schema design for the CampusPilot AI campus navigation graph.
4
5 ## Graph Modeling Decisions
6 A graph database (Neo4j) is used over relational databases (like PostgreSQL with PostGIS extension) because campus
7 navigation indoors consists...
8 ## Node Types
9
10 - **QRPoint**: Represents a physical QR code checkpoint placed at a corridor junction, lift lobby, or entrance.
11 - Properties: `id` (UUID), `code` (string, the content of the QR code), `name` (string), `building` (string), `floor`
12 (integer), `description` (string, e.g., classrooms, laboratories, faculty offices, libraries, or restrooms).
13 - Properties: `id` (UUID), `name` (string), `building` (string), `floor` (integer), `type` (string).
14 - **Corridor**: Connective hallway segments or hallway intersection points.
15 - Properties: `id` (UUID), `building` (string), `floor` (integer), `name` (string).
16 - **Lift**: Lifts connecting multiple floors.
17 - Properties: `id` (UUID), `building` (string), `name` (string).
18 - **Stairs**: Staircases connecting multiple floors.
19 - Properties: `id` (UUID), `building` (string), `name` (string).
20 - **Entrance**: Building entrances connecting the outdoor space to the indoor graph.
21 - Properties: `id` (UUID), `building` (string), `name` (string).
22
23 ## Relationships
24
25 - **CONNECTS_TO**: Represents a path connecting two physical locations.
26 - Properties:
27 - `distance_meters` (float): Distance in meters between the locations.
28 - `wheelchair_accessible` (boolean): `true` if the route does not contain steps and conforms to ramp slopes.
29 - `tactile_paving` (boolean): `true` if tactile paving is present for visually impaired navigation.
```

```
30 - `stairs` (boolean): `true` if this connection represents traversing stairs.  
31 - `lift` (boolean): `true` if this connection represents traversing a lift.
```

## 06 Frontend — App Shell

### main.tsx

TypeScript/React | frontend/src/main.tsx

11 lines | 0.3 KB

```
1 import {StrictMode} from 'react';
2 import {createRoot} from 'react-dom/client';
3 import App from './App.tsx';
4 import 'leaflet/dist/leaflet.css';
5 import './index.css';
6
7 createRoot(document.getElementById('root')!).render(
8   <StrictMode>
9     <App />
10  </StrictMode>,
11 );
```

### App.tsx

TypeScript/React | frontend/src/App.tsx

1030 lines | 47.0 KB

```
1 /**
2  * @license
3  * SPDX-License-Identifier: Apache-2.0
4  */
5
6 import React, { useState, useEffect, useRef } from 'react';
7 import {
8   Home,
9   Navigation,
10  QRCode,
11  Bookmark,
12  User,
13  Search,
14  Mic,
15  Keyboard,
16  Compass,
17  MapPin,
18  ChevronRight,
19  Info,
20  RotateCcw,
21  Volume2,
22  BookOpen,
23  DownloadCloud,
24  Bot,
25  CheckCircle2
26 } from 'lucide-react';
27
28 import { NavigationMode } from './types';
29 import { PROFILES, MOCK_HOTSPOTS } from './data';
30
31 // Import our cohesive views
32 import SavedMapView from './components/SavedMapView';
33 import ReportIssueView from './components/ReportIssueView';
34 import AccessibilityView from './components/AccessibilityView';
35 import SOSScreen from './components/SOSScreen';
36 import RouteOptionsView from './components/RouteOptionsView';
37 import ActiveNavigationView from './components/ActiveNavigationView';
38 import OutdoorNavigationView from './components/OutdoorNavigationView';
39 import IndoorMapSelector from './components/IndoorMapSelector';
40 import CampusMap from './components/CampusMap';
41 import BottomSheet from './components/BottomSheet';
42 import LaptopLanding from './components/LaptopLanding';
43 import QRScanner from './components/QRScanner';
44 import NavigationChat from './components/NavigationChat';
45 import { BuildingDef, getBuilding, INDOOR_MAPS, BUILDINGS } from './indoorMaps';
46
47 // ■■■ POI icon identifier helper (icons should be rendered via SVG/Lucide components) ■■■■■■■■■■
48
49 function getPoiIcon(tags: Record<string, string>): string {
50   const a = tags.amenity || '';
51   const s = tags.shop || '';
52   const l = tags.leisure || '';
53   const t = tags.tourism || '';
54   const p = tags.place || '';
```

```

55
56 // Return icon identifiers for use with icon rendering system
57 if (p === 'suburb' || p === 'neighbourhood' || p === 'quarter') return 'suburbs';
58 if (p === 'district' || p === 'city_district' || p === 'borough') return 'cityscape';
59 if (p === 'town' || p === 'city') return 'city';
60 if (p === 'village' || p === 'hamlet') return 'village';
61 // Amenities
62 if (a === 'restaurant' || a === 'fast_food' || a === 'food_court') return 'restaurant';
63 if (a === 'cafe') return 'cafe';
64 if (a === 'bar' || a === 'pub') return 'bar';
65 if (a === 'hospital' || a === 'clinic') return 'hospital';
66 if (a === 'pharmacy') return 'pharmacy';
67 if (a === 'bank' || a === 'atm') return 'bank';
68 if (a === 'fuel') return 'fuel';
69 if (a === 'school') return 'school';
70 if (a === 'university' || a === 'college') return 'university';
71 if (a === 'library') return 'library';
72 if (a === 'cinema' || a === 'theatre') return 'theatre';
73 if (a === 'police') return 'police';
74 if (a === 'place_of_worship') return 'worship';
75 if (a === 'marketplace') return 'marketplace';
76 if (a === 'supermarket') return 'supermarket';
77 if (s === 'mall' || s === 'department_store') return 'mall';
78 if (s === 'supermarket' || s === 'convenience') return 'supermarket';
79 if (s === 'clothes') return 'clothing';
80 if (s === 'electronics') return 'electronics';
81 if (l === 'park') return 'park';
82 if (l === 'sports_centre' || l === 'stadium') return 'sports';
83 if (l === 'swimming_pool') return 'pool';
84 if (t === 'hotel' || t === 'hostel') return 'hotel';
85 if (t === 'museum') return 'museum';
86 if (t === 'attraction') return 'attraction';
87 return 'location';
88 }
89
90 function haversineKm(lat1: number, lng1: number, lat2: number, lng2: number): number {
91   const R = 6371;
92   const dLat = (lat2 - lat1) * Math.PI / 180;
93   const dLng = (lng2 - lng1) * Math.PI / 180;
94   const a = Math.sin(dLat / 2) ** 2
95     + Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI / 180) * Math.sin(dLng / 2) ** 2;
96   return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
97 }
98
99 export default function App() {
100   const [isLaptop, setIsLaptop] = useState(window.innerWidth >= 1024);
101
102   useEffect(() => {
103     const handleResize = () => {
104       setIsLaptop(window.innerWidth >= 1024);
105     };
106     window.addEventListener('resize', handleResize);
107     return () => window.removeEventListener('resize', handleResize);
108   }, []);
109
110   // Navigation & configuration states
111   const [activeTab, setActiveTab] = useState<'home' | 'routes' | 'scan' | 'saved' | 'access'>('home');
112   const [accessibilityMode, setAccessibilityMode] = useState<NavigationMode>('wheelchair');
113   const [sosActive, setSosActive] = useState<boolean>(false);
114   const [isQrScanning, setIsQrScanning] = useState(false);
115
116   // Search & dynamic route states
117   const [searchQuery, setSearchQuery] = useState('');
118   const [selectedHotspot, setSelectedHotspot] = useState<string | null>(null);
119   const [isActiveNavigation, setIsActiveNavigation] = useState(false);
120   const [isIndoorNav, setIsIndoorNav] = useState(true);
121   const [activeRouteConfig, setActiveRouteConfig] = useState<{ origin: string; destination: string } | null>(null);
122
123   // Indoor map selector states
124   const [showIndoorMapSelector, setShowIndoorMapSelector] = useState(false);
125   const [selectedBuildingId, setSelectedBuildingId] = useState<string>('building-1-tech');
126
127   // Helper to get coordinates
128   const getCoordsForName = (name: string): [number, number] | undefined => {
129     const spot = MOCK_HOTSPOTS.find(h => h.name.toLowerCase() === name.toLowerCase());
130     if (spot && spot.lat && spot.lng) {
131       return [spot.lat, spot.lng];
132     }
133   }

```

```

133   return undefined;
134   };
135
136   // Custom destination pin drop and user location tracking
137   const [userLocation, setUserLocation] = useState<[number, number] | null>(null);
138   const [customPinCoords, setCustomPinCoords] = useState<[number, number] | null>(null);
139
140   const activeOriginCoords = activeRouteConfig?.origin === 'Current Location'
141   ? (userLocation || [19.1334, 72.9133])
142   : (activeRouteConfig ? getCoordsForName(activeRouteConfig.origin) : undefined);
143
144   const activeDestCoords = activeRouteConfig?.destination === 'Dropped Pin'
145   ? (customPinCoords || undefined)
146   : (activeRouteConfig ? getCoordsForName(activeRouteConfig.destination) : undefined);
147
148   // Issues Form triggers
149   const [prefilledIssueLocation, setPrefilledIssueLocation] = useState<string | null>(null);
150   const [activeSubTab, setActiveSubTab] = useState<'view' | 'report'>('view');
151   const [incidentSubmittedToast, setIncidentSubmittedToast] = useState(false);
152
153
154   // QR Scanning & AI Chat States
155   const [scanSuccessCard, setScanSuccessCard] = useState(false);
156   const [scanSuccessData, setScanSuccessData] = useState<any>(null);
157   const [isChatOpen, setIsChatOpen] = useState(false);
158
159   // Sound cues simulation toggle
160   const [audioCuesEnabled, setAudioCuesEnabled] = useState(true);
161
162   // Outdoor navigation system (Google Maps-style full experience)
163   const [showOutdoorNav, setShowOutdoorNav] = useState(false);
164   const [outdoorNavInitialDest, setOutdoorNavInitialDest] = useState<{
165     lat: number; lng: number; name: string;
166   } | null>(null);
167
168   // Nearby places (fetched from Overpass API when user location is known)
169   const [nearbyPlaces, setNearbyPlaces] = useState<Array<{
170     id: string; name: string; icon: string; lat: number; lng: number; distKm?: number;
171   }>>([]);
172   const [isFetchingNearby, setIsFetchingNearby] = useState(false);
173   const nearbyFetchedRef = useRef(false);
174
175   // Fetch nearby POIs on mount from Overpass API
176   useEffect(() => {
177     if (nearbyFetchedRef.current) return;
178     nearbyFetchedRef.current = true;
179     setIsFetchingNearby(true);
180
181     const doFetch = (lat: number, lng: number) => {
182       // Query 1: specific venues within 2 km
183       const poiQuery = [
184         '[out:json][timeout:15];',
185         '(',
186         `node["name"]["amenity"~"restaurant|cafe|bar|bank|hospital|pharmacy|fuel|supermarket|school|university|library|cinema|fast_food|pub|nightclub"]["shop"~"mall|supermarket|convenience|department_store|clothes|electronics"](around:2000,${lat},${lng});`,
187         `node["name"]["leisure"~"park|sports_centre|stadium|swimming_pool"](around:1500,${lat},${lng});`,
188         `node["name"]["tourism"~"hotel|museum|attraction|hostel"](around:2000,${lat},${lng});`,
189         ');',
190         'out body 20;',
191       ].join('');
192
193       // Query 2: localities / suburbs / neighbourhoods within 8 km
194       const localityQuery = [
195         '[out:json][timeout:15];',
196         '(',
197         `node["name"]["place"~"suburb|neighbourhood|quarter|district|city_district|borough|town|village"](around:8000,${lat},${lng});`,
198         ');',
199         'out body 20;',
200       ].join('');
201
202       const fetchOverpass = (q: string) =>
203       fetch('https://overpass-api.de/api/interpreter', {
204         method: 'POST',
205         body: `data=${encodeURIComponent(q)}`,
206         headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
207       }).then(r => r.json());
208
209       Promise.allSettled([fetchOverpass(poiQuery), fetchOverpass(localityQuery)])

```

```

211 .then([poiResult, localityResult]) => {
212   const poiEls = poiResult.status === 'fulfilled' ? (poiResult.value.elements || []) : [];
213   const localityEls = localityResult.status === 'fulfilled' ? (localityResult.value.elements || []) : [];
214
215   const toPlace = (el: any, prefix: string) => ({
216     id: `${prefix}-${el.id}`,
217     name: el.tags.name as string,
218     icon: getPoiIcon(el.tags),
219     lat: el.lat as number,
220     lng: el.lon as number,
221     distKm: haversineKm(lat, lng, el.lat, el.lon),
222   });
223
224   const pois = poiEls
225     .filter((el: any) => el.tags?.name && el.lat && el.lon)
226     .map((el: any) => toPlace(el, 'poi'));
227
228   const localities = localityEls
229     .filter((el: any) => el.tags?.name && el.lat && el.lon)
230     .map((el: any) => toPlace(el, 'loc'));
231
232   // Localities sorted first, then POIs; deduplicate by name
233   const seen = new Set<string>();
234   const merged = [...localities, ...pois]
235     .sort((a, b) => a.distKm - b.distKm)
236     .filter(p => {
237       const key = p.name.toLowerCase();
238       if (seen.has(key)) return false;
239       seen.add(key);
240       return true;
241     })
242     .slice(0, 12);
243
244   setNearbyPlaces(merged);
245   })
246   .catch(() => { /* silently keep fallback chips */ })
247   .finally(() => setIsFetchingNearby(false));
248 };
249
250 // Ask GPS directly – no reliance on CampusMap callback
251 if (navigator.geolocation) {
252   navigator.geolocation.getCurrentPosition(
253     (pos) => doFetch(pos.coords.latitude, pos.coords.longitude),
254     () => doFetch(19.1334, 72.9133), // fallback: IIT Bombay
255     { enableHighAccuracy: false, timeout: 8000, maximumAge: 60000 }
256   );
257 } else {
258   doFetch(19.1334, 72.9133); // Fallback coordinates
259 }
260 }, []);
261
262
263 // Triggering SOS Action
264 const triggerSos = () => {
265   setSosActive(true);
266 };
267
268 // Switch to Route configuration Options
269 const handleSelectHotspot = (hotspotId: string) => {
270   const matchedHotspot = MOCK_HOTSPOTS.find(h => h.id === hotspotId);
271   if (!matchedHotspot) return;
272
273   setActiveRouteConfig({
274     origin: 'Main Gate',
275     destination: matchedHotspot.name
276   });
277   setActiveTab('routes');
278 };
279
280 const handleStartNavigation = () => {
281   setIsActiveNavigation(true);
282 };
283
284 // Handle reporting directly from navigation corridors
285 const handleReportObstacleFromNav = (fullLocationPath: string) => {
286   setPrefilledIssueLocation(fullLocationPath);
287   setActiveSubTab('report');
288   setActiveTab('access'); // Navigate to reporting sub-page under Access

```



```

289   };
290
291
292   const activeProfile = PROFILES.find(p => p.id === accessibilityMode) || PROFILES[0];
293
294   if (isLaptop) {
295     return <LaptopLanding />;
296   }
297
298   return (
299     <div className="min-h-screen flex flex-col lg:flex-row antialiased select-none font-sans bg-zinc-50 text-zinc-900">
300
301       {/* SOS Alert Fullscreen Trigger */}
302       {sosActive && (
303         <SOSScreen
304           onCancel={() => setSosActive(false)}
305           lastKnownLocation="Engineering Block A – Ground Floor Entrance"
306         />
307       )}
308
309       {/* Outdoor Navigation – full Google Maps-style experience */}
310       {showOutdoorNav && (
311         <OutdoorNavigationView
312           onBack={() => {
313             setShowOutdoorNav(false);
314             setOutdoorNavInitialDest(null);
315             setCustomPinCoords(null);
316           }}
317           onSosClick={triggerSos}
318           initialDestination={outdoorNavInitialDest ?? undefined}
319         />
320       )}
321
322       {/* Indoor Map Selector – full-screen building picker */}
323       {showIndoorMapSelector && (
324         <IndoorMapSelector
325           onBack={() => setShowIndoorMapSelector(false)}
326           onSelectMap={(buildingId) => {
327             setSelectedBuildingId(buildingId);
328             setIsIndoorNav(true);
329             setIsActiveNavigation(true);
330             setShowIndoorMapSelector(false);
331           }}
332         />
333       )}
334
335       {/* Left Sidebar Layout for Desktop Web users (lg:flex) */}
336       <aside className="hidden lg:flex flex-col fixed left-0 top-0 h-screen w-64 bg-zinc-900 text-white p-6 z-40 border-r
337 border-zinc-800" style={{flex: 1, items: 'center', gap: 'xs', mb: '8'}}>
338         <Compass className="w-8 h-8 text-teal-400 animate-spin-slow rotate-45" />
339         <div className="flex flex-col">
340           <h1 className="text-xl font-bold font-sans tracking-wide">CampusPilot AI</h1>
341           <span className="text-[10px] text-teal-400 font-mono tracking-widest uppercase">Accessibility-First</span>
342         </div>
343       </div>
344
345       {/* Desktop Side Navigation Buttons */}
346       <nav className="flex flex-col gap-xs flex-1">
347         <button
348           onClick={() => {
349             setActiveTab('home');
350             setIsActiveNavigation(false);
351             setActiveRouteConfig(null);
352           }}
353           className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-sm flex items-center gap-sm transition-all text-left ${
354             activeTab === 'home' && !isActiveNavigation && !activeRouteConfig
355               ? 'bg-[#60f8cb] text-stone-950 shadow-md'
356               : 'text-zinc-400 hover:text-white hover:bg-zinc-800'
357             }`>
358           >
359           <Home className="w-5 h-5" />
360           Home Main
361         </button>
362
363         <button
364           onClick={() => {
365             setActiveTab('routes');
366             setIsActiveNavigation(false);

```

```

367   if (!activeRouteConfig) {
368     setActiveRouteConfig({ origin: 'Library', destination: 'Science Hall' });
369   }
370 }
371 className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-sm flex items-center gap-sm transition-all text-left ${
372   activeTab === 'routes' || activeRouteConfig
373     ? 'bg-[#60f8cb] text-stone-950 shadow-md'
374     : 'text-zinc-400 hover:text-white hover:bg-zinc-800'
375   }`}
376 >
377 <Navigation className="w-5 h-5 rotate-45" />
378 Route Options
379 </button>
380
381 <button
382   onClick={() => {
383     setActiveTab('scan');
384     setIsActiveNavigation(false);
385     setActiveRouteConfig(null);
386   }}
387   className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-sm flex items-center gap-sm transition-all text-left ${
388     activeTab === 'scan'
389       ? 'bg-[#60f8cb] text-stone-950 shadow-md'
390       : 'text-zinc-400 hover:text-white hover:bg-zinc-800'
391     }`}
392 >
393 <QRCode className="w-5 h-5" />
394 Scan Checkpoint
395 </button>
396
397 <button
398   onClick={() => {
399     setActiveTab('saved');
400     setIsActiveNavigation(false);
401     setActiveRouteConfig(null);
402   }}
403   className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-sm flex items-center gap-sm transition-all text-left ${
404     activeTab === 'saved'
405       ? 'bg-[#60f8cb] text-stone-950 shadow-md'
406       : 'text-zinc-400 hover:text-white hover:bg-zinc-800'
407     }`}
408 >
409 <Bookmark className="w-5 h-5" />
410 Saved Maps
411 </button>
412
413 <button
414   onClick={() => {
415     setActiveTab('access');
416     setActiveSubTab('view');
417     setIsActiveNavigation(false);
418     setActiveRouteConfig(null);
419   }}
420   className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-sm flex items-center gap-sm transition-all text-left ${
421     activeTab === 'access'
422       ? 'bg-[#60f8cb] text-stone-950 shadow-md'
423       : 'text-zinc-400 hover:text-white hover:bg-zinc-800'
424     }`}
425 >
426 <User className="w-5 h-5" />
427 Preferences & Issues
428 </button>
429 </nav>
430
431 {/* Desktop Active Preferences State Summary */}
432 <div className="bg-zinc-850 p-4 rounded-2xl border border-zinc-800/80 mb-6 flex flex-col gap-sm">
433   <div className="flex items-center gap-xs">
434     <span className="text-xs font-extrabold tracking-wider uppercase text-[#60f8cb]">
435       {activeProfile.name} Mode Active
436     </span>
437   </div>
438   <p className="text-[11px] text-zinc-400 italic">
439     "{activeProfile.description}"
440   </p>
441   <div className="flex items-center justify-end border-t border-zinc-800 pt-sm mt-xs">
442     <button
443       onClick={() => setAudioCuesEnabled(prev => !prev)}
444       className={`p-1.5 hover:bg-zinc-800 rounded-lg transition-colors ${

```

```

445 audioCuesEnabled ? 'text-teal-400' : 'text-zinc-400'
446 }'}
447 title="Toggle Sound Beacons"
448 >
449 <Volume2 className="w-4 h-4" />
450 </button>
451 </div>
452 </div>
453
454 {/ * Big Dangerous SOS trigger */}
455 <button
456   onClick={triggerSos}
457   className="w-full py-3.5 bg-red-650 hover:bg-red-700 text-white font-sans font-black tracking-widest text-xs uppercase
458   rounded-xl tr...
459   Emergency SOS Action
460 </button>
461 </aside>
462
463 <div className="flex-grow flex flex-col lg:pl-64 min-h-screen">
464
465 {/ * Top AppBar for Mobile views (Hidden on Desktop) */}
466 <header className="flex lg:hidden justify-between items-center w-full px-6 h-16 border-b border-zinc-200 bg-white z-40
467   shadow-sm">
468   <div className="flex items-center gap-2">
469     <Compass className="w-5 h-5 text-[#002f5c] animate-spin-slow rotate-45" />
470     <span className="font-bold font-sans text-base tracking-wide text-zinc-900">
471       CampusPilot AI
472     </span>
473   </div>
474   <span className="text-xs font-semibold text-zinc-400 tracking-wider">
475     {activeProfile.name}
476   </span>
477 </header>
478
479 {/ * View Switcher content */}
480 <div className="flex-grow">
481   {isActiveNavigation ? (
482     {/ * ACTIVE: SVG blueprint dynamic floor walk navigation */}
483     <ActiveNavigationView
484       onBack={() => {
485         setIsActiveNavigation(false);
486         if (activeRouteConfig?.destination === 'Dropped Pin') {
487           setActiveRouteConfig(null);
488           setActiveTab('home');
489         } else {
490           setActiveTab('saved');
491         }
492       }}
493       onStopNav={() => {
494         setIsActiveNavigation(false);
495         if (activeRouteConfig?.destination === 'Dropped Pin') {
496           setActiveRouteConfig(null);
497           setActiveTab('home');
498         }
499       }}
500       onReportObstacle={handleReportObstacleFromNav}
501       onSosClick={triggerSos}
502       buildingName={isIndoorNav ? (BUILDINGS.find(m => m.id === selectedBuildingId)?.name || "Engineering & Tech Building") :
503         (isIndoorNav ? "Engineering & Tech Building" : "Engineering & Tech Building")}
504       themeMode="light"
505       routeOriginCoords={activeOriginCoords}
506       routeDestinationCoords={activeDestCoords}
507       selectedMapId={selectedBuildingId}
508     />
509   ) : activeRouteConfig ? (
510     {/ * ACTIVE 2: Multi-alternative Route selection container */}
511     <RouteOptionsView
512       onBack={() => {
513         setActiveRouteConfig(null);
514         setActiveTab('home');
515       }}
516       onStartRoute={(id) => {
517         if (activeRouteConfig && activeRouteConfig.origin === 'Main Gate') {
518           setIsIndoorNav(false);
519         } else {
520           setIsIndoorNav(true);
521         }
522         setActiveNavigation(true);
523       }}

```

```

523   originName={activeRouteConfig.origin}
524   destName={activeRouteConfig.destination}
525   currentProfileMode={accessibilityMode}
526   themeMode="light"
527   />
528   ) : (
529     <>
530     { /* TAB 1: Searchable Interactive Map Home Dashboard */ }
531     { activeTab === 'home' && (
532       <div className="relative w-full h-[calc(100vh-76px)] overflow-hidden flex flex-col">
533         { /* Live Leaflet map - fills the full area */ }
534         <div className="absolute inset-0 z-0">
535           <CampusMap
536             className="w-full h-full"
537             onMapClick={({ coords }) => {
538               setCustomPinCoords(coords);
539               setSelectedHotspot(null); // Clear hotspot highlight
540             }}
541             onUserLocationChange={({ coords }) => setUserLocation(coords)}
542             customPinCoords={customPinCoords}
543           />
544         </div>
545
546         { /* Top-right audio toggle & AI Agent Chat toggle */ }
547         <div className="absolute right-6 top-6 flex flex-col gap-3 z-10">
548           <button
549             onClick={() => setAudioCuesEnabled(p => !p)}
550             className={`h-12 w-12 rounded-full shadow-lg border border-zinc-200 flex items-center justify-center cursor-pointer
551             ${audioCuesEnabled ? 'bg-teal-50 text-teal-600' : 'bg-white text-zinc-500'}
552             `}
553             title={audioCuesEnabled ? 'Audio Beacon Cues Engaged' : 'Audio Cues Muted'}
554             >
555             <Volume2 className="w-5 h-5" />
556           </button>
557           <button
558             onClick={() => setIsChatOpen(true)}
559             className="h-12 w-12 rounded-full shadow-lg border border-zinc-200 bg-white text-teal-600 flex items-center
560             justify"
561             title="Chat with AI Navigation Agent"
562             >
563             <Bot className="w-5 h-5" />
564           </button>
565         </div>
566
567         { /* Draggable Bottom Sheet or Dropped Pin overlay */ }
568         <div className="absolute bottom-0 left-0 right-0 z-20">
569           { customPinCoords ? (
570             <div className="bg-white rounded-t-3xl border-t border-zinc-200/80 shadow-2xl p-6 relative">
571               <div className="max-w-xl mx-auto w-full flex flex-col gap-4">
572                 <div className="w-12 h-1.5 bg-zinc-200 rounded-full mx-auto mb-2" />
573
574                 <div className="flex items-start gap-4">
575                   <div className="w-12 h-12 rounded-2xl bg-red-50 text-red-500 flex items-center justify-center shrink-0 border b...
576                 </div>
577                   <div className="flex-1 text-left">
578                     <h3 className="font-extrabold text-[#002f5c] text-base">Dropped Pin</h3>
579                     <p className="text-xs font-mono text-zinc-550 mt-1 font-semibold">
580                       { customPinCoords[0].toFixed(6)}, { customPinCoords[1].toFixed(6)}
581                     </p>
582                   </div>
583                 </div>
584
585                 <div className="flex gap-3 mt-2">
586                   <button
587                     onClick={() => {
588                       if (customPinCoords) {
589                         setOutdoorNavInitialDest({
590                           lat: customPinCoords[0],
591                           lng: customPinCoords[1],
592                           name: `${customPinCoords[0].toFixed(5)}, ${customPinCoords[1].toFixed(5)}`
593                         });
594                       }
595                       setCustomPinCoords(null);
596                       setShowOutdoorNav(true);
597                     }}
598                     className="flex-1 flex items-center justify-center gap-2 py-3.5 px-4 rounded-xl font-sans font-bold text-sm text...
599                   >
600                   <Navigation className="w-4 h-4 rotate-45" />

```

```

601 Get Directions
602 </button>
603 <button
604   onClick={() => setCustomPinCoords(null)}
605   className="flex-1 flex items-center justify-center gap-2 py-3.5 px-4 rounded-xl font-sans font-bold text-sm bord...
606   >
607   Clear Pin
608 </button>
609 </div>
610 </div>
611 </div>
612 ) : (
613 <BottomSheet defaultSnap="mid">
614   <div className="max-w-xl mx-auto w-full flex flex-col gap-4">
615
616     {/* Quick Action Buttons */}
617     <div className="flex gap-3">
618       <button
619         onClick={() => {
620           setOutdoorNavInitialDest(null);
621           setShowOutdoorNav(true);
622         }}
623         className="flex-1 flex items-center justify-center gap-2 py-3.5 px-4 rounded-xl font-sans font-bold text-sm shad...
624       >
625       <Navigation className="w-4 h-4 rotate-45" />
626       Outdoor Route
627     </button>
628     <button
629       onClick={() => {
630         setActiveRouteConfig({ origin: 'Ground Floor Lobby', destination: 'Engineering Block A' });
631         setActiveTab('routes');
632       }}
633       className="flex-1 flex items-center justify-center gap-2 py-3.5 px-4 rounded-xl font-sans font-bold text-sm shad...
634     >
635     <BookOpen className="w-4 h-4" />
636     Indoor Nav
637   </button>
638 </div>
639
640   {/* Nearby Places (dynamic from Overpass/OSM) */}
641   <div>
642     <div className="flex items-center justify-between mb-2">
643       <h3 className="text-xs font-extrabold text-zinc-500 tracking-wider">
644         {nearbyPlaces.length > 0 ? 'Nearby Places' : 'Quick Destinations'}
645       </h3>
646       {isFetchingNearby && (
647         <div className="flex items-center gap-1.5">
648           <div className="w-3 h-3 border-2 border-zinc-200 border-t-teal-500 rounded-full animate-spin" />
649           <span className="text-[10px] font-bold text-zinc-400">Finding nearby...</span>
650         </div>
651       )}
652     </div>
653
654     <div className="flex gap-2 overflow-x-auto no-scrollbar pb-1">
655       {/* Loading skeletons */}
656       {isFetchingNearby && nearbyPlaces.length === 0 && (
657         [1, 2, 3, 4].map(i => (
658           <div
659             key={i}
660             className="flex-shrink-0 h-8 rounded-full bg-zinc-100 animate-pulse"
661             style={{ width: `${60 + i * 12}px` }}
662           />
663         ))
664       )}
665
666       {/* Real nearby POIs */}
667       {nearbyPlaces.map(place => (
668         <button
669           key={place.id}
670           onClick={() => {
671             setOutdoorNavInitialDest({
672               lat: place.lat,
673               lng: place.lng,
674               name: place.name
675             });
676             setShowOutdoorNav(true);
677           }}
678           className="flex-shrink-0 flex items-center gap-1.5 bg-zinc-100 border border-zinc-200 rounded-full py-2 px-3...

```

```

679 >
680 <span className="text-sm">{place.icon}</span>
681 <span className="group-hover:text-teal-700 transition-colors">{place.name}</span>
682 {place.distKm !== undefined && (
683 <span className="text-[10px] font-semibold text-zinc-400 group-hover:text-teal-500">
684 {place.distKm < 1
685 ? `${Math.round(place.distKm * 1000)}m`
686 : `${place.distKm.toFixed(1)}km`}
687 </span>
688 )}
689 </button>
690 )})
691
692 {/* Fallback chips when no location / fetch failed */}
693 {!isFetchingNearby && nearbyPlaces.length === 0 && [
694 { id: 'dorm', name: 'Dormitory' },
695 { id: 'lib', name: 'Main Library' },
696 { id: 'science', name: 'Science Lab' },
697 { id: 'cafe', name: 'Cafeteria' }
698 ].map(spot => (
699 <button
700 key={spot.id}
701 onClick={() => {
702   setOutdoorNavInitialDest(null);
703   setShowOutdoorNav(true);
704 }}
705 className="flex-shrink-0 flex items-center gap-1.5 bg-zinc-100 border border-zinc-200 rounded-full py-2 px-3...
706 >
707 {spot.name}
708 </button>
709 )})
710 </div>
711 </div>
712
713 {/* Recent Routes */}
714 <div>
715 <h3 className="text-sm font-bold text-zinc-800 mb-3">Recent Routes</h3>
716 <div className="space-y-2">
717 <div
718   onClick={() => { setActiveRouteConfig({ origin: 'Main Gate', destination: 'Library' }); setActiveTab('route...
719   className="flex items-center justify-between p-3 bg-white border border-zinc-200 rounded-xl cursor-pointer hov...
720 >
721 <div className="flex items-center gap-3">
722 <div className="w-10 h-10 rounded-full bg-indigo-50 flex items-center justify-center text-indigo-600 shri...
723 <RotateCcw className="w-4 h-4" />
724 </div>
725 <div className="text-left">
726 <p className="font-sans font-bold text-sm text-zinc-900">Main Gate to Library</p>
727 <p className="text-xs text-zinc-500 font-semibold">Outdoor • 5 mins</p>
728 </div>
729 </div>
730 <ChevronRight className="w-4 h-4 text-zinc-400" />
731 </div>
732 <div
733   onClick={() => { setActiveRouteConfig({ origin: 'Science Lab 304', destination: 'Engineering Block A' }); s...
734   className="flex items-center justify-between p-3 bg-white border border-zinc-200 rounded-xl cursor-pointer hov...
735 >
736 <div className="flex items-center gap-3">
737 <div className="w-10 h-10 rounded-full bg-indigo-50 flex items-center justify-center text-indigo-600 shri...
738 <RotateCcw className="w-4 h-4" />
739 </div>
740 <div className="text-left">
741 <p className="font-sans font-bold text-sm text-zinc-900">Science Lab 304</p>
742 <p className="text-xs text-zinc-500 font-semibold">Indoor • Building B</p>
743 </div>
744 </div>
745 <ChevronRight className="w-4 h-4 text-zinc-400" />
746 </div>
747 </div>
748 </div>
749 </div>
750 </BottomSheet>
751 )}
752 </div>
753 </div>
754 )}
755
756 {/* TAB 2: Direct alternative Route Options Configuration – show default route if none selected */}

```

```

757 {activeTab === 'routes' && !activeRouteConfig && (
758   <RouteOptionsView
759     onBack={() => {
760       setActiveTab('home');
761     }}
762     onStartRoute={(id) => {
763       setIsIndoorNav(true);
764       setIsActiveNavigation(true);
765     }}
766     originName="Library"
767     destName="Science Hall"
768     currentProfileMode={accessibilityMode}
769     themeMode="light"
770   />
771 )}
772
773 {/* TAB 3: QR Checkout Scanner view */}
774 {activeTab === 'scan' && (
775   <div className="relative w-full h-[calc(100vh-76px)] bg-black/90 overflow-hidden">
776     <QRScanner
777       onScanSuccess={(data) => {
778         setScanSuccessData(data);
779         setScanSuccessCard(true);
780       }}
781       onCancel={() => {
782         setActiveTab('home');
783       }}
784     />
785
786     {/* QR Alignment Success Dialog Box popover */}
787     {scanSuccessCard && (
788       <div className="absolute inset-0 bg-black/80 backdrop-blur-md z-50 flex items-end justify-center select-none">
789         <div className="w-full max-w-md bg-white rounded-t-3xl pt-4 pb-12 px-6 shadow-2xl relative">
790           <div className="w-12 h-1.5 bg-zinc-200 rounded-full mx-auto mb-6" />
791
792           <div className="flex flex-col items-center text-center">
793             <div className="w-14 h-14 bg-emerald-50 text-emerald-600 rounded-full flex items-center justify-center mb-4">
794               <CheckCircle2 className="w-7 h-7" />
795             </div>
796             <h2 className="text-lg font-extrabold text-zinc-900 mb-1">
797               Location Identified
798             </h2>
799             <div className="bg-zinc-100 rounded-2xl p-4 w-full flex items-center gap-3 border border-zinc-200 mb-6">
800               <MapPin className="w-5 h-5 text-teal-500 shrink-0" />
801               <p className="text-xs font-semibold text-zinc-700 text-left leading-normal">
802                 {scanSuccessData?.building || 'Engineering Block A'}<br />
803                 <span className="text-[11px] text-zinc-400 font-normal">{scanSuccessData?.locationName || 'Entrance Ground...
804               </p>
805             </div>
806
807             <button
808               onClick={() => {
809                 setScanSuccessCard(false);
810                 setIsIndoorNav(true);
811                 setSelectedBuildingId('building-1-tech');
812                 setIsActiveNavigation(true);
813               }}
814               className="w-full bg-[#10b981] hover:bg-[#059669] text-white font-bold h-12 rounded-xl flex items-center justify-c...
815             >
816               <Navigation className="w-4 h-4 rotate-45" />
817               Start Indoor Guidance
818             </button>
819
820             <button
821               onClick={() => {
822                 setScanSuccessCard(false);
823                 setScanSuccessData(null);
824               }}
825               className="w-full p-2 text-xs text-zinc-550 hover:text-zinc-800 mt-2 font-bold"
826             >
827               Scan another checkpoint
828             </button>
829           </div>
830         </div>
831       </div>
832     </div>
833   )}
834

```

```

835  /* TAB 4: Saved Maps offline database list */
836  {activeTab === 'saved' && (
837    <SavedMapView
838    onScanClick={() => setActiveTab('scan')}
839    onNavigateToBlockA={() => setShowIndoorMapSelector(true)}
840    />
841  )}
842
843  /* TAB 5: Access profile custom selection & active incidents list */
844  {activeTab === 'access' && (
845    <div className="w-full">
846      <div className="flex border-b border-zinc-200 bg-white sticky top-0 z-10 justify-around p-2 select-none">
847        <button
848          onClick={() => setActiveSubTab('view')}
849          className={`text-sm font-bold tracking-wide py-2 px-6 rounded-full transition-all cursor-pointer ${
850            activeSubTab === 'view'
851              ? 'bg-[#002f5c] text-white'
852              : 'text-zinc-500 hover:text-zinc-700'
853            }`}
854          >
855          Navigation Modes
856        </button>
857        <button
858          onClick={() => setActiveSubTab('report')}
859          className={`text-sm font-bold tracking-wide py-2 px-6 rounded-full transition-all cursor-pointer ${
860            activeSubTab === 'report'
861              ? 'bg-[#002f5c] text-white'
862              : 'text-zinc-500 hover:text-zinc-700'
863            }`}
864          >
865          Report Incidents
866        </button>
867      </div>
868
869      {activeSubTab === 'view' ? (
870        <AccessibilityView
871        currentMode={accessibilityMode}
872        onSaveMode={(mode) => setAccessibilityMode(mode)}
873        onClose={() => setActiveTab('home')}
874        />
875      ) : (
876        <ReportIssueView
877        initialLocation={prefilledIssueLocation || undefined}
878        onSubmitSuccess={() => {
879          setPrefilledIssueLocation(null);
880          setActiveSubTab('view');
881          setIncidentSubmittedToast(true);
882          setTimeout(() => setIncidentSubmittedToast(false), 4000);
883        }}
884        />
885      )}
886    </div>
887  )}
888 </>
889 )}
890 </div>
891
892  /* Global Floating Access Emergency SOS Red Trigger Button on Bottom bar of Map Home screen */
893  {activeTab === 'home' && !isActiveNavigation && !activeRouteConfig && (
894    <button
895      onClick={triggerSos}
896      aria-label="Emergency SOS"
897      className="fixed bottom-[96px] right-6 z-40 w-12 h-12 rounded-full bg-red-600 border-[3px] border-white shadow-xl flex
898      items-center...
899      <span className="text-white text-[12px] font-black tracking-widest leading-none select-none">SOS</span>
900    </button>
901  )}
902
903  /* Bottom Navigation Bar for Mobile devices (Hidden on Desktop Web view) */
904  {!isActiveNavigation && (
905    <nav className="flex lg:hidden bg-[#f0f0f8] border-t border-zinc-200 fixed bottom-0 left-0 w-full z-[45] h-[76px]
906    justify-around ...
907    <button
908      onClick={() => {
909        setActiveTab('home');
910        setIsActiveNavigation(false);
911        setActiveRouteConfig(null);
912      }}

```



```
913   className={`transition-all duration-200 flex flex-col items-center justify-center h-[56px] w-[72px] rounded-2xl ${
914     activeTab === 'home' && !isActiveNavigation && !activeRouteConfig
915     ? 'bg-[#60f8cb] text-zinc-900 font-bold shadow-sm'
916     : 'text-zinc-500 hover:text-zinc-800'
917   }}
918   >
919   <Home className="w-5 h-5 mb-0.5" />
920   <span className="text-[10px] font-sans font-bold">Home</span>
921 </button>
922
923   {/* Routes */}
924   <button
925     onClick={() => {
926       setActiveTab('routes');
927       setIsActiveNavigation(false);
928       setActiveRouteConfig(null);
929     }}
930     className={`transition-all duration-200 flex flex-col items-center justify-center h-[56px] w-[72px] rounded-2xl ${
931       activeTab === 'routes' || activeRouteConfig
932       ? 'bg-[#60f8cb] text-zinc-900 font-bold shadow-sm'
933       : 'text-zinc-500 hover:text-zinc-800'
934     }}
935     >
936     <Navigation className="w-5 h-5 rotate-45 mb-0.5" />
937     <span className="text-[10px] font-sans font-bold">Routes</span>
938   </button>
939
940   {/* Scan */}
941   <button
942     onClick={() => {
943       setActiveTab('scan');
944       setIsActiveNavigation(false);
945       setActiveRouteConfig(null);
946     }}
947     className={`transition-all duration-200 flex flex-col items-center justify-center h-[56px] w-[72px] rounded-2xl ${
948       activeTab === 'scan'
949       ? 'bg-[#60f8cb] text-zinc-900 font-bold shadow-sm'
950       : 'text-zinc-500 hover:text-zinc-800'
951     }}
952     >
953     <QRCode className="w-5 h-5 mb-0.5" />
954     <span className="text-[10px] font-sans font-bold">Scan QR</span>
955   </button>
956
957   {/* Saved */}
958   <button
959     onClick={() => {
960       setActiveTab('saved');
961       setIsActiveNavigation(false);
962       setActiveRouteConfig(null);
963     }}
964     className={`transition-all duration-200 flex flex-col items-center justify-center h-[56px] w-[72px] rounded-2xl ${
965       activeTab === 'saved'
966       ? 'bg-[#60f8cb] text-zinc-900 font-bold shadow-sm'
967       : 'text-zinc-500 hover:text-zinc-800'
968     }}
969     >
970     <Bookmark className="w-5 h-5 mb-0.5" />
971     <span className="text-[10px] font-sans font-bold">Saved</span>
972   </button>
973
974   {/* Access */}
975   <button
976     onClick={() => {
977       setActiveTab('access');
978       setActiveSubTab('view');
979       setIsActiveNavigation(false);
980       setActiveRouteConfig(null);
981     }}
982     className={`transition-all duration-200 flex flex-col items-center justify-center h-[56px] w-[72px] rounded-2xl ${
983       activeTab === 'access'
984       ? 'bg-[#60f8cb] text-zinc-900 font-bold shadow-sm'
985       : 'text-zinc-500 hover:text-zinc-800'
986     }}
987     >
988     <User className="w-5 h-5 mb-0.5" />
989     <span className="text-[10px] font-sans font-bold">Access</span>
990   </button>
```

```
991 </nav>
992 })
993 {isChatOpen && (
994 <div className="fixed inset-0 z-50 bg-zinc-950 flex flex-col">
995 <NavigationChat
996   currentProfileMode={accessibilityMode}
997   currentLocation={scanSuccessData?.name || 'Campus Entrance'}
998   lastQrLocation={scanSuccessData?.id || undefined}
999   onNavigateToDestination={({origin, dest} => {
1000     setIsChatOpen(false);
1001     setActiveRouteConfig({ origin, destination: dest });
1002     setIsIndoorNav(true);
1003     setIsActiveNavigation(true);
1004   })}
1005   onClose={() => setIsChatOpen(false)}
1006 />
1007 </div>
1008 )}
1009
1010 /* Global Floating Toast for Incident Submission */
1011 {incidentSubmittedToast && (
1012 <div className="fixed bottom-24 left-1/2 transform -translate-x-1/2 bg-[#0d9488] text-white px-5 py-3.5 rounded-2xl
1013   flex items-center justify-center gap-2">
1014   <span>Report Submitted</span>
1015   <svg className="w-5 h-5 fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24">
1016     <path strokeLinecap="round" strokeLinejoin="round" d="M9 12l2 4 4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" />
1017   </svg>
1018   <div className="text-left">
1019     <h4 className="font-bold text-sm leading-tight text-white">Report Submitted</h4>
1020     <p className="text-[11px] text-teal-100 mt-1 leading-normal">
1021       Facility staff have been notified. This report is now active in the routing engine.
1022     </p>
1023   </div>
1024 </div>
1025 )}
1026
1027 </div>
1028 </div>
1029 );
1030 }
```

## types.ts

TypeScript | frontend/src/types.ts

56 lines | 1.2 KB

```
1 /**
2  * @license
3  * SPDX-License-Identifier: Apache-2.0
4  */
5
6 export type NavigationMode = 'standard' | 'wheelchair' | 'vision' | 'hearing' | 'cognitive' | 'chronic';
7
8 export interface AccessibilityProfile {
9   id: NavigationMode;
10   name: string;
11   description: string;
12   icon: string;
13   details: string;
14   color: string;
15   accentColor: string;
16 }
17
18 export interface SavedMap {
19   id: string;
20   name: string;
21   floorsCount: number;
22   sizeMb: number;
23   imageThumbnail: string;
24   category: 'academic' | 'facility' | 'other';
25 }
26
27 export interface HazardIssue {
28   id: string;
29   type: 'construction' | 'lift' | 'toilet' | 'wet_floor' | 'door' | 'other';
30   typeName: string;
31   location: string;
32   details: string;
33   photoAttached: boolean;
```

```
34   photoUrl?: string;
35   timestamp: string;
36   isCustom?: boolean;
37 }
38
39 export interface RouteOption {
40   id: string;
41   name: string;
42   estMinutes: number;
43   distanceMiles: number;
44   isOptimal: boolean;
45   isAccessible: boolean;
46   features: string[];
47   warnings: string[];
48 }
49
50 export interface NavStep {
51   stepIndex: number;
52   instruction: string;
53   subInstructions: string;
54   directionIcon: 'straight' | 'turn_left' | 'turn_right' | 'elevator' | 'stairs' | 'warning';
55   distanceAhead: string;
56 }
```

data.ts

TypeScript | frontend/src/data.ts

203 lines | 7.0 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import { AccessibilityProfile, SavedMap, HazardIssue, RouteOption, NavStep } from './types';
7
8  export const PROFILES: AccessibilityProfile[] = [
9    {
10     id: 'wheelchair',
11     name: 'Wheelchair',
12     description: 'Prioritizes ramps, elevators, and wide paths.',
13     details: 'Step-free matching, automatic elevator warnings, extra hallway width margins.',
14     icon: 'accessibility',
15     color: 'bg-primary text-on-primary',
16     accentColor: '#60f8cb'
17   },
18   {
19     id: 'vision',
20     name: 'Vision Impaired',
21     description: 'High contrast, voice guidance, tactile cues.',
22     details: 'Auditory notifications, sound beacon alignment, screen reader optimizations.',
23     icon: 'eye-off',
24     color: 'bg-surface-container-high text-on-surface-variant',
25     accentColor: '#3fdeb3'
26   },
27   {
28     id: 'hearing',
29     name: 'Hearing Impaired',
30     description: 'Visual alerts and text-based notifications.',
31     details: 'Visual-flashing notifications, captions on route checkpoints, visual warning icons.',
32     icon: 'ear-off',
33     color: 'bg-surface-container-high text-on-surface-variant',
34     accentColor: '#3fdeb3'
35   },
36   {
37     id: 'cognitive',
38     name: 'Cognitive Support',
39     description: 'Simplified interfaces, step-by-step clear instructions.',
40     details: 'Minimalist screen elements, plain-text direction steps, step landmark descriptions.',
41     icon: 'brain',
42     color: 'bg-surface-container-high text-on-surface-variant',
43     accentColor: '#3fdeb3'
44   },
45   {
46     id: 'chronic',
47     name: 'Invisible/Chronic',
48     description: 'Focus on rest stops, low-exertion routes.',
49     details: 'Prioritizes rest point seating, low-elevation changes, slow-climb pacing options.',
50     icon: 'heart',
```

```
51   color: 'bg-surface-container-high text-on-surface-variant',
52   accentColor: '#3fdeb3'
53 },
54 {
55   id: 'standard',
56   name: 'Standard Walking',
57   description: 'Direct routes, typical walking conditions.',
58   details: 'Unrestricted stairs, normal velocity estimation, open path priorities.',
59   icon: 'footprints',
60   color: 'bg-surface-container-high text-on-surface-variant',
61   accentColor: '#3fdeb3'
62 }
63 ];
64
65 export const INITIAL_MAPS: SavedMap[] = [
66   {
67     id: 'building-1-tech',
68     name: 'Engineering & Tech Building',
69     floorsCount: 3,
70     sizeMb: 7.5,
71     imageThumbnail: 'https://lh3.googleusercontent.com/aida-public/AB6AXuB4EJiY_W3Qp7ZYtW50TfdAa_bx0iSQin07ai5gxTzts3dZ2Y6J3
72     gXabEggydyVh6vM00Pmic',
73   },
74   {
75     id: 'building-2-medical',
76     name: 'Medical & Student Life',
77     floorsCount: 3,
78     sizeMb: 5.2,
79     imageThumbnail: 'https://lh3.googleusercontent.com/aida-public/AB6AXuC14v84jd0i6c8wRXsEunr2ESvdqt0lkLkDegE5i7ola496ed5Zu
80     qgXp00qt1r3pue$Eity',
81   },
82   {
83     id: 'building-3-admin',
84     name: 'Admin & Arts Building',
85     floorsCount: 4,
86     sizeMb: 6.8,
87     imageThumbnail: 'https://lh3.googleusercontent.com/aida-public/AB6AXuB4EJiY_W3Qp7ZYtW50TfdAa_bx0iSQin07ai5gxTzts3dZ2Y6J3
88     gXabEggydyVh6vM00P...',
89   }
90 ];
91
92
93 export const INITIAL_ISSUES: HazardIssue[] = [
94   {
95     id: 'issue-1',
96     type: 'wet_floor',
97     typeName: 'Wet Floor',
98     location: 'Engineering Block A – Corridor B',
99     details: 'Large water leak causing major pooling by Room 104. Slippery marble tiles.',
100    photoAttached: true,
101    timestamp: '2026-06-07T13:45:00Z'
102  },
103  {
104    id: 'issue-2',
105    type: 'lift',
106    typeName: 'Lift Not Working',
107    location: 'Science Hall – East Wing Lift 3',
108    details: 'Elevator out of service for scheduled inspection. No service to floors 2 and 3.',
109    photoAttached: false,
110    timestamp: '2026-06-07T14:10:00Z'
111  },
112  {
113    id: 'issue-3',
114    type: 'construction',
115    typeName: 'Construction Work',
116    location: 'Quad Main Walkway',
117    details: 'Repaving pathway tiles in front of Administration building. Fenced off completely.',
118    photoAttached: true,
119    timestamp: '2026-06-07T12:00:00Z'
120  }
121 ];
122
123 export const MOCK_HOTSPOTS = [
124   { name: 'Library', icon: 'local_library', id: 'main-library', lat: 19.1334, lng: 72.9133 },
125   { name: 'Cafeteria', icon: 'restaurant', id: 'cafeteria', lat: 19.1340, lng: 72.9140 },
126   { name: 'Admin Block', icon: 'business_center', id: 'admin-block', lat: 19.1325, lng: 72.9120 },
127   { name: 'Medical Center', icon: 'local_hospital', id: 'medical-center', lat: 19.1350, lng: 72.9150 },
128   { name: 'Dormitory', icon: 'home', id: 'dormitory', lat: 19.1310, lng: 72.9100 },
```

```
129 { name: 'Science Lab', icon: 'science', id: 'science-lab', lat: 19.1345, lng: 72.9110 },
130 { name: 'Main Gate', icon: 'door_front', id: 'main-gate', lat: 19.1360, lng: 72.9160 }
131 ];
132
133 export const RECENT_ROUTES = [
134   {
135     id: 'route-recent-1',
136     origin: 'Main Gate',
137     destination: 'Main Library',
138     mode: 'Outdoor',
139     duration: '5 mins',
140     accessible: true,
141   },
142   {
143     id: 'route-recent-2',
144     origin: 'Engineering Block A',
145     destination: 'Science Lab 304',
146     mode: 'Indoor',
147     duration: 'Building B',
148     accessible: false,
149   }
150 ];
151
152 // Navigation step list for the indoor floor blueprint view.
153 export const DYNAMIC_STEPS: NavStep[] = [
154   {
155     stepIndex: 1,
156     instruction: 'Depart from Ground Floor Entrance',
157     subInstructions: 'Proceed straight down Lobby corridor',
158     directionIcon: 'straight',
159     distanceAhead: '15m ahead'
160   },
161   {
162     stepIndex: 2,
163     instruction: 'Pass through automatic glass doors',
164     subInstructions: 'Ramp access is ready right-side',
165     directionIcon: 'straight',
166     distanceAhead: '5m ahead'
167   },
168   {
169     stepIndex: 3,
170     instruction: 'Turn left at Corridor B',
171     subInstructions: 'Lift 2 is 20m ahead',
172     directionIcon: 'turn_left',
173     distanceAhead: '20m ahead'
174   },
175   {
176     stepIndex: 4,
177     instruction: 'Take Lift 2 to the 2nd Floor',
178     subInstructions: 'Elevator audio cues active in Lift 2',
179     directionIcon: 'elevator',
180     distanceAhead: 'In 12m'
181   },
182   {
183     stepIndex: 5,
184     instruction: 'Exit elevator and turn right',
185     subInstructions: 'Pass Science Department display cabinets',
186     directionIcon: 'turn_right',
187     distanceAhead: '10m ahead'
188   },
189   {
190     stepIndex: 6,
191     instruction: 'Walk down central floor hallway',
192     subInstructions: 'Accessible single door with handrail',
193     directionIcon: 'straight',
194     distanceAhead: '30m ahead'
195   },
196   {
197     stepIndex: 7,
198     instruction: 'Arrive at Room 204 on your left',
199     subInstructions: 'Automated door button is mounted left of frame',
200     directionIcon: 'warning',
201     distanceAhead: 'Dest'
202   }
203 ];
```



```
77 headers: { 'Content-Type': 'application/json' },
78 body: JSON.stringify({ origin, destination, profile })
79 });
80 if (response.ok) {
81   const data = await response.json();
82   localStorage.setItem(cacheKey, JSON.stringify(data));
83   return data;
84 }
85 } catch (err) {
86   // Backend unavailable, fallback to cache or offline calculation
87 }
88 }
89
90 // Serve from offline cache if available.
91 const cached = localStorage.getItem(cacheKey);
92 if (cached) {
93   return JSON.parse(cached);
94 }
95
96 // Offline route calculation – profile-aware fallback when backend is unreachable.
97
98 // Generate simulated route options based on profile constraints
99 const minutes = Math.floor(Math.random() * 6) + 4;
100 const miles = parseFloat((0.2 + Math.random() * 0.3).toFixed(2));
101
102 const options: RouteOption[] = [];
103
104 if (profile === 'wheelchair') {
105   options.push({
106     id: 'opt-1',
107     name: 'Step-Free Ramp Route',
108     estMinutes: minutes,
109     distanceMiles: miles,
110     isOptimal: true,
111     isAccessible: true,
112     features: ['Elevators', 'Ramps', 'Extra-wide corridors'],
113     warnings: ['Heavy doors at main corridor B']
114   });
115   options.push({
116     id: 'opt-2',
117     name: 'Elevator-Only Service Route',
118     estMinutes: minutes + 3,
119     distanceMiles: miles + 0.1,
120     isOptimal: false,
121     isAccessible: true,
122     features: ['Elevators', 'Automatic push-doors'],
123     warnings: ['Slightly longer distance']
124   });
125 } else if (profile === 'vision') {
126   options.push({
127     id: 'opt-1',
128     name: 'Tactile Navigation Walkway',
129     estMinutes: minutes + 1,
130     distanceMiles: miles,
131     isOptimal: true,
132     isAccessible: true,
133     features: ['Tactile guidance lines', 'Sound beacons active', 'Continuous handrails'],
134     warnings: ['Wet floor reported in Corridor B']
135   });
136 } else if (profile === 'chronic') {
137   options.push({
138     id: 'opt-1',
139     name: 'Rest-Bench Friendly Path',
140     estMinutes: minutes + 2,
141     distanceMiles: miles,
142     isOptimal: true,
143     isAccessible: true,
144     features: ['Frequent bench seating', 'Low-elevation change', 'Air conditioned corridors'],
145     warnings: []
146   });
147 } else {
148   // Standard and others
149   options.push({
150     id: 'opt-1',
151     name: 'Direct Central Stairs Route',
152     estMinutes: minutes - 2,
153     distanceMiles: miles - 0.05,
154     isOptimal: true,
```

```
155   isAccessible: profile === 'standard',
156   features: ['Direct stairwells', 'Central walkway'],
157   warnings: ['Contains 2 sets of stairs']
158 });
159 options.push({
160   id: 'opt-2',
161   name: 'Accessible Outer Bypass',
162   estMinutes: minutes + 1,
163   distanceMiles: miles + 0.05,
164   isOptimal: false,
165   isAccessible: true,
166   features: ['Step-free outer path', 'Ramps'],
167   warnings: []
168 });
169 }
170
171 // Cache final result
172 localStorage.setItem(cacheKey, JSON.stringify(options));
173 return options;
174 }
175
176 /**
177  * Look up a scanned QR checkpoint's metadata.
178  * Fetches the local JSON file.
179  */
180 export async function lookupQR(code: string): Promise<any> {
181   if (API_BASE_URL) {
182     try {
183       const response = await fetch(`${API_BASE_URL}/qr/lookup?code=${encodeURIComponent(code)}`);
184       if (response.ok) {
185         const data = await response.json();
186         unlockBuildingMap(data.building);
187         return data;
188       }
189     } catch (err) {
190       // Backend unavailable, fallback to local
191     }
192   }
193
194   // Fetch local checkpoint list
195   try {
196     const response = await fetch('/qr_map.json');
197     if (response.ok) {
198       const map = await response.json();
199       const match = map[code];
200       if (match) {
201         unlockBuildingMap(match.building);
202         return match;
203       }
204     }
205   } catch (err) {
206     // Local file not available
207   }
208
209   throw new Error('QR Checkpoint code not recognized or invalid.');
```

```
210 }
211
212 /**
213  * Report a new hazard/obstacle to the map network.
214  */
215 export async function reportIncident(
216   incident: Omit<HazardIssue, 'id' | 'timestamp'>
217 ): Promise<HazardIssue> {
218   const mockResult: HazardIssue = {
219     ...incident,
220     id: `issue-${Math.floor(Math.random() * 10000)}`,
221     timestamp: new Date().toISOString(),
222     isCustom: true
223   };
224
225   if (API_BASE_URL) {
226     try {
227       const response = await fetch(`${API_BASE_URL}/alert`, {
228         method: 'POST',
229         headers: { 'Content-Type': 'application/json' },
230         body: JSON.stringify(mockResult)
231       });
232       if (response.ok) {
```



```
233     return await response.json();
234   }
235   } catch (err) {
236     // Backend unavailable, persist locally
237   }
238 }
239
240 // Persist locally so the report survives a page reload.
241 const existing = localStorage.getItem('custom_issues');
242 const issues: HazardIssue[] = existing ? JSON.parse(existing) : [];
243 issues.push(mockResult);
244 localStorage.setItem('custom_issues', JSON.stringify(issues));
245
246 return mockResult;
247 }
248
249 /**
250  * Unlock a building map in the device's persistent cache.
251  * Storing this in local storage ensures users do not have to rescan QR codes.
252  */
253 export function unlockBuildingMap(buildingName: string): void {
254   const unlocked = getUnlockedBuildings();
255   if (!unlocked.includes(buildingName)) {
256     unlocked.push(buildingName);
257     localStorage.setItem('unlocked_buildings', JSON.stringify(unlocked));
258   }
259 }
260
261 /**
262  * Check if a building's indoor map is already cached/unlocked on the device.
263  */
264 export function isBuildingMapUnlocked(buildingName: string): boolean {
265   return getUnlockedBuildings().includes(buildingName);
266 }
267
268 /**
269  * Get all unlocked buildings from storage.
270  */
271 export function getUnlockedBuildings(): string[] {
272   const cached = localStorage.getItem('unlocked_buildings');
273   return cached ? JSON.parse(cached) : ['Engineering Block A'];
274 }
275
276 /**
277  * Upload an image (File or base64 data URL string) to Cloudinary.
278  * Uses direct client-side unsigned uploads.
279  */
280 export async function uploadToCloudinary(fileOrDataURL: File | string): Promise<string> {
281   const cloudName = import.meta.env.VITE_CLOUDINARY_CLOUD_NAME;
282   const uploadPreset = import.meta.env.VITE_CLOUDINARY_UPLOAD_PRESET;
283
284   if (!cloudName || !uploadPreset || cloudName === 'your_cloud_name' || uploadPreset === 'your_unsigned_preset') {
285     // Cloudinary not configured – return a local object URL as fallback.
286     return typeof fileOrDataURL === 'string' ? fileOrDataURL : URL.createObjectURL(fileOrDataURL);
287   }
288
289   const formData = new FormData();
290   formData.append('file', fileOrDataURL);
291   formData.append('upload_preset', uploadPreset);
292
293   const response = await fetch(`https://api.cloudinary.com/v1_1/${cloudName}/image/upload`, {
294     method: 'POST',
295     body: formData
296   });
297
298   if (!response.ok) {
299     const errorData = await response.json();
300     throw new Error(errorData.error?.message || 'Failed to upload image to Cloudinary.');
```

## 07 Frontend — Components

### CampusMap.tsx

TypeScript/React | frontend/src/components/CampusMap.tsx

305 lines | 9.5 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useEffect, useRef, useState } from 'react';
7  import { MapContainer, TileLayer, Marker, useMap, Circle, useMapEvents } from 'react-leaflet';
8  import L from 'leaflet';
9  import 'leaflet-routing-machine';
10 import { renderToStaticMarkup } from 'react-dom/server';
11
12 // Fix default leaflet marker icon paths (broken in Vite builds)
13 delete (L.Icon.Default.prototype as any)._getIconUrl;
14 L.Icon.Default.mergeOptions({
15   iconRetinaUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon-2x.png',
16   iconUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon.png',
17   shadowUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-shadow.png',
18 });
19
20 // Custom pulsing user-location marker (matches the design screenshot)
21 const userLocationIcon = L.divIcon({
22   className: '',
23   html: `
24     <div class="user-location-marker">
25       <div class="ping-ring"></div>
26       <div class="ping-ring ping-ring-2"></div>
27       <div class="dot"></div>
28     </div>
29   `,
30   iconSize: [40, 40],
31   iconAnchor: [20, 20],
32 });
33
34
35
36 // Component that smoothly re-centers map when user position changes
37 function LocationUpdater({ position }: { position: [number, number] }) {
38   const map = useMap();
39   const didFlyRef = useRef(false);
40
41   useEffect(() => {
42     if (!didFlyRef.current) {
43       map.setView(position, 17, { animate: false });
44       didFlyRef.current = true;
45     } else {
46       map.flyTo(position, map.getZoom(), { animate: true, duration: 1 });
47     }
48   }, [position, map]);
49
50   return null;
51 }
52
53 // Default fallback: IIT Bombay campus (a real university campus)
54 const DEFAULT_CENTER: [number, number] = [19.1334, 72.9133];
55
56 interface RoutingControlProps {
57   origin: [number, number];
58   destination: [number, number];
59   onRouteCalculated?: (steps: any[], summary: { distanceMeters: number; durationSeconds: number }) => void;
60 }
61
62 function RoutingControl({ origin, destination, onRouteCalculated }: RoutingControlProps) {
63   const map = useMap();
64
65   useEffect(() => {
66     if (!origin || !destination) return;
67
68     const routingControl = L.Routing.control({
69       waypoints: [
70         L.latLng(origin[0], origin[1]),
```

```
71 L.latLng(destination[0], destination[1])
72 ],
73 routeWhileDragging: false,
74 addWaypoints: false,
75 fitSelectedRoutes: true,
76 showAlternatives: false,
77 show: false,
78 lineOptions: {
79   styles: [{ color: '#2dd4bf', weight: 6, opacity: 0.8 }],
80   extendToWaypoints: true,
81   missingRouteTolerance: 0
82 }
83 }).addTo(map);
84
85 if (onRouteCalculated) {
86   routingControl.on('routesfound', (e: any) => {
87     const routes = e.routes;
88     if (routes && routes.length > 0) {
89       const route = routes[0];
90       const summary = {
91         distanceMeters: route.summary.totalDistance || 0,
92         durationSeconds: route.summary.totalTime || 0
93       };
94
95       // Map instructions to navigation steps
96       const steps = (route.instructions || []).map((inst: any, idx: number) => {
97         const coordIndex = inst.index;
98         const latLng = route.coordinates[coordIndex];
99         const coords: [number, number] | undefined = latLng ? [latLng.lat, latLng.lng] : undefined;
100
101         let directionIcon = 'straight';
102         const type = (inst.type || '').toLowerCase();
103         if (type.includes('left')) {
104           directionIcon = 'turn_left';
105         } else if (type.includes('right')) {
106           directionIcon = 'turn_right';
107         } else if (type.includes('elevator')) {
108           directionIcon = 'elevator';
109         } else if (type.includes('stair') || type.includes('warn')) {
110           directionIcon = 'warning';
111         }
112
113         return {
114           stepIndex: idx + 1,
115           instruction: inst.text || 'Continue along the walkway',
116           subInstructions: `Proceed for ${inst.distance || 0}.toFixed(0)}m`,
117           directionIcon,
118           distanceAhead: `${inst.distance || 0}.toFixed(0)}m`,
119           coords
120         };
121       });
122
123       onRouteCalculated(steps, summary);
124     }
125   });
126 }
127
128 return () => {
129   try {
130     map.removeControl(routingControl);
131   } catch (e) {
132     // ignore
133   }
134 };
135 }, [map, origin, destination, onRouteCalculated]);
136
137 return null;
138 }
139
140 function MapClickTracker({ onClick }: { onClick: (coords: [number, number]) => void }) {
141   useMapEvents({
142     click(e) {
143       onClick([e.latLng.lat, e.latLng.lng]);
144     }
145   });
146   return null;
147 }
148
```

```

149 interface CampusMapProps {
150   /** Extra CSS classes on the wrapper div */
151   className?: string;
152   /** Whether map controls (zoom) are shown */
153   showControls?: boolean;
154   routeOrigin?: [number, number];
155   routeDestination?: [number, number];
156   onRouteCalculated?: (steps: any[], summary: { distanceMeters: number; durationSeconds: number }) => void;
157   focusedStepCoords?: [number, number] | null;
158   onMapClick?: (coords: [number, number]) => void;
159   onUserLocationChange?: (coords: [number, number]) => void;
160   customPinCoords?: [number, number] | null;
161 }
162
163 export default function CampusMap({
164   className = '',
165   showControls = false,
166   routeOrigin,
167   routeDestination,
168   onRouteCalculated,
169   focusedStepCoords,
170   onMapClick,
171   onUserLocationChange,
172   customPinCoords
173 }: CampusMapProps) {
174   const [userPos, setUserPos] = useState<[number, number] | null>(null);
175   const [locating, setLocating] = useState(true);
176   const watchIdRef = useRef<number | null>(null);
177
178   useEffect(() => {
179     if (!navigator.geolocation) {
180       setLocating(false);
181       return;
182     }
183
184     navigator.geolocation.getCurrentPosition(
185       (pos) => {
186         const coords: [number, number] = [pos.coords.latitude, pos.coords.longitude];
187         setUserPos(coords);
188         if (onUserLocationChange) onUserLocationChange(coords);
189         setLocating(false);
190       },
191       () => setLocating(false),
192       { enableHighAccuracy: true, timeout: 8000 }
193     );
194
195     watchIdRef.current = navigator.geolocation.watchPosition(
196       (pos) => {
197         const coords: [number, number] = [pos.coords.latitude, pos.coords.longitude];
198         setUserPos(coords);
199         if (onUserLocationChange) onUserLocationChange(coords);
200       },
201       () => {},
202       { enableHighAccuracy: true }
203     );
204
205     return () => {
206       if (watchIdRef.current !== null) {
207         navigator.geolocation.clearWatch(watchIdRef.current);
208       }
209     };
210   }, []);
211
212   const center = userPos ?? DEFAULT_CENTER;
213   const mapCenter = focusedStepCoords ?? customPinCoords ?? center;
214
215   return (
216     <div className={`relative w-full h-full ${className}`}>
217       <MapContainer
218         center={mapCenter}
219         zoom={17}
220         zoomControl={showControls}
221         scrollWheelZoom={false}
222         dragging={true}
223         doubleClickZoom={showControls}
224         attributionControl={true}
225         style={{ width: '100%', height: '100%' }}
226     >

```

```

227 <TileLayer
228 url="https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png"
229 attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OSM</a> &copy; <a href="https://carto.c...
230 maxZoom={20}
231 />
232
233 {/ * Smooth location/focal updater */}
234 <LocationUpdater position={mapCenter} />
235
236 {/ * Click events handler for dropping a pin */}
237 {onMapClick && <MapClickTracker onClick={onMapClick} />}
238
239 {/ * User location dot */}
240 {userPos && (
241 <>
242 <Circle
243 center={userPos}
244 radius={18}
245 pathOptions={{
246 color: '#60f8cb',
247 fillColor: '#60f8cb',
248 fillOpacity: 0.18,
249 weight: 1.5,
250 }}
251 />
252 <Marker position={userPos} icon={userLocationIcon} />
253 </>
254 )}
255
256 {/ * Custom dropped pin red marker */}
257 {customPinCoords && (
258 <Marker
259 position={customPinCoords}
260 icon={L.icon({
261 iconUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon.png',
262 shadowUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-shadow.png',
263 iconSize: [25, 41],
264 iconAnchor: [12, 41]
265 })}
266 />
267 )}
268
269 {/ * Dynamic focused step pulsing dot indicator */}
270 {focusedStepCoords && (
271 <Marker
272 position={focusedStepCoords}
273 icon={L.divIcon({
274 className: '',
275 html: `
276 <div class="focused-step-marker">
277 <div style="position: absolute; width: 20px; height: 20px; border: 3.5px solid #002f5c; border-radius: 50%; background:
278 #eaf5f0>
279 `
280 ,
281 iconSize: [20, 20],
282 iconAnchor: [10, 10]
283 })}
284 />
285 )}
286
287 {/ * Route display if coordinates available */}
288 {routeOrigin && routeDestination && (
289 <RoutingControl
290 origin={routeOrigin}
291 destination={routeDestination}
292 onRouteCalculated={onRouteCalculated}
293 />
294 )}
295 </MapContainer>
296
297 {/ * Locating overlay */}
298 {locating && (
299 <div className="absolute inset-0 flex flex-col items-center justify-center bg-[#eaf5f0]/80 backdrop-blur-sm z-[1000]
300 p-10 w-10 h-10 rounded-full border-4 border-teal-400 border-t-transparent animate-spin mb-3" />
301 <p className="text-xs font-bold text-zinc-500 tracking-wide">Locating you...</p>
302 </div>
303 )}
304 );

```

305 }

## NavigationChat.tsx

TypeScript/React | frontend/src/components/NavigationChat.tsx

269 lines | 10.0 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useState, useEffect, useRef } from 'react';
7  import { Send, Mic, Cpu, Sparkles, Navigation, Volume2, AlertTriangle, Compass, Bot, CheckCircle, X } from '
8  import { NavigationMode, RouteOption } from '../types';
9  import { queryAgent } from '../services/api';
10
11  interface Message {
12    id: string;
13    sender: 'user' | 'agent';
14    text: string;
15    timestamp: Date;
16    agentSteps?: {
17      stepName: string;
18      details: string;
19      status: 'pending' | 'success' | 'info';
20    }[];
21    actionButton?: {
22      label: string;
23      onClick: () => void;
24    };
25  }
26
27  interface NavigationChatProps {
28    currentProfileMode: NavigationMode;
29    currentLocation: string;
30    lastQrLocation?: string;
31    onNavigateToDestination: (origin: string, destination: string) => void;
32    onClose?: () => void;
33  }
34
35  export default function NavigationChat({
36    currentProfileMode,
37    currentLocation,
38    lastQrLocation,
39    onNavigateToDestination,
40    onClose
41  }: NavigationChatProps) {
42    const [messages, setMessages] = useState<Message[]>([
43      {
44        id: 'msg-init-1',
45        sender: 'agent',
46        text: `CampusPilot AI Navigator ready.\n\nActive profile: ${currentProfileMode}\nCurrent location:
47        ${currentLocation}\nWhere would you like to go?`,
48        timestamp: new Date(),
49      }
50    ]);
51
52    const [inputValue, setInputValue] = useState('');
53    const [isVoiceListening, setIsVoiceListening] = useState(false);
54    const [isAgentTyping, setIsAgentTyping] = useState(false);
55    const chatEndRef = useRef<HTMLDivElement>(null);
56
57    // Auto scroll to bottom
58    useEffect(() => {
59      chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
60    }, [messages, isAgentTyping]);
61
62    const handleSendText = async (text: string) => {
63      const trimmed = text.trim();
64      if (!trimmed) return;
65
66      // Add user message
67      const userMsg: Message = {
68        id: `msg-user-${Date.now()}`,
69        sender: 'user',
70        text: trimmed,
71        timestamp: new Date()
72      };
73      setMessages([...messages, userMsg]);
74
75      // Add agent response
76      const agentMsg: Message = {
77        id: `msg-agent-${Date.now()}`,
78        sender: 'agent',
79        text: `Processing your request: ${trimmed}...`,
80        timestamp: new Date(),
81        agentSteps: [
82          {
83            stepName: 'Analyzing request',
84            details: `Analyzing your request: ${trimmed}`,
85            status: 'pending'
86          },
87          {
88            stepName: 'Fetching data',
89            details: `Fetching data from the database...`,
90            status: 'pending'
91          },
92          {
93            stepName: 'Generating response',
94            details: `Generating a response based on the data...`,
95            status: 'pending'
96          },
97          {
98            stepName: 'Finalizing response',
99            details: `Finalizing the response and preparing to send it back to you...`,
100            status: 'pending'
101          }
102        ]
103      };
104      setMessages([...messages, agentMsg]);
105
106      // Simulate agent processing time
107      await new Promise((resolve) => {
108        setTimeout(resolve, 2000);
109      });
110
111      // Final agent response
112      const finalAgentMsg: Message = {
113        id: `msg-agent-final-${Date.now()}`,
114        sender: 'agent',
115        text: `Here's the information you requested: ${trimmed}...`,
116        timestamp: new Date(),
117        agentSteps: [
118          {
119            stepName: 'Finalizing response',
120            details: `Finalizing the response and preparing to send it back to you...`,
121            status: 'pending'
122          }
123        ]
124      };
125      setMessages([...messages, finalAgentMsg]);
126
127      // Simulate agent typing
128      setIsAgentTyping(true);
129      setTimeout(() => {
130        setIsAgentTyping(false);
131      }, 2000);
132
133      // Close chat if requested
134      if (onClose) {
135        onClose();
136      }
137    };
138  }
```

```

73   setMessages(prev => [...prev, userMsg]);
74   setInputValue('');
75   setIsAgentTyping(true);
76
77   try {
78     // Call the real backend agent
79     const result = await queryAgent(
80       currentLocation,
81       trimmed,
82       currentProfileMode,
83       lastQrLocation
84     );
85
86     // Map real agent steps to display format
87     const agentSteps: Message['agentSteps'] = result.agent_steps.map((step, i) => ({
88       stepName: `${i + 1}. Tool: ${step.tool}`,
89       details: `Input: ${step.input}\nOutput: ${step.output}`,
90       status: 'success' as const
91     }));
92
93     // Add intent step at the front if available
94     if (result.intent) {
95       agentSteps.unshift({
96         stepName: '0. Intent Classified',
97         details: `Intent: [${result.intent.toUpperCase()}] | Profile: [${result.profile_applied?.toUpperCase()} ||
98         currentProfileMode.toUpperCase()
99       });
100     }
101
102     let responseText = result.response;
103     if (result.intent === 'emergency') {
104       // Emergency responses are pre-LLM – flag them distinctly
105       agentSteps.length = 0;
106       agentSteps.push({
107         stepName: 'Emergency Override',
108         details: 'Pre-LLM heuristic triggered. Agent bypassed. Response time < 1ms.',
109         status: 'info' as const
110       });
111     }
112
113     const agentMsg: Message = {
114       id: `msg-agent-${Date.now()}`,
115       sender: 'agent',
116       text: responseText,
117       timestamp: new Date(),
118       agentSteps,
119       actionButton: result.route_data ? {
120         label: 'Start Guidance',
121         onClick: () => onNavigateToDestination(currentLocation, trimmed)
122       } : undefined
123     };
124
125     setMessages(prev => [...prev, agentMsg]);
126
127   } catch (err) {
128     setMessages(prev => [...prev, {
129       id: `msg-err-${Date.now()}`,
130       sender: 'agent',
131       text: 'Unable to reach the navigation service. Please check your connection and try again.',
132       timestamp: new Date()
133     }]);
134   } finally {
135     setIsAgentTyping(false);
136   }
137 };
138
139 return (
140   <div className="flex flex-col bg-zinc-950 text-white h-[calc(100vh-76px)] overflow-hidden font-sans relative">
141     {/ * ■■■■ HUD HEADER ■■■■ */ /}
142     <header className="px-5 py-3.5 bg-zinc-900 border-b border-zinc-800/80 flex items-center justify-between shrink-0 z-10">
143       <div className="flex items-center gap-2">
144         <Bot className="w-5 h-5 text-teal-400 animate-pulse" />
145         <div className="text-left">
146           <h2 className="text-sm font-extrabold text-white tracking-wide leading-none">CampusPilot AI Navigator</h2>
147           <span className="text-[10px] text-teal-400 font-mono tracking-widest uppercase mt-1 block">Agentic ReAct – Live</span>
148         </div>
149       </div>
150     {onClose && (

```

```

151 <button
152   onClick={onClose}
153   className="text-zinc-400 hover:text-white p-1.5 rounded-xl hover:bg-zinc-800 transition-colors cursor-pointer"
154   aria-label="Close Chat"
155 >
156 <X className="w-5 h-5" />
157 </button>
158 })
159 </header>
160
161 {/* ■■■ CHAT BUBBLE STREAM ■■■ */}
162 <div className="flex-1 overflow-y-auto px-5 py-4 space-y-6 no-scrollbar pb-32 z-0">
163   {messages.map((msg) => (
164     <div
165       key={msg.id}
166       className={`flex flex-col max-w-[85%] ${
167         msg.sender === 'user' ? 'ml-auto items-end' : 'mr-auto items-start'
168       }`}
169     >
170       {/* Sender Label */}
171       <span className="text-[10px] text-zinc-500 font-mono mb-1 select-none">
172         {msg.sender === 'user' ? 'YOU' : 'CAMPUSPILOT_AGENT'}
173       </span>
174
175       {/* Bubble */}
176       <div
177         className={`rounded-2xl p-4 text-sm font-medium leading-relaxed shadow-md select-text ${
178           msg.sender === 'user'
179             ? 'bg-teal-600 text-white rounded-tr-none'
180             : 'bg-zinc-900 text-zinc-105 border border-zinc-850 rounded-tl-none space-y-3'
181         }`}
182       >
183         {/* Message text */}
184         <div className="whitespace-pre-wrap text-left font-sans prose prose-invert prose-sm">
185           {msg.text}
186         </div>
187
188         {/* Agent step trace – shows real tool calls from LangChain */}
189         {msg.agentSteps && msg.agentSteps.length > 0 && (
190           <div className="mt-3 space-y-1.5 border-t border-zinc-800 pt-3">
191             <span className="text-[9px] font-mono text-zinc-600 tracking-widest uppercase block mb-1">Agent Trace</span>
192             {msg.agentSteps.map((step, i) => (
193               <div key={i} className="flex items-start gap-2 text-[11px]">
194                 <span className={`mt-0.5 w-2 h-2 rounded-full shrink-0 ${
195                   step.status === 'success' ? 'bg-teal-400' :
196                   step.status === 'info' ? 'bg-blue-400' : 'bg-zinc-500'
197                 }`} />
198                 <div>
199                   <span className="font-bold text-zinc-300">{step.stepName}</span>
200                   <span className="text-zinc-500 ml-1 break-all">{step.details}</span>
201                 </div>
202               </div>
203             ))}
204           </div>
205         )}
206
207         {/* Action Button inside message */}
208         {msg.actionButton && (
209           <div className="mt-4 pt-2">
210             <button
211               onClick={msg.actionButton.onClick}
212               className="w-full bg-[#60f8cb] text-zinc-950 hover:bg-[#4edab0] font-bold h-11 rounded-xl flex items-center justify-center...
213             <Navigation className="w-4 h-4 rotate-45" />
214             {msg.actionButton.label} </button>
215           </div>
216         )}
217       </div>
218     </div>
219   ))}
220
221   {/* Typing placeholder animation */}
222   {isAgentTyping && (
223     <div className="flex flex-col items-start mr-auto max-w-[80%]">
224       <span className="text-[10px] text-zinc-500 font-mono mb-1">CAMPUSPILOT_AGENT</span>
225       <div className="bg-zinc-900 border border-zinc-850 p-4 rounded-2xl rounded-tl-none flex items-center gap-2">
226         <div className="h-2 w-2 bg-teal-400 rounded-full animate-bounce" style={{ animationDelay: '0ms' }} />
227         <div className="h-2 w-2 bg-teal-400 rounded-full animate-bounce" style={{ animationDelay: '150ms' }} />
228

```



```
229 <div className="h-2 w-2 bg-teal-400 rounded-full animate-bounce" style={{ animationDelay: '300ms' }} />
230 </div>
231 </div>
232 }}
233
234 <div ref={chatEndRef} />
235 </div>
236
237 {/} ■■■■ BOTTOM CONTROL SHEETS (INPUT BAR) ■■■■ */}
238 <footer className="absolute bottom-0 left-0 right-0 bg-zinc-900 border-t border-zinc-850 px-4 pt-3 pb-6 shrink-0 z-10">
239 <form
240   onSubmit={(e) => {
241     e.preventDefault();
242     handleSendText(inputValue);
243   }}
244   className="relative flex items-center w-full max-w-xl mx-auto"
245   >
246   {/} Text Input */}
247   <div className="relative flex-1">
248     <input
249       type="text"
250       placeholder="Ask CampusPilot Navigator..."
251       value={inputValue}
252       onChange={(e) => setInputValue(e.target.value)}
253     />
254     <div className="block w-full bg-zinc-950 border border-zinc-800 text-white rounded-xl py-3.5 pl-4 pr-12 text-sm font-semibold focus:outline-none">
255       {/} Send button */}
256       <button
257         type="submit"
258         disabled={!inputValue.trim()}
259         className="absolute inset-y-0 right-2 pr-3 flex items-center text-teal-400 hover:text-teal-300 disabled:text-zinc-650 cursor-pointer"
260       >
261         <Send className="w-4 h-4" />
262       </button>
263     </div>
264   </div>
265 </form>
266 </footer>
267 </div>
268 );
269 }
```

## ActiveNavigationView.ts

TypeScript/React | frontend/src/components/ActiveNavigationView.tsx

x

569 lines | 24.7 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useState, useMemo, useEffect } from 'react';
7  import { ArrowLeft, CheckCircle2, AlertTriangle, ArrowUp, Compass, ZoomIn, ZoomOut, Eye, XCircle, Search } from 'lucide-react';
8  import { BUILDING_STEPS } from '../data';
9  import { getBuilding, renderBuildingSVG, GRID_LINES, GRID_COLS } from '../indoorMaps';
10 import CampusMap from './CampusMap';
11 import BottomSheet from './BottomSheet';
12 import { generateDynamicRoute } from '../utils/routing';
13 import { NavStep } from '../types';
14
15 interface ActiveNavigationViewProps {
16   onBack: () => void;
17   onStopNav: () => void;
18   onReportObstacle: (location: string) => void;
19   onSosClick?: () => void;
20   buildingName?: string;
21   isIndoor?: boolean;
22   themeMode?: 'light' | 'dark';
23   routeOriginCoords?: [number, number];
24   routeDestinationCoords?: [number, number];
25   selectedMapId?: string;
26 }
27
28 export default function ActiveNavigationView({
29   onBack,
30   onStopNav,
31   onReportObstacle,
```

```

32 onSosClick,
33 buildingName = 'Engineering Block A',
34 isIndoor = true,
35 themeMode = 'light',
36 routeOriginCoords,
37 routeDestinationCoords,
38 selectedMapId = 'eng-block-a'
39 }: ActiveNavigationViewProps {
40   const [currentStepIdx, setCurrentStepIdx] = useState<number>(0);
41   const [showObstacleSuccess, setShowObstacleSuccess] = useState(false);
42
43   // State for dynamic outdoor routing
44   const [outdoorSteps, setOutdoorSteps] = useState<NavStep[]>([]);
45   const [currentOutdoorStepIdx, setCurrentOutdoorStepIdx] = useState<number>(0);
46   const [routeSummary, setRouteSummary] = useState<{ distanceMeters: number; durationSeconds: number } | null>(null);
47
48   // State for dynamic indoor routing
49   const [selectedRoomId, setSelectedRoomId] = useState<string | null>(null);
50   const [selectedFloorIdx, setSelectedFloorIdx] = useState<number>(0);
51
52   // Resolve building data from registry
53   const building = useMemo(() => getBuilding(selectedMapId), [selectedMapId]);
54
55   const indoorMap = useMemo(() => {
56     if (!building || !building.floors || building.floors.length === 0) return null;
57     return building.floors[selectedFloorIdx] || building.floors[0];
58   }, [building, selectedFloorIdx]);
59
60   const allRooms = useMemo(() => {
61     if (!building) return [];
62     return building.floors.flatMap((floor, fIdx) =>
63       (floor.rooms || []).map(r => ({ ...r, floorName: floor.name, floorIdx: fIdx }))
64     );
65   }, [building]);
66
67   useEffect(() => {
68     if (building && selectedRoomId) {
69       const fIdx = building.floors.findIndex(f => f.rooms?.some(r => r.id === selectedRoomId));
70       if (fIdx >= 0 && fIdx !== selectedFloorIdx) {
71         setSelectedFloorIdx(fIdx);
72         setCurrentStepIdx(0);
73       }
74     }
75     }, [building, selectedRoomId, selectedFloorIdx]);
76
77   const dynamicRoute = useMemo(() => {
78     if (!building || !selectedRoomId) return null;
79     let targetRoom = null;
80     let targetFloor = null;
81     for (const floor of building.floors) {
82       targetRoom = floor.rooms?.find(r => r.id === selectedRoomId);
83       if (targetRoom) {
84         targetFloor = floor;
85         break;
86       }
87     }
88     if (!targetRoom || !targetFloor || !targetFloor.entrance) return null;
89     return generateDynamicRoute(targetFloor.entrance, targetRoom, targetRoom.name);
90   }, [building, selectedRoomId]);
91
92   // Use dynamic route if selected, else map-specific steps or fallback
93   const stepsList = dynamicRoute ? dynamicRoute.steps : (indoorMap ? indoorMap.steps : DYNAMIC_STEPS);
94   const totalSteps = stepsList.length;
95   const activeStep = stepsList[currentStepIdx] || stepsList[0];
96
97   // Use dynamic dots or map-specific or fallback
98   const dotCoords = dynamicRoute ? dynamicRoute.dotCoords : (indoorMap ? indoorMap.dotCoords : [
99     { cx: 350, cy: 520 }, { cx: 350, cy: 480 }, { cx: 350, cy: 450 },
100    { cx: 400, cy: 420 }, { cx: 480, cy: 300 }, { cx: 560, cy: 300 }, { cx: 560, cy: 380 },
101    ]);
102
103   const activeCoord = dotCoords[currentStepIdx] || dotCoords[0];
104   const routePath = dynamicRoute ? dynamicRoute.routePath : (indoorMap ? indoorMap.routePath : 'M350 540 Q 350 300 500 300');
105   const hazard = useMapFeature?.hazard || { x: 432, y: 240, label: 'Spill Hazard' };
106
107   const handleNextStep = () => {
108     if (currentStepIdx < totalSteps - 1) {
109       setCurrentStepIdx(prev => prev + 1);

```

```
110   } else {
111     setShowObstacleSuccess(true);
112     setTimeout(() => {
113       setShowObstacleSuccess(false);
114       onStopNav();
115     }, 2500);
116   }
117   };
118
119   const handlePrevStep = () => {
120     if (currentStepIdx > 0) {
121       setCurrentStepIdx(prev => prev - 1);
122     }
123   };
124
125   // Failsafe fallback outdoor steps between hotspots (e.g. library, main gate, science building, etc.)
126   const getFailsafeOutdoorSteps = () => {
127     return [
128       {
129         stepIndex: 1,
130         instruction: 'Depart towards the main campus walkway',
131         subInstructions: 'Proceed straight for 80m',
132         directionIcon: 'straight',
133         distanceAhead: '80m',
134         coords: routeOriginCoords || [19.1360, 72.9160]
135       },
136       {
137         stepIndex: 2,
138         instruction: 'Turn left onto Central Mall Path',
139         subInstructions: 'Pass by the Student Cafeteria on your right',
140         directionIcon: 'turn_left',
141         distanceAhead: '120m',
142         coords: [19.1340, 72.9140]
143       },
144       {
145         stepIndex: 3,
146         instruction: 'Turn right at the Quad corridor sign',
147         subInstructions: 'Use wheelchair step-free ramp pathway',
148         directionIcon: 'turn_right',
149         distanceAhead: '150m',
150         coords: [19.1334, 72.9133]
151       },
152       {
153         stepIndex: 4,
154         instruction: 'Arrive at ' + (buildingName === 'Dropped Pin' ? 'Selected Location' : buildingName),
155         subInstructions: 'Your destination is ahead',
156         directionIcon: 'warning',
157         distanceAhead: 'Dest',
158         coords: routeDestinationCoords || [19.1325, 72.9120]
159       }
160     ];
161   };
162
163   const handleRouteCalculated = (steps: NavStep[], summary: { distanceMeters: number; durationSeconds: number }) => {
164     if (steps && steps.length > 0) {
165       setOutdoorSteps(steps);
166       setRouteSummary(summary);
167       setCurrentOutdoorStepIdx(0);
168     }
169   };
170
171   const activeOutdoorSteps = outdoorSteps.length > 0 ? outdoorSteps : getFailsafeOutdoorSteps();
172   const totalOutdoorSteps = activeOutdoorSteps.length;
173   const activeOutdoorStep = activeOutdoorSteps[currentOutdoorStepIdx] || activeOutdoorSteps[0];
174   const focusedCoords = activeOutdoorStep ? activeOutdoorStep.coords : null;
175
176   const handleNextOutdoorStep = () => {
177     if (currentOutdoorStepIdx < totalOutdoorSteps - 1) {
178       setCurrentOutdoorStepIdx(prev => prev + 1);
179     } else {
180       setShowObstacleSuccess(true);
181       setTimeout(() => {
182         setShowObstacleSuccess(false);
183         onStopNav();
184       }, 2500);
185     }
186   };
187
```

```

188 const handlePrevOutdoorStep = () => {
189   if (currentOutdoorStepIdx > 0) {
190     setCurrentOutdoorStepIdx(prev => prev - 1);
191   }
192 };
193
194 const currentPercent = Math.round(((currentStepIdx + 1) / totalSteps) * 100);
195
196 const renderStepIcon = (direction: string) => {
197   const defaultClass = "w-7 h-7 text-[#002f5c]";
198   if (direction === 'turn_left') {
199     return (
200       <svg className={defaultClass} fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24">
201         <path strokeLinecap="round" strokeLinejoin="round" d="M15 19l-7-7 7-7" />
202       </svg>
203     );
204   }
205   if (direction === 'turn_right') {
206     return (
207       <svg className={defaultClass} fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24">
208         <path strokeLinecap="round" strokeLinejoin="round" d="M9 5l7 7 7 7" />
209       </svg>
210     );
211   }
212   if (direction === 'elevator') {
213     return <span className="text-xl">■</span>;
214   }
215   return <ArrowUp className={defaultClass} />;
216 };
217
218 if (!isIndoor) {
219   // OUTDOOR ACTIVE NAVIGATION LAYOUT
220   return (
221     <div className="w-full flex flex-col bg-[#eef1fa] min-h-screen select-none font-sans relative pb-32">
222       {showObstacleSuccess && (
223         <div className="fixed top-24 left-1/2 transform -translate-x-1/2 p-4 rounded-2xl bg-teal-50 border border-teal-500/30
224           text-teal-600 le2 className="w-5 h-5 text-teal-600 animate-bounce" />
225         <span className="font-semibold text-xs">Destination Reached! Closing guide...</span>
226       </div>
227     )}
228
229     {/* Fullscreen map simulator background */}
230     <div className="absolute inset-0 z-0 overflow-hidden bg-[#e6e6f5] flex items-center justify-center">
231       <CampusMap
232         showControls={false}
233         routeOrigin={routeOriginCoords}
234         routeDestination={routeDestinationCoords}
235         onRouteCalculated={handleRouteCalculated}
236         focusedStepCoords={focusedCoords}
237       />
238     </div>
239
240     {/* Floating Zoom & Map Controls */}
241     <div className="absolute right-6 top-[220px] flex flex-col gap-3 z-10">
242       <button className="h-12 w-12 bg-white rounded-full shadow-lg flex items-center justify-center border border-zinc-200
243         text-zinc-700" />
244       </button>
245       <button className="h-12 w-12 bg-white rounded-full shadow-lg flex items-center justify-center border border-zinc-200
246         text-zinc-700" />
247       </button>
248       <button className="h-12 w-12 bg-white rounded-full shadow-lg flex items-center justify-center border border-zinc-200
249         text-zinc-700" />
250       </button>
251     </div>
252
253     {/* Top floating instruction box */}
254     <div className="relative z-10 px-2 pt-2 w-full max-w-md mx-auto">
255       <div className="bg-white rounded-[20px] p-4 shadow-lg border border-zinc-100 flex flex-col gap-3 text-left">
256         <div className="flex items-center justify-between">
257           <span className="bg-[#60f8cb]/20 text-[#00875a] text-[10px] font-extrabold uppercase tracking-widest px-3 py-1.5
258             rounded-ls" />
259           Outdoor Navigation
260         </span>
261         <span className="text-[10px] font-black text-zinc-400 uppercase tracking-widest">
262           Step {currentOutdoorStepIdx + 1} of {totalOutdoorSteps}
263         </span>
264       </div>
265     </div>

```

```

266 <div className="flex items-center gap-4">
267 <div className="h-12 w-12 bg-[#002f5c] rounded-full flex items-center justify-center shrink-0 shadow-sm text-white">
268 {renderStepIcon(activeOutdoorStep?.directionIcon || 'straight')}
269 </div>
270 <div className="flex-1">
271 <h2 className="text-sm font-bold text-[#002f5c] leading-snug">
272 {activeOutdoorStep?.instruction}
273 </h2>
274 <p className="text-xs text-blue-600 font-bold mt-1">
275 {activeOutdoorStep?.subInstructions || activeOutdoorStep?.distanceAhead}
276 </p>
277 </div>
278 </div>
279 </div>
280 </div>
281
282 {/* Bottom Navigation Dashboard status sheet */}
283 <div className="absolute bottom-0 left-1/2 -translate-x-1/2 w-full max-w-md z-20">
284 <BottomSheet defaultSnap="peek">
285 <div className="flex flex-col gap-5 text-left w-full mt-1">
286 {/* Target building & time metadata */}
287 <div className="flex items-start justify-between">
288 <div>
289 <h2 className="text-xl font-bold text-[#002f5c] font-sans">
290 {buildingName === 'Dropped Pin' && routeDestinationCoords
291 ? `Selected Location (${routeDestinationCoords[0].toFixed(5)}, ${routeDestinationCoords[1].toFixed(5)})`
292 : buildingName}
293 </h2>
294 <p className="text-xs text-zinc-550 font-medium mt-1">
295 Arriving in {routeSummary ? Math.ceil(routeSummary.durationSeconds / 60) : '5'} mins • Speed: Standard
296 </p>
297 </div>
298 <span className="text-lg font-black text-emerald-700 shrink-0">
299 {routeSummary ? (routeSummary.distanceMeters / 1000).toFixed(2) + ' km' : '0.40 km'}
300 </span>
301 </div>
302
303 {/* Dynamic progress indicator */}
304 <div className="w-full h-2.5 bg-zinc-100 rounded-full overflow-hidden">
305 <div
306 className="h-full bg-teal-600 rounded-full transition-all duration-350"
307 style={{ width: `${Math.round(((currentOutdoorStepIdx + 1) / totalOutdoorSteps) * 100)}%` }}
308 />
309 </div>
310
311 {/* Step Navigator controls */}
312 <div className="flex gap-2.5">
313 {currentOutdoorStepIdx > 0 && (
314 <button
315 onClick={handlePrevOutdoorStep}
316 className="flex-1 h-12 border border-zinc-250 hover:bg-zinc-100 text-zinc-700 font-bold text-xs rounded-xl
317 cursor-pointer ...
318 Previous Step
319 </button>
320 )}
321 <button
322 onClick={handleNextOutdoorStep}
323 className="flex-1 h-12 bg-[#002f5c] hover:bg-[#002447] text-white font-sans font-bold text-xs rounded-xl flex
324 items-center j...
325 {currentOutdoorStepIdx === totalOutdoorSteps - 1 ? 'Finish' : 'Next Step'}
326 </button>
327 </div>
328
329 {/* Action triggers grid */}
330 <div className="flex flex-col gap-3 border-t border-zinc-150 pt-4">
331 <div className="flex gap-3">
332 {/* Report obstacle */}
333 <button
334 onClick={() => onReportObstacle("Campus Walkway — near " + buildingName)}
335 className="flex-1 h-14 border-2 border-orange-500/70 text-orange-600 font-bold text-sm rounded-2xl flex items-center
336 justi...
337 <AlertTriangle className="w-4 h-4" />
338 Report Obstacle
339 </button>
340 {/* SOS Square button */}
341 <button
342 onClick={onSosClick}
343 aria-label="SOS Beacon Trigger"

```

```

344   className="h-14 px-4 bg-[#e1d48] hover:bg-[#be123c] text-white rounded-2xl flex items-center justify-center shrink-0
345   curs...
346   <span className="text-white text-[13px] font-black tracking-wider select-none uppercase">SOS</span>
347 </button>
348 </div>
349
350   {/* Stop Navigation */}
351   <button
352     onClick={onStopNav}
353     className="w-full h-14 bg-[#c82323] hover:bg-[#a81a1a] text-white font-bold rounded-2xl flex items-center
354     justify-center gap...
355     <XCircle className="w-5 h-5" />
356     Stop Navigation
357   </button>
358 </div>
359 </div>
360 </BottomSheet>
361 </div>
362 </div>
363 );
364 }
365
366 // INDOOR SCHEMA FLOOR BLUEPRINT NAVIGATION LAYOUT
367 return (
368   <div className="fixed inset-0 w-full flex flex-col bg-zinc-50 overflow-hidden select-none font-sans text-left z-50">
369     {/* Top sticky app bar */}
370     <header className="bg-white border-b border-zinc-150 shrink-0 z-50">
371       <div className="flex justify-between items-center w-full px-6 h-[72px]">
372         <div className="flex items-center gap-3">
373           <button
374             onClick={onBack}
375             className="p-2 mr-1 rounded-full text-zinc-650 hover:bg-zinc-100 transition-colors"
376             >
377             <ArrowLeft className="w-5 h-5" />
378           </button>
379           <div className="flex flex-col">
380             <h1 className="text-base font-bold text-[#002f5c] truncate">
381               {buildingName}
382             </h1>
383             <span className="text-[10px] text-zinc-400 font-extrabold uppercase tracking-widest mt-0.5">
384               Active Navigation
385             </span>
386           </div>
387         </div>
388       </div>
389
390       {/* Floor Selector */}
391       {building && building.floors.length > 1 && (
392         <div className="px-6 pb-2 pt-2 flex gap-2 overflow-x-auto no-scrollbar bg-white">
393           {building.floors.map((f, idx) => (
394             <button
395               key={f.id}
396               onClick={() => {
397                 setSelectedFloorIdx(idx);
398               }}
399               className={`px-4 py-1.5 rounded-full text-xs font-bold whitespace-nowrap transition-all border cursor-pointer ${
400                 idx === selectedFloorIdx
401                   ? 'bg-[#002f5c] text-white border-[#002f5c]'
402                   : 'bg-white text-zinc-500 border-zinc-200 hover:bg-zinc-50'
403               }`}
404             >
405               Floor {idx + 1}
406             </button>
407           )})}
408         </div>
409       )}
410
411       {/* Dynamic Destination Selector */}
412       {allRooms && allRooms.length > 0 && (
413         <div className="px-6 pb-4 pt-1 flex flex-col gap-1.5 bg-white">
414           <label className="text-[10px] font-extrabold text-zinc-400 uppercase tracking-widest pl-1">Target Destination</label>
415           <div className="flex items-center gap-2.5 bg-zinc-50 border border-zinc-200 rounded-xl px-3 py-2.5 shadow-sm
416             flex-grow-1 text-zinc-400 shrink-0" />
417           <select
418             value={selectedRoomId || ''}
419             onChange={(e) => {
420               setSelectedRoomId(e.target.value);
421               setCurrentStepIdx(0);

```

```
422 }}
423 className="flex-1 bg-transparent border-none p-0 text-sm font-bold text-[#002f5c] focus:outline-none focus:ring-0
424 cursor-pointer"
425 <option value="">Select a room to navigate to...</option>
426 {allRooms.map(room => (
427 <option key={room.id} value={room.id}>{room.name} (Floor {room.floorIdx + 1})</option>
428 ))}
429 </select>
430 </div>
431 </div>
432 )}
433 </header>
434
435 {showObstacleSuccess && (
436 <div className="fixed top-24 left-1/2 transform -translate-x-1/2 p-4 rounded-2xl bg-teal-50 border border-teal-500/30
437 text-teal-600 text-center">
438 <span className="font-semibold text-xs">Destination Reached! Closing guide...</span>
439 </div>
440 )}
441
442 {/* Map blueprint canvas */}
443 <main className="flex-1 relative overflow-hidden bg-zinc-100 flex items-center justify-center p-4">
444 {/* Schematic floor plan – dynamic per building */}
445 <div className="absolute inset-0 flex items-center justify-center opacity-90 p-4">
446 <svg
447 className="w-full h-full max-h-full max-w-[500px] drop-shadow-md"
448 fill="none"
449 viewBox="0 0 800 600"
450 xmlns="http://www.w3.org/2000/svg"
451 >
452 {/* Grid Lines */}
453 <g stroke="currentColor" strokeOpacity="0.04" strokeWidth="1" className="text-zinc-650">
454 <path d={GRID_LINES} />
455 <path d={GRID_COLS} />
456 </g>
457
458 {/* Dynamic building rooms & walls from registry */}
459 {indoorMap && renderBuildingSVG(indoorMap.id)}
460
461 {/* Custom active pathway */}
462 <path
463 className="dashed-route"
464 d={routePath}
465 fill="none"
466 stroke="#00c49a"
467 strokeLinecap="round"
468 strokeWidth="9"
469 />
470
471 {/* Dynamic User Position dot */}
472 <circle cx={activeCoord.cx} cy={activeCoord.cy} fill="#00c49a" opacity="0.25" r="18" className="transition-all
473 duration-500">
474 <circle cx={activeCoord.cx} cy={activeCoord.cy} fill="#10b981" r="9" stroke="ffffff" strokeWidth="2.5"
475 className="transition-all">
476 {/* Hazard warning flag */}
477 <g transform={`translate(${hazard.x + 38}, ${hazard.y - 5})`} strokeWidth="1">
478 <path d="M0 32 L18 0 L36 32 Z" fill="#f97316" stroke="#ca8a04" strokeLinejoin="round" strokeWidth="2" />
479 <text fill="ffffff" fontFamily="Inter" fontSize="18" fontWeight="black" textAnchor="middle" x="18" y="26">!</text>
480 <rect fill="#fee2e2" height="28" rx="14" stroke="#ef4444" strokeWidth="1" width="112" x={hazard.x} y={hazard.y}
481 className="hazard-flag">
482 <text fill="black" x={hazard.x + 56} y={hazard.y + 14}>Hazard!</text>
483 </text>
484 </svg>
485 </div>
486
487 {/* Map float zoom overlay links */}
488 <div className="absolute right-6 top-6 flex flex-col gap-3">
489 <button className="h-12 w-12 bg-white rounded-full shadow-lg flex items-center justify-center text-on-surface
490 text-sm font-medium">Zoom In
491 </button>
492 </div>
493 </main>
494
495 {/* Solid bottom navigation dashboard */}
496 <footer className="bg-white border-t border-zinc-200 shadow-xl rounded-t-[24px] shrink-0 relative z-10 p-5 flex
497 flex-wrap wrap">
498 <div className="flex items-start justify-between gap-4">
499 <div className="flex-1 pr-1">
```

```
500 <span className="text-[10px] font-extrabold text-[#002f5c] uppercase tracking-widest mb-1.5 block">
501 Step {activeStep.stepIndex} of {totalSteps}
502 </span>
503 <h2 className="text-lg font-bold text-zinc-900 font-sans leading-snug mb-2">
504 {activeStep.instruction}
505 </h2>
506 <p className="text-xs text-zinc-550 leading-normal">
507 {activeStep.subInstructions}
508 </p>
509 </div>
510
511 {/* Large direction symbol badge container */}
512 <div className="h-14 w-14 bg-indigo-50 rounded-2xl flex items-center justify-center text-[#002f5c] border
513 border-zinc-200">{activeStep.directionIcon}</div>
514 </div>
515 </div>
516
517 {/* Dynamic progress bar */}
518 <div className="w-full h-2.5 bg-zinc-200 rounded-full overflow-hidden select-none">
519 <div
520 className="h-full bg-emerald-500 rounded-full transition-all duration-350 ease-out"
521 style={{ width: `${currentPercent}%` }}>
522 </div>
523 </div>
524
525 {/* Buttons / Controls section */}
526 <div className="flex gap-3">
527 {currentStepIdx > 0 && (
528 <button
529 onClick={handlePrevStep}
530 className="flex-1 h-12 border border-zinc-250 hover:bg-zinc-100 text-zinc-700 font-bold text-xs rounded-xl
531 cursor-pointer transition-colors">
532 Previous Step
533 </button>
534 )}
535 <button
536 onClick={handleNextStep}
537 className="flex-1 h-12 bg-[#002f5c] hover:bg-[#002447] text-white font-sans font-bold text-xs rounded-xl flex
538 items-center justify-center">
539 {currentStepIdx === totalSteps - 1 ? 'Finish' : 'Next Step'}
540 </button>
541 </div>
542
543 <div className="flex gap-2.5 mt-1 border-t border-zinc-150 pt-4">
544 <button
545 onClick={() => onReportObstacle(buildingName + ' - ' + activeStep.instruction)}
546 className="flex-1 py-2 text-xs border border-orange-500/30 text-orange-600 font-bold rounded-lg flex items-center
547 justify-center gap-1">
548 <AlertTriangle className="w-3.5 h-3.5" />
549 Report Issue
550 </button>
551 {onSosClick && (
552 <button
553 onClick={onSosClick}
554 className="px-4 py-2 text-xs bg-[#c82323] hover:bg-[#a81a1a] text-white font-bold rounded-lg cursor-pointer"
555 >
556 SOS Alert
557 </button>
558 )}
559 <button
560 onClick={onStopNav}
561 className="px-4 py-2 text-xs bg-red-50 hover:bg-red-100 text-red-600 border border-red-200 font-bold rounded-lg
562 cursor-pointer">
563 Stop
564 </button>
565 </div>
566 </footer>
567 </div>
568 );
569 }
```

## OutdoorNavigationView.tsx

TypeScript/React | frontend/src/components/OutdoorNavigationView.tsx

1009 lines | 43.9 KB

```
1 /**
2  * @license
```



[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]



```

627   className="w-full pl-10 pr-10 py-2.5 bg-zinc-50 border border-zinc-200 rounded-xl text-sm font-semibold text-zinc-900
628   placeHolder="Search for a location"
629   {searchQuery && (
630     <button
631       onClick={() => { setSearchQuery(''); setSearchResults([]); }}
632       className="absolute right-3 top-1/2 -translate-y-1/2 text-zinc-400 hover:text-zinc-600"
633     >
634       <X className="w-4 h-4" />
635     </button>
636   )}
637 </div>
638 </div>
639
640   {/* Search results dropdown */}
641   {(isSearching || searchResults.length > 0) && (
642     <div className="border-t border-zinc-100 max-h-72 overflow-y-auto">
643       {isSearching && (
644         <div className="flex items-center gap-3 px-5 py-3 text-zinc-500">
645           <Loader2 className="w-4 h-4 animate-spin text-blue-500" />
646           <span className="text-sm font-semibold">Searching..</span>
647         </div>
648       )}
649       {searchResults.map((r, idx) => (
650         <button
651           key={idx}
652           onClick={() => handleSelectDestination(r)}
653           className="w-full flex items-start gap-3 px-4 py-3.5 hover:bg-zinc-50 transition-colors border-b border-zinc-50
654           last:border-b-0"
655         >
656           <div className="w-9 h-9 rounded-xl bg-blue-50 flex items-center justify-center shrink-0 mt-0.5">
657             <MapPin className="w-4 h-4 text-blue-500" />
658           </div>
659           <div className="flex-1 min-w-0">
660             <p className="text-sm font-bold text-zinc-900 truncate">{r.name}</p>
661             <p className="text-xs text-zinc-400 truncate mt-0.5 font-medium">{r.address}</p>
662           </div>
663         </button>
664       )})}
665     </div>
666   )}
667 </div>
668
669   {/* Bottom hint chip */}
670   {!searchQuery && (
671     <div className="absolute bottom-10 left-0 right-0 flex justify-center z-10 px-6 pointer-events-none">
672       <div className="bg-white/95 backdrop-blur-sm rounded-2xl px-5 py-3.5 shadow-xl border border-zinc-100 flex items-center
673       gap-2">
674         <MapPin className="w-4 h-4 text-blue-600" />
675       </div>
676       <p className="text-sm font-semibold text-zinc-700">
677         Tap anywhere on the map to set a pin
678       </p>
679     </div>
680   )}
681 </div>
682 </div>
683 </div>
684
685   {/* ===== PREVIEW PHASE ===== */}
686   {/* PREVIEW PHASE */}
687   {/* ===== */}
688   {phase === 'preview' && destination && (
689     <div>
690       {/* Top bar */}
691       <div className="relative z-10 px-3 pt-3">
692         <div className="bg-white rounded-2xl shadow-lg border border-zinc-100">
693           <div className="flex items-center gap-3 px-4 py-3.5">
694             <button
695               onClick={() => {
696                 setPhase('search');
697                 setDestination(null);
698                 setRouteSteps([]);
699                 setRouteSummary(null);
700                 setSearchQuery('');
701                 if (userLocation) setMapCenter(userLocation);
702               }}
703               className="p-2 rounded-full hover:bg-zinc-100 transition-colors shrink-0"
704             >

```

```

705 <ArrowLeft className="w-5 h-5 text-zinc-700" />
706 </button>
707 <div className="flex-1 min-w-0">
708 <p className="text-[10px] font-bold text-blue-600 uppercase tracking-widest">Destination</p>
709 <p className="text-sm font-bold text-zinc-900 truncate mt-0.5">
710 {isReverseGeocoding ? 'Locating place...' : destination.name}
711 </p>
712 </div>
713 <div className="w-9 h-9 rounded-xl bg-red-50 flex items-center justify-center shrink-0">
714 <MapPin className="w-4 h-4 text-red-500" />
715 </div>
716 </div>
717 </div>
718 </div>
719
720 {/* Bottom preview card */}
721 <div className="absolute bottom-0 left-0 right-0 z-10">
722 <div className="bg-white rounded-t-3xl shadow-2xl border-t border-zinc-100/80 p-5 pb-8">
723
724 {isRouteCalculating && (
725 <div className="flex flex-col items-center gap-3 py-6">
726 <Loader2 className="w-9 h-9 text-blue-500 animate-spin" />
727 <p className="text-sm font-semibold text-zinc-500">Calculating best route...</p>
728 </div>
729 )}
730
731 {routeError && !isRouteCalculating && (
732 <div className="flex flex-col items-center gap-3 py-4 text-center">
733 <div className="w-14 h-14 rounded-2xl bg-amber-50 flex items-center justify-center">
734 <AlertTriangle className="w-7 h-7 text-amber-500" />
735 </div>
736 <div>
737 <p className="text-sm font-bold text-zinc-800">Route not available</p>
738 <p className="text-xs text-zinc-400 mt-1 font-medium">
739 Try a different destination or check your network.
740 </p>
741 </div>
742 <button
743 onClick={() => { setPhase('search'); setDestination(null); }}
744 className="px-5 py-2.5 bg-[#002f5c] text-white text-xs font-bold rounded-xl cursor-pointer"
745 >
746 Choose Another Destination
747 </button>
748 </div>
749 )}
750
751 {!isRouteCalculating && !routeError && routeSteps.length > 0 && (
752 <>
753 {/* Stats row */}
754 <div className="flex gap-3 mb-5">
755 <div className="flex-1 bg-blue-50 rounded-2xl p-4 flex flex-col items-center gap-1">
756 <span className="text-xl font-black text-blue-700">
757 {routeSummary ? fmtDist(routeSummary.totalDistanceMeters) : '-'}
758 </span>
759 <span className="text-[11px] font-bold text-blue-400 uppercase tracking-wider">Distance</span>
760 </div>
761 <div className="flex-1 bg-emerald-50 rounded-2xl p-4 flex flex-col items-center gap-1">
762 <span className="text-xl font-black text-emerald-700">
763 {routeSummary ? fmtDuration(routeSummary.totalDurationSeconds) : '-'}
764 </span>
765 <span className="text-[11px] font-bold text-emerald-400 uppercase tracking-wider">Time</span>
766 </div>
767 <div className="flex-1 bg-zinc-50 rounded-2xl p-4 flex flex-col items-center gap-1">
768 <span className="text-xl font-black text-zinc-700">{routeSteps.length}</span>
769 <span className="text-[11px] font-bold text-zinc-400 uppercase tracking-wider">Steps</span>
770 </div>
771 </div>
772
773 {/* First 3 steps preview */}
774 <div className="space-y-2 mb-5">
775 {routeSteps.slice(0, 3).map((step, idx) => (
776 <div
777 key={idx}
778 className="flex items-center gap-3 bg-zinc-50 border border-zinc-100 rounded-xl px-3 py-2.5"
779 >
780 <div className="w-8 h-8 rounded-lg bg-[#002f5c] flex items-center justify-center text-white shrink-0">
781 <DirectionArrow direction={step.direction} className="w-4 h-4" />
782 </div>

```







```

3  * SPDX-License-Identifier: Apache-2.0
4  */
5
6  import React, { useState, useEffect } from 'react';
7  import { ArrowLeft, Clock, CheckCircle2, ShieldAlert, Coffee, HelpCircle, Navigation, Info, User, Check, Footprints,
8  ArrowUp, OnNavigationMade, RouteOption } from '../types';
9  import { getRoute } from '../services/api';
10
11 interface RouteOptionsViewProps {
12   onBack: () => void;
13   onStartRoute: (routeId: string) => void;
14   originName?: string;
15   destName?: string;
16   currentProfileMode?: NavigationMode;
17   themeMode?: 'light' | 'dark';
18 }
19
20 export default function RouteOptionsView({
21   onBack,
22   onStartRoute,
23   originName = 'Library',
24   destName = 'Science Hall',
25   currentProfileMode = 'standard',
26   themeMode = 'light'
27 }: RouteOptionsViewProps) {
28   const isWheelchairMode = currentProfileMode === 'wheelchair';
29   const [selectedRouteId, setSelectedRouteId] = useState<string>(
30     isWheelchairMode ? 'wheelchair-safe' : 'route-optimal'
31   );
32   const [assistantMessage, setAssistantMessage] = useState<string>('');
33
34   // Fetch real routes from backend
35   const [routes, setRoutes] = useState<RouteOption[]>([]);
36   const [loading, setLoading] = useState(true);
37
38   useEffect(() => {
39     setLoading(true);
40     getRoute(originName, destName, currentProfileMode)
41       .then(data => {
42         setRoutes(data);
43         if (data.length > 0) {
44           setSelectedRouteId(data[0].id);
45         }
46       })
47       .catch(() => {
48         // Route fetch failed, display fallback routes
49       })
50       .finally(() => setLoading(false));
51   }, [originName, destName, currentProfileMode]);
52
53   const handleAssistantQuery = (query: string) => {
54     switch (query) {
55       case 'coffee':
56         setAssistantMessage('The Hub Cafe is located inside Engineering Block A Lobby, directly along your route. The counter
57         is already ramped.');

```

```

81 <button
82   aria-label="Go Back"
83   onClick={onBack}
84   className="p-1.5 hover:bg-zinc-200 rounded-full transition-colors"
85 >
86 <ArrowLeft className="w-5 h-5 text-zinc-850" />
87 </button>
88 <h1 className="text-xl font-black text-[#0f2942]">
89   Route to Room 204
90 </h1>
91 </div>
92 <button className="w-8 h-8 rounded-full border border-zinc-350 flex items-center justify-center text-zinc-650
93   hover:text-zinc-800" />
94 </button>
95 </header>
96 ) : (
97 <header className="flex justify-between items-center w-full mb-6 pb-3 border-b border-zinc-200">
98 <div className="flex items-center gap-3">
99 <button
100   aria-label="Go Back"
101   onClick={onBack}
102   className="p-1.5 hover:bg-zinc-200 rounded-full transition-colors"
103 >
104 <ArrowLeft className="w-5 h-5 text-zinc-850" />
105 </button>
106 <h1 className="text-lg font-bold text-zinc-900 truncate max-w-[280px]">
107   {originName} to {destName}
108 </h1>
109 </div>
110 </header>
111 )}
112
113 {/* Assistant dynamic notification overlay */}
114 {assistantMessage && (
115 <div className="mb-4 bg-blue-50 border border-blue-250 rounded-xl p-3 flex items-start gap-2.5 shadow-sm relative">
116 <div className="p-1.5 bg-blue-100 rounded-lg text-blue-800">
117 <Info className="w-4 h-4 shrink-0" />
118 </div>
119 <div className="flex-1">
120 <p className="text-xs font-bold text-zinc-800">Assistant Note</p>
121 <p className="text-xs text-zinc-650 mt-0.5 leading-relaxed">
122   {assistantMessage}
123 </p>
124 </div>
125 <button
126   onClick={() => setAssistantMessage('')}
127   className="text-zinc-400 hover:text-zinc-600 text-[10px] font-bold px-1.5 py-0.5"
128 >
129   Close
130 </button>
131 </div>
132 )}
133
134 {/* Available Routes or Route Options title */}
135 <h2 className="text-base font-extrabold text-zinc-800 mb-1">
136   {isWheelchairMode ? 'Available Routes' : 'Route Options'}
137 </h2>
138 {!loading && routes.length > 0 && (
139 <p className="text-[10px] font-mono text-zinc-400 mb-4">
140   {routes.length} route{routes.length !== 1 ? 's' : ''} available
141 </p>
142 )}
143
144 {/* Loading state */}
145 {loading && (
146 <div className="flex flex-col items-center justify-center py-16 gap-3">
147 <Loader2 className="w-8 h-8 text-teal-500 animate-spin" />
148 <p className="text-xs font-bold text-zinc-400">Fetching routes from agent...</p>
149 </div>
150 )}
151
152 {!loading && isWheelchairMode ? (
153   /* WHEELCHAIR MODE LAYOUT (Light Mode screenshot sample) */
154 <div className="space-y-4">
155   {/* Card 1: Wheelchair Safe Route (Selected) */}
156 <div
157   onClick={() => setSelectedRouteId('wheelchair-safe')}
158   className={`border rounded-2xl p-4 transition-all relative overflow-hidden flex flex-col gap-3 ${

```

```

159 selectedRouteId === 'wheelchair-safe'
160 ? 'bg-white border-emerald-500 shadow-[0_0_15px_rgba(0,196,154,0.1)]'
161 : 'bg-white border-zinc-200 opacity-80'
162 }}
163 >
164 {selectedRouteId === 'wheelchair-safe' && (
165 <div className="absolute left-0 top-0 bottom-0 w-1 bg-emerald-500" />
166 )}
167
168 <div className="flex justify-between items-start">
169 <div className="flex gap-3">
170 {/* Left wheelchair icon circle */}
171 <div className="w-10 h-10 rounded-full bg-emerald-100 text-emerald-600 flex items-center justify-center shrink-0">
172 <CheckCircle2 className="w-5 h-5" />
173 </div>
174 <div>
175 <h3 className="font-extrabold text-zinc-900 text-sm">
176 Wheelchair Safe Route
177 </h3>
178 <div className="flex items-center gap-1 text-[11px] text-emerald-650 font-bold mt-0.5">
179 <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />
180 <span>Optimal Path</span>
181 </div>
182 </div>
183 </div>
184
185 {/* Right duration badge */}
186 <div className="bg-[#f0effa] text-zinc-850 text-xs font-bold py-1.5 px-3 rounded-full flex items-center gap-1 shrink-0">
187 <Clock className="w-3.5 h-3.5 text-zinc-500" />
188 <span>{routes[0]?.estMinutes || 4} min</span>
189 </div>
190 </div>
191
192 {/* Badges */}
193 <div className="flex flex-wrap gap-1.5">
194 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full flex
195 items-center gap-1">
196 <span>Uses Lift 2</span>
197 </span>
198 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full flex
199 items-center gap-1">
200 <span>Step-free</span>
201 </span>
202 </div>
203
204 {/* Buttons inside selected Card 1 */}
205 {selectedRouteId === 'wheelchair-safe' && (
206 <div className="flex flex-col gap-2 mt-2">
207 <button
208 onClick={(e) => {
209 e.stopPropagation();
210 onStartRoute('wheelchair-safe');
211 }}
212 className="bg-[#006d51] hover:bg-[#005740] text-white font-bold text-xs py-3 px-4 rounded-xl flex items-center justify-center">
213 <Navigation className="w-3.5 h-3.5 fill-current" />
214 Start Guidance
215 </button>
216 <button
217 onClick={(e) => {
218 e.stopPropagation();
219 setAssistantMessage('Map preview loaded. Elevator access is highlighted on the floor plan.');
```



```

237 : 'bg-white border-zinc-200 opacity-80'
238 }'}
239 >
240 {selectedRouteId === 'standard-route' && (
241 <div className="absolute left-0 top-0 bottom-0 w-1 bg-emerald-500" />
242 )}
243
244 <div className="flex justify-between items-start">
245 <div className="flex gap-3">
246 {/* Left walking icon circle */}
247 <div className="w-10 h-10 rounded-full bg-zinc-100 text-zinc-500 flex items-center justify-center shrink-0">
248 <Footprints className="w-5 h-5" />
249 </div>
250 <div>
251 <h3 className="font-extrabold text-zinc-900 text-sm">
252 Standard Route
253 </h3>
254 <div className="bg-red-50 text-red-650 border border-red-500/10 text-[10px] font-bold px-2 py-0.5 rounded mt-1
255 isShieldAlert className="w-3.5 h-3.5" />
256 Involves Stairs
257 </div>
258 </div>
259 </div>
260
261 {/* Right duration badge */}
262 <div className="bg-[#f0effa] text-zinc-850 text-xs font-bold py-1.5 px-3 rounded-full flex items-center gap-1 shrink-0">
263 <Clock className="w-3.5 h-3.5 text-zinc-500" />
264 <span>{routes[1]?.estMinutes || 2} min</span>
265 </div>
266 </div>
267
268 {/* If selected standard route */}
269 {selectedRouteId === 'standard-route' && (
270 <div className="flex flex-col gap-2 mt-2">
271 <button
272 onClick={(e) => {
273 e.stopPropagation();
274 onStartRoute('standard-route');
275 }}
276 className="bg-primary hover:bg-[#002f5c] text-white font-bold text-xs py-3 px-4 rounded-xl flex items-center
277 justify-center ...
278 <Navigation className="w-3.5 h-3.5" />
279 Start Guidance
280 </button>
281 </div>
282 )}
283 </div>
284
285 {/* Quick Assistance Section */}
286 <section className="pt-4">
287 <h3 className="text-xs font-extrabold text-zinc-450 uppercase tracking-widest mb-3">
288 Quick Assistance
289 </h3>
290 <div className="flex flex-col gap-2">
291 <button
292 onClick={() => handleAssistantQuery('lift')}
293 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-3 px-4 rounded-full
294 flex it...
295 <HelpCircle className="w-4 h-4 text-blue-500 shrink-0" />
296 Where is nearest lift?
297 </button>
298 <button
299 onClick={() => handleAssistantQuery('obstacle')}
300 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-3 px-4 rounded-full
301 flex it...
302 <AlertTriangle className="w-4 h-4 text-red-500 shrink-0" />
303 Report obstacle
304 </button>
305 <button
306 onClick={() => handleAssistantQuery('toilet')}
307 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-3 px-4 rounded-full
308 flex it...
309 <span className="text-teal-500 shrink-0 text-sm">
310 <CheckCircle2 className="w-4 h-4" />
311 </span>
312 Find accessible toilet
313 </button>
314 </div>

```

```

315 </section>
316 </div>
317 ) : (
318 /* STANDARD MODE LAYOUT (Dark Mode screenshot sample) */
319 <div className="space-y-4">
320 /* Card 1: Most Direct (Selected) */
321 <div
322   onClick={() => setSelectedRouteId('route-optimal')}
323   className={`border rounded-2xl p-4 transition-all relative overflow-hidden flex flex-col gap-3 ${
324     selectedRouteId === 'route-optimal'
325     ? 'bg-white border-teal-500 shadow-[0_0_15px_rgba(0,196,154,0.1)]'
326     : 'bg-white border-zinc-200 opacity-80'
327   }`}
328   >
329     {selectedRouteId === 'route-optimal' && (
330       <div className="absolute left-0 top-0 bottom-0 w-1 bg-teal-500" />
331     )}
332
333     <div className="flex justify-between items-start">
334       <div className="flex gap-3">
335         <div className="w-10 h-10 rounded-full bg-emerald-50 text-emerald-600 flex items-center justify-center shrink-0">
336           <Navigation className="w-5 h-5 rotate-45" />
337         </div>
338         <div>
339           <h3 className="font-extrabold text-zinc-900 text-sm">
340             Most Direct
341           </h3>
342           <p className="text-xs text-zinc-500 mt-1 font-semibold">
343             Est. {routes[0]?.estMinutes || 5} mins • {routes[0]?.distanceMiles || 0.2} miles
344           </p>
345         </div>
346       </div>
347
348       {/* Selected badge */}
349       <span className="bg-emerald-500 text-white border border-emerald-500/10 font-sans font-bold text-[10px] px-2.5 py-1 rounded-6" />
350       <span className="w-3.5 h-3.5 stroke-[3px]" /> Selected
351     </span>
352   </div>
353
354   {/* Button inside Card 1 */}
355   {selectedRouteId === 'route-optimal' && (
356     <div className="mt-2">
357       <button
358         onClick={(e) => {
359           e.stopPropagation();
360           onStartRoute('route-optimal');
361         }}
362         className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-xs flex items-center justify-center gap-2 shadow-sm
363           ${themeMode === 'light'
364             ? 'bg-[#002f5c] hover:bg-[#002447]'
365             : 'bg-[#1b3a5f] hover:bg-[#152e4c]'
366           }`}
367         >
368         <Navigation className="w-3.5 h-3.5" />
369         Start Navigation
370       </button>
371     </div>
372   )}
373 </div>
374
375 {/* Card 2: Accessible Route */}
376 <div
377   onClick={() => setSelectedRouteId('route-accessible')}
378   className={`border rounded-2xl p-4 transition-all relative overflow-hidden flex flex-col gap-3 ${
379     selectedRouteId === 'route-accessible'
380     ? 'bg-white border-teal-500 shadow-[0_0_15px_rgba(0,196,154,0.1)]'
381     : 'bg-white border-zinc-200 opacity-80'
382   }`}
383   >
384     {selectedRouteId === 'route-accessible' && (
385       <div className="absolute left-0 top-0 bottom-0 w-1 bg-teal-500" />
386     )}
387
388     <div className="flex justify-between items-start">
389       <div className="flex gap-3">
390         <div className="w-10 h-10 rounded-full bg-zinc-150 text-zinc-550 flex items-center justify-center shrink-0">
391           <Navigation className="w-5 h-5 rotate-45" />
392         </div>

```

```
393 <div>
394 <h3 className="font-extrabold text-zinc-900 text-sm">
395 Accessible Route
396 </h3>
397 <p className="text-xs text-zinc-500 mt-1 font-semibold">
398 Est. {routes[1]?.estMinutes || 8} mins • {routes[1]?.distanceMiles || 0.3} miles
399 </p>
400 </div>
401 </div>
402 </div>
403
404 {/* Badges */}
405 <div className="flex flex-wrap gap-1.5 mt-1">
406 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full">
407 Elevators
408 </span>
409 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full">
410 Ramps
411 </span>
412 </div>
413
414 {selectedRouteId === 'route-accessible' && (
415 <div className="mt-2">
416 <button
417 onClick={{e} => {
418 e.stopPropagation();
419 onStartRoute('route-accessible');
420 }}
421 className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-xs flex items-center justify-center gap-2 shadow-sm
422 themeMode === 'light'
423 ? 'bg-[#002f5c] hover:bg-[#002447]'
424 : 'bg-[#1b3a5f] hover:bg-[#152e4c]'
425 }}`
426 >
427 <Navigation className="w-3.5 h-3.5" />
428 Start Navigation
429 </button>
430 </div>
431 )}
432 </div>
433
434 {/* Ask Assistant Section */}
435 <section className="pt-4">
436 <h3 className="text-xs font-extrabold text-zinc-450 uppercase tracking-widest mb-3">
437 Ask Assistant
438 </h3>
439 <div className="flex flex-wrap gap-2">
440 <button
441 onClick={() => handleAssistantQuery('coffee')}
442 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-2.5 px-4
443 rounded-full flex ...
444 <Coffee className="w-4 h-4 text-emerald-500" />
445 Coffee on route?
446 </button>
447 <button
448 onClick={() => handleAssistantQuery('toilet')}
449 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-2.5 px-4
450 rounded-full flex ...
451 <HelpCircle className="w-4 h-4 text-blue-500" />
452 Nearest restroom?
453 </button>
454 <button
455 onClick={() => handleAssistantQuery('stairs')}
456 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-2.5 px-4
457 rounded-full flex ...
458 <ShieldAlert className="w-4 h-4 text-amber-500" />
459 Avoid stairs
460 </button>
461 </div>
462 </section>
463 </div>
464 )}
465 </div>
466 );
467 }
```

264 lines | 10.8 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useState, useEffect, useRef } from 'react';
7  import { Camera, CameraOff, QrCode, Keyboard, AlertCircle, Info, RefreshCw } from 'lucide-react';
8  import { lookupQR } from '../services/api';
9
10 interface QRScannerProps {
11   onScanSuccess: (data: any) => void;
12   onCancel: () => void;
13 }
14
15 export default function QRScanner({ onScanSuccess, onCancel }: QRScannerProps) {
16   const [stream, setStream] = useState<MediaStream | null>(null);
17   const [cameraActive, setCameraActive] = useState(false);
18   const [cameraError, setCameraError] = useState<string | null>(null);
19
20   const [manualCode, setManualCode] = useState('');
21   const [showManualInput, setShowManualInput] = useState(false);
22   const [loading, setLoading] = useState(false);
23   const [errorMsg, setErrorMsg] = useState<string | null>(null);
24
25   const videoRef = useRef<HTMLVideoElement>(null);
26   const localStreamRef = useRef<MediaStream | null>(null);
27
28   const startCamera = async () => {
29     try {
30       setCameraError(null);
31
32       // Stop any existing stream
33       if (localStreamRef.current) {
34         localStreamRef.current.getTracks().forEach(track => track.stop());
35       }
36
37       if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) {
38         throw new Error('Camera access API is not supported in this browser environment or is blocked by security policies');
39       }
40
41       const mediaStream = await navigator.mediaDevices.getUserMedia({
42         video: { facingMode: { ideal: 'environment' } }
43       });
44
45       localStreamRef.current = mediaStream;
46       setStream(mediaStream);
47       setCameraActive(true);
48
49       setTimeout(() => {
50         if (videoRef.current) {
51           videoRef.current.srcObject = mediaStream;
52         }
53       }, 100);
54     } catch (err: any) {
55       let errorMessage = err.message || 'Camera access is blocked or unsupported';
56       if (err.name === 'NotAllowedError' || err.name === 'PermissionDeniedError') {
57         errorMessage = 'Camera permission is blocked. Please check your browser permissions and try again.';
58       }
59       setCameraError(errorMessage);
60       setCameraActive(false);
61     }
62   };
63
64   const stopCamera = () => {
65     if (localStreamRef.current) {
66       localStreamRef.current.getTracks().forEach(track => track.stop());
67       localStreamRef.current = null;
68     }
69     setStream(null);
70     setCameraActive(false);
71   };
72
73   // Request camera access on mount
74   useEffect(() => {
75     startCamera();
76     return () => {
```

```

77   stopCamera();
78   };
79   }, []);
80
81   // Handle manual code or dropdown code lookup
82   const handleVerifyCode = async (codeToVerify: string) => {
83     if (!codeToVerify.trim()) return;
84     setLoading(true);
85     setErrorMsg(null);
86     try {
87       const result = await lookupQR(codeToVerify.trim().toUpperCase());
88       onScanSuccess(result);
89     } catch (err: any) {
90       setErrorMsg(err.message || 'Verification failed. Please check the checkpoint code.');
```

```

91     } finally {
92       setLoading(false);
93     }
94   };
95
96   return (
97     <div className="relative w-full h-[calc(100vh-76px)] bg-zinc-950 flex flex-col justify-between overflow-hidden
98     text-████████████████████ OR SIMULATED CORRIDOR VIEW ██████ */>
99     <div className="absolute inset-0 z-0">
100       {cameraActive ? (
101         <video
102           ref={videoRef}
103           autoPlay
104           playsInline
105           muted
106           className="w-full h-full object-cover opacity-80"
107         />
108       ) : (
109         <div className="relative w-full h-full bg-zinc-900/90 flex flex-col items-center justify-center p-6">
110           /* Camera unavailable fallback view */
111           <div className="relative z-10 text-center max-w-xs space-y-3 p-4 rounded-2xl bg-zinc-950/80 border border-zinc-850
112           backdrop-blur-md className="w-10 h-10 text-zinc-500 mx-auto animate-pulse" />
113           <h3 className="text-sm font-bold text-zinc-300">Camera Unavailable</h3>
114           <p className="text-[11px] text-zinc-500 font-semibold leading-normal">
115             {cameraError ? `${cameraError}.` : 'Camera permission is required.'} Please enable camera access to continue.
116           </p>
117
118           <button
119             type="button"
120             onClick={startCamera}
121             className="w-full mt-2 py-2.5 bg-teal-600 hover:bg-teal-700 text-white font-bold text-xs uppercase tracking-wider
122             rounded-xl t...
123             Enable Camera Scanner
124           </button>
125         </div>
126       </div>
127     )}
128   </div>
129
130   /* █████ TOP BAR HUD INFO █████ */
131   <div className="relative z-10 w-full px-4 pt-6 flex justify-center">
132     <div className="inline-flex items-center gap-2.5 bg-black/80 backdrop-blur-md rounded-full px-5 py-2.5 border
133     border-0.5px border-teal-400 text-teal-400 shrink-0" />
134     <span className="text-[11px] font-bold tracking-wide">
135       Align QR code or use the quick simulation dashboard below
136     </span>
137   </div>
138 </div>
139
140   /* █████ VIEW FINDER BRACKET SIGHTS █████ */
141   <div className="relative z-10 flex-grow flex items-center justify-center p-8 select-none">
142     <div className="relative w-64 h-64 flex items-center justify-center">
143       /* Viewfinder corner brackets */
144       <div className="absolute top-0 left-0 w-10 h-10 border-t-[4px] border-l-[4px] border-teal-400 rounded-tl-2xl"></div>
145       <div className="absolute top-0 right-0 w-10 h-10 border-t-[4px] border-r-[4px] border-teal-400 rounded-tr-2xl"></div>
146       <div className="absolute bottom-0 left-0 w-10 h-10 border-b-[4px] border-l-[4px] border-teal-400 rounded-bl-2xl"></div>
147       <div className="absolute bottom-0 right-0 w-10 h-10 border-b-[4px] border-r-[4px] border-teal-400 rounded-br-2xl"></div>
148
149       /* Laser scanning line */
150       <div
151         className="absolute top-0 left-0 w-full h-[3px] bg-teal-400 shadow-[0_0_12px_#2dd4bf] rounded-full"
152         style={{
153           animation: 'scan 2.5s ease-in-out infinite',
154           animationName: 'qrScanMove'

```

```

155     }}
156     />
157
158     <style>{`
159     @keyframes qrScanMove {
160     0%, 100% { top: 0%; opacity: 0.3; }
161     50% { top: 100%; opacity: 1; }
162     }
163     `}</style>
164
165
166 </div>
167 </div>
168
169 /* ■■■■ BOTTOM CONTROL INTERFACE ■■■■ */
170 <div className="relative z-10 px-4 pb-28 max-w-md mx-auto w-full">
171 /* Error Dialog Banner */
172 {errorMsg && (
173 <div className="bg-red-950/80 border border-red-500/30 backdrop-blur-md rounded-2xl p-3 mb-3 text-center text-xs
174 font-bolder text-white">
175 <span>{errorMsg}</span>
176 </div>
177 )}
178
179 <div className="bg-zinc-900/90 backdrop-blur-lg rounded-3xl p-5 shadow-2xl border border-zinc-800/50 space-y-4">
180
181 {showManualInput ? (
182 /* Manual Input form */
183 <div className="space-y-3 text-center">
184 <h4 className="text-xs font-extrabold text-zinc-400 uppercase tracking-widest">
185 Manual Checkout Entry
186 </h4>
187 <div className="flex gap-2 justify-center">
188 <input
189 type="text"
190 placeholder="e.g. QR-ENG-G01"
191 value={manualCode}
192 onChange={(e) => setManualCode(e.target.value)}
193 className="bg-zinc-950 border border-zinc-800 text-center font-mono focus:border-teal-500 focus:ring-1
194 focus:ring-teal-500 o...
195 </div>
196 <div className="flex gap-2 pt-1">
197 <button
198 onClick={() => handleVerifyCode(manualCode)}
199 disabled={loading}
200 className="flex-1 bg-teal-600 hover:bg-teal-700 text-white font-bold py-3 rounded-xl text-xs uppercase tracking-wider
201 transi...
202 Verify Code
203 </button>
204 <button
205 onClick={() => {
206 setShowManualInput(false);
207 setErrorMsg(null);
208 }}
209 className="px-5 bg-zinc-800 hover:bg-zinc-750 text-zinc-300 font-bold py-3 rounded-xl text-xs uppercase tracking-wider
210 transi...
211 Cancel
212 </button>
213 </div>
214 </div>
215 ) : (
216 /* Simulator dropdown selector & toggles */
217 <div className="space-y-4">
218 <div className="space-y-2">
219 <label className="text-[10px] font-black uppercase text-zinc-400 tracking-widest block text-left">
220 Checkpoint Verification
221 </label>
222 <div className="grid grid-cols-1 gap-2">
223 <input
224 type="text"
225 placeholder="Enter checkpoint code"
226 value={manualCode}
227 onChange={(e) => setManualCode(e.target.value.toUpperCase())}
228 disabled={loading}
229 className="w-full bg-zinc-950 border border-zinc-800 text-zinc-300 font-sans font-mono px-4 py-3 rounded-xl text-xs
230 font-b...
231 <button
232 onClick={() => handleVerifyCode(manualCode)}

```

```
233   disabled={loading || !manualCode.trim()}
234   className="w-full bg-teal-600 hover:bg-teal-700 disabled:opacity-50 text-white font-bold py-3 rounded-xl text-xs
235   uppercase...
236   {loading ? 'Verifying...' : 'Verify Checkpoint'}
237 </button>
238 </div>
239 </div>
240
241 <div className="flex justify-between items-center border-t border-zinc-850 pt-3.5">
242   <button
243     onClick={() => setShowManualInput(true)}
244     className="text-xs font-extrabold text-teal-400 hover:text-teal-300 cursor-pointer flex items-center gap-1.5"
245     >
246     <Keyboard className="w-4 h-4" />
247     Manual Entry
248   </button>
249
250   <button
251     onClick={onCancel}
252     className="text-xs font-bold text-zinc-400 hover:text-white cursor-pointer"
253     >
254     Cancel Scan
255   </button>
256 </div>
257 </div>
258 )}
259
260 </div>
261 </div>
262 </div>
263 );
264 }
```

**SOSScreen.tsx**

TypeScript/React | frontend/src/components/SOSScreen.tsx

21 lines | 0.4 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React from 'react';
7  import SosAlertView from './SosAlertView';
8
9  interface SOSScreenProps {
10   onCancel: () => void;
11   lastKnownLocation?: string;
12 }
13
14 export default function SOSScreen({ onCancel, lastKnownLocation }: SOSScreenProps) {
15   return (
16     <SosAlertView
17       onCancel={onCancel}
18       lastKnownLocation={lastKnownLocation}
19     />
20   );
21 }
```

**SosAlertView.tsx**

TypeScript/React | frontend/src/components/SosAlertView.tsx

127 lines | 5.9 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useState, useEffect } from 'react';
7  import { AlertTriangle, Send, Bluetooth, Wifi, MapPin, MessageSquare } from 'lucide-react';
8
9  interface SosAlertViewProps {
10   onCancel: () => void;
11   lastKnownLocation?: string;
12 }
13
14 export default function SosAlertView({
15   onCancel,
```

```

16   lastKnownLocation = 'Engineering Block A – Ground Floor Entrance'
17   }: SosAlertViewProps) {
18     const [statusMessage, setStatusMessage] = useState('');
19
20     const handleSendSms = () => {
21       setStatusMessage('Initiating emergency alert transmission to Campus Security...');
22       setTimeout(() => setStatusMessage('Emergency alert submitted. Campus Security has been notified.'), 1500);
23     };
24
25     const handleBroadcastMesh = () => {
26       setStatusMessage('Initiating emergency beacon broadcast to connected devices...');
27       setTimeout(() => setStatusMessage('Emergency beacon has been broadcast to connected campus devices.'), 1500);
28     };
29
30     return (
31       <div className="fixed inset-0 bg-[#0d0000] text-white flex flex-col font-sans select-none overflow-y-auto pb-12
32       z-[9999] pointer-events-none" />
33       <div
34         className="absolute inset-0 opacity-[0.02] pointer-events-none"
35         style={{
36           backgroundImage: `url("data:image/svg+xml,%3Csvg viewBox='0 0 200 200' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter
37           id='noise'%3E%
38           />
39
40       /* Header */
41       <header className="w-full px-6 pt-8 pb-4 flex justify-center items-center z-10 relative">
42         <h1 className="text-xl font-black font-sans tracking-widest text-[#ef4444] flex items-center gap-3 uppercase
43         animate-pulse className="w-5 h-5 text-[#ef4444]" />
44         SOS ACTIVE
45         <AlertTriangle className="w-5 h-5 text-[#ef4444]" />
46       </h1>
47     </header>
48
49     /* Main beacon visualizer */
50     <main className="flex-1 flex flex-col items-center justify-center px-6 z-10 relative max-w-md mx-auto w-full">
51       /* Pulsing ring waves – 3 layers */
52       <div className="relative flex items-center justify-center w-64 h-64 mb-10">
53         /* Outermost slow pulse */
54         <div className="absolute inset-0 rounded-full bg-red-600 opacity-10 animate-ping" style={{ animationDuration: '1.8s' }} />
55         /* Mid ring */
56         <div className="absolute inset-6 rounded-full border border-red-600/30 animate-ping" style={{ animationDuration: '1.2s' }} />
57         /* Inner glow */
58         <div className="absolute inset-12 rounded-full bg-red-900/30 animate-pulse" />
59
60       /* Central SOS circle */
61       <div className="relative w-40 h-40 bg-[#c01e1e] rounded-full flex flex-col items-center justify-center
62       shadow-[0_0_0px_1px_gba(100% 0% 0%)] text-4xl font-black tracking-[0.15em] leading-none select-none">SOS</div>
63       <span className="text-red-200 text-[10px] font-bold tracking-widest uppercase leading-none select-none">Emergency</span>
64     </div>
65
66     /* Temporary dynamic transmission feedback message */
67     {statusMessage && (
68       <div className="w-full bg-stone-900 border border-red-500/30 rounded-2xl p-4 mb-5 text-center text-xs font-semibold
69       text-[#00f0f0]">
70       {statusMessage}
71     </div>
72   )}
73
74   /* Location Info Card */
75   <div className="w-full bg-[#2a0000]/90 rounded-2xl p-5 border border-red-900/30 mb-6 shadow-xl relative overflow-hidden
76   text-white">
77     <div className="flex items-start gap-4">
78       <MapPin className="w-6 h-6 text-red-500 shrink-0 mt-0.5" />
79       <div className="space-y-3">
80         <div>
81           <h2 className="text-xs font-bold text-red-400 uppercase tracking-widest">
82             Last Known Location
83           </h2>
84           <p className="text-base text-white font-bold leading-snug mt-1">
85             {lastKnownLocation}
86           </p>
87         </div>
88         <div className="flex items-center gap-2 bg-[#3f0f0f] border border-red-500/20 rounded-full px-3 py-1.5 w-fit">
89           <Bluetooth className="w-3.5 h-3.5 text-red-400 fill-current" />
90           <span className="text-[10px] font-bold tracking-wider text-red-300 uppercase">
91             Shared via Bluetooth mesh
92           </span>
93         </div>
94       </div>

```



```
94     </div>
95   </div>
96
97   {/* Action buttons stack */}
98   <div className="w-full flex flex-col gap-3">
99     <button
100       onClick={handleSendSms}
101       className="w-full h-14 bg-[#c82323] hover:bg-[#a81a1a] text-white font-bold rounded-2xl flex items-center
102       justify-center gap-3 sha...
103     <MessageSquare className="w-5 h-5 fill-current" />
104     Send SMS Alert
105   </button>
106   <button
107     onClick={handleBroadcastMesh}
108     className="w-full h-14 bg-[#60f8cb] hover:bg-[#4edab0] text-stone-950 font-bold rounded-2xl flex items-center
109     justify-center gap-3...
110   <Wifi className="w-5 h-5" />
111   Broadcast via Mesh
112 </button>
113 </div>
114 </main>
115
116   {/* Footer actions */}
117   <footer className="w-full px-6 pt-6 flex flex-col items-center gap-6 z-10 relative max-w-md mx-auto">
118     <button
119       onClick={onCancel}
120       className="font-sans font-bold text-sm text-zinc-400 hover:text-white underline underline-offset-4 p-2
121       transition-colors uppercase t...
122     Cancel SOS
123   </button>
124 </footer>
125 </div>
126 );
127 }
```

## ProfileSelector.tsx

TypeScript/React | frontend/src/components/ProfileSelector.tsx

112 lines | 4.1 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6   import React from 'react';
7   import { EyeOff, EarOff, Heart, Brain, Footprints, Info } from 'lucide-react';
8   import { NavigationMode } from '../types';
9   import { PROFILES } from '../data';
10
11   interface ProfileSelectorProps {
12     currentMode: NavigationMode;
13     onSelectMode: (mode: NavigationMode) => void;
14     compact?: boolean;
15   }
16
17   export default function ProfileSelector({
18     currentMode,
19     onSelectMode,
20     compact = false
21   }: ProfileSelectorProps) {
22     // Render icon matching the profile ID
23     const getProfileIcon = (id: NavigationMode, isSelected: boolean) => {
24       const iconClass = `w-6 h-6 transition-transform duration-200 ${
25         isSelected ? 'text-teal-400 scale-110' : 'text-zinc-500 group-hover:scale-105'
26       }`;
27
28       switch (id) {
29         case 'wheelchair':
30           return <span className={iconClass} > <span> </span> </span>;
31         case 'vision':
32           return <EyeOff className={iconClass} />;
33         case 'hearing':
34           return <EarOff className={iconClass} />;
35         case 'cognitive':
36           return <Brain className={iconClass} />;
37         case 'chronic':
38           return <Heart className={iconClass} />;
39         case 'standard':
```

```
40   return <Footprints className={iconClass} />;
41   default:
42     return <Footprints className={iconClass} />;
43   }
44   };
45
46   return (
47     <div className="space-y-4 w-full">
48       {!compact && (
49         <div className="bg-zinc-50 rounded-2xl p-4 border border-zinc-200/80 flex items-start gap-3">
50           <Info className="w-5 h-5 text-[#002f5c] shrink-0 mt-0.5" />
51           <p className="text-xs font-semibold leading-relaxed text-zinc-650">
52             We adapt all pathfinding weights, lift prioritization, and audio warning systems instantly according to your selected
53             profile. No ...
54           </p>
55         </div>
56       )}
57
58       <div className={`grid gap-3 ${compact ? 'grid-cols-2' : 'grid-cols-1 md:grid-cols-2'}`}>
59         {PROFILES.map(profile => {
60           const isSelected = currentMode === profile.id;
61           return (
62             <button
63               key={profile.id}
64               type="button"
65               onClick={() => onSelectMode(profile.id)}
66               className={`group flex text-left p-4 rounded-2xl border transition-all duration-200 cursor-pointer outline-none
67                 ${isSelected ? 'border-[#002f5c] bg-[#002f5c] text-white shadow-md' : 'border-zinc-200 bg-white hover:bg-zinc-50 text-zinc-900'}
68               `}
69             >
70               <div className="absolute left-0 top-0 bottom-0 w-1.5 bg-[#60f8cb]" />
71               <div className="flex gap-4 items-start w-full">
72                 <div className="flex-grow min-w-0">
73                   <h3 className={`font-sans font-bold text-sm leading-tight transition-colors ${
74                     isSelected ? 'text-[#60f8cb]' : 'text-zinc-850'
75                   }`}>
76                     {profile.name}
77                   </h3>
78                   <p className={`text-xs mt-1 leading-snug line-clamp-2 ${
79                     isSelected ? 'text-zinc-300' : 'text-zinc-500 font-medium'
80                   }`}>
81                     {profile.description}
82                   </p>
83                   <div className="mt-2.5 pt-2 border-t border-zinc-800 text-[10px] text-teal-300 font-mono tracking-wide">
84                     {profile.details}
85                   </div>
86                 </div>
87                 <div>
88                   <img alt="Profile icon" />
89                 </div>
90               </div>
91             </button>
92           );
93         });
94       </div>
95     </div>
96   );
97 }
```

```

1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useState } from 'react';
7  import { Bookmark, ShieldCheck, Lock } from 'lucide-react';
8  import { NavigationMode } from '../types';
9  import { PROFILES } from '../data';
10 import ProfileSelector from './ProfileSelector';
11
12 interface AccessibilityViewProps {
13   currentMode: NavigationMode;
14   onSaveMode: (mode: NavigationMode) => void;
15   onClose?: () => void;
16 }
17
18 export default function AccessibilityView({
19   currentMode,
20   onSaveMode,
21   onClose
22 }: AccessibilityViewProps) {
23   const [selectedMode, setSelectedMode] = useState<NavigationMode>(currentMode);
24   const [toastMessage, setToastMessage] = useState(false);
25
26   const handleSave = () => {
27     onSaveMode(selectedMode);
28     setToastMessage(true);
29     setTimeout(() => {
30       setToastMessage(false);
31       if (onClose) {
32         onClose();
33       }
34     }, 1500);
35   };
36
37   return (
38     <div className="w-full max-w-xl mx-auto bg-white flex flex-col rounded-3xl overflow-hidden pb-32">
39       <header className="px-6 pt-6 pb-4 bg-white relative">
40         <h1 className="text-2xl font-bold font-sans tracking-tight text-[#002f5c] mb-2">
41           Choose Your Navigation Mode
42         </h1>
43         <p className="text-sm text-zinc-500 font-medium">
44           No account needed. We personalize routes based on your selection.
45         </p>
46       </header>
47
48       {/* Profile select list */}
49       <div className="px-6 py-3 flex-1 overflow-y-auto pb-6">
50         <ProfileSelector currentMode={selectedMode} onSelectMode={setSelectedMode} />
51       </div>
52
53       {/* Floating feedback alert */}
54       {toastMessage && (
55         <div className="fixed bottom-32 left-1/2 transform -translate-x-1/2 bg-[#002f5c] text-white px-6 py-3 rounded-full flex
56           items-center gap-2">
57           <span className="text-sm font-semibold tracking-wide">
58             {PROFILES.find(p => p.id === selectedMode)?.name} Mode Active!
59           </span>
60         </div>
61       )}
62     </div>
63
64     {/* Fixed bottom action bar */}
65     <div className="fixed bottom-0 left-0 w-full bg-white border-t border-zinc-150 px-6 pt-4 pb-24 z-40 shadow-lg">
66       <button
67         type="button"
68         onClick={handleSave}
69         className="w-full h-14 rounded-2xl bg-[#002f5c] hover:bg-[#002447] text-white font-sans font-bold text-base flex
70         items-center justify-center">
71         <Bookmark className="w-5 h-5 fill-current" />
72         Save Preference
73       </button>
74       <p className="text-center text-xs text-zinc-400 mt-3 flex items-center justify-center gap-1">
75         <Lock className="w-3.5 h-3.5 text-zinc-400" />
76         <span>Preference is stored locally on this device only.</span>
77       </p>
78     </div>

```

```
79 </div>
80 );
81 }
```

## SavedMapView.tsx

TypeScript/React | frontend/src/components/SavedMapView.tsx

231 lines | 10.0 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useState } from 'react';
7  import { Trash2, QRCode, Smartphone, DownloadCloud, Info, Building, MapPin, Compass, Layers } from 'lucide-react';
8  import { SavedMap } from '../types';
9  import { INITIAL_MAPS } from '../data';
10 import { INDOOR_MAPS } from '../indoorMaps';
11
12 interface SavedMapViewProps {
13   onScanClick: () => void;
14   onNavigateToBlockA: () => void;
15 }
16
17 export default function SavedMapView({ onScanClick, onNavigateToBlockA }: SavedMapViewProps) {
18   const [mapsList, setMapsList] = useState<SavedMap[]>(INITIAL_MAPS);
19   const [downloadModalOpen, setDownloadModalOpen] = useState(false);
20   const [searchUrl, setSearchUrl] = useState('');
21   const [downloadSuccess, setDownloadSuccess] = useState(false);
22
23   // Deletion logic
24   const handleDeleteMap = (id: string, e: React.MouseEvent) => {
25     e.stopPropagation();
26     setMapsList(prev => prev.filter(m => m.id !== id));
27   };
28
29   const handleDownloadMap = () => {
30     if (!searchUrl.trim()) return;
31     setDownloadSuccess(true);
32     setTimeout(() => {
33       const newMap: SavedMap = {
34         id: `map-${Date.now()}`,
35         name: searchUrl.trim(),
36         floorsCount: Math.floor(Math.random() * 5) + 3,
37         sizeMb: parseFloat((Math.random() * 3 + 1).toFixed(1)),
38         imageThumbnail: 'https://lh3.googleusercontent.com/aida-public/AB6AXuB4EJiY_W3Qp7ZYtW50TfdAa_bx0iSQin07ai5gxTzts3dZ2Y6J3gXabWggydyVhBvAdemic'
39       };
40     }, 1500);
41     setMapsList(prev => [newMap, ...prev]);
42     setDownloadModalOpen(false);
43     setDownloadSuccess(false);
44     setSearchUrl('');
45   };
46
47   const handleRestoreDefaultMaps = () => {
48     setMapsList(INITIAL_MAPS);
49   };
50
51   const renderMapOverlayIcon = (id: string) => {
52     const defaultClass = "w-6 h-6 text-[#002f5c]";
53     if (id === 'building-1-tech') {
54       return <Building className={defaultClass} />;
55     }
56     return <MapPin className={defaultClass} />;
57   };
58
59   return (
60     <div className="w-full max-w-xl mx-auto px-6 py-6 pb-[100px] bg-white min-h-screen text-left">
61       {/* Upper header */}
62       <header className="mb-6">
63         <div className="flex items-center gap-2 mb-4">
64           <Compass className="w-6 h-6 text-[#002f5c] animate-spin-slow rotate-45" />
65           <span className="font-extrabold font-sans text-xl tracking-tight text-[#002f5c]">
66             CampusPilot AI
67           </span>
68         </div>
69       </header>
70       <h2 className="text-2xl font-bold font-sans tracking-tight text-[#002f5c] mb-2">
```

```

71 Saved Building Maps
72 </h2>
73 <p className="text-sm text-zinc-500 font-medium">
74 Access your downloaded maps offline for quick navigation.
75 </p>
76 </header>
77
78 {/* Indoor Navigation CTA */}
79 <button
80 onClick={onNavigateToBlockA}
81 className="w-full mb-6 py-4 bg-gradient-to-r from-[#002f5c] to-[#003d7a] hover:from-[#002447] hover:to-[#003060]
82 text-white font-bold ...
83 <Layers className="w-5 h-5" />
84 <span className="text-sm">Browse All Indoor Maps ({INDOOR_MAPS.length} Buildings)</span>
85 </button>
86
87 {/* Manual download options bar */}
88 <div className="flex gap-2 mb-6 justify-between items-center">
89 <button
90 onClick={() => setDownloadModalOpen(true)}
91 className="px-4 py-2 text-xs bg-[#002f5c] hover:bg-[#002447] text-white font-bold rounded-xl flex items-center gap-2
92 shadow-sm trans...
93 <DownloadCloud className="w-4 h-4" />
94 Download New Map
95 </button>
96 {mapsList.length === 0 && (
97 <button
98 onClick={handleRestoreDefaultMaps}
99 className="px-4 py-2 text-xs border border-zinc-200 hover:bg-zinc-50 rounded-xl text-zinc-650 font-bold transition-all"
100 >
101 Reset Default Maps
102 </button>
103 )}
104 </div>
105
106 {/* List view of maps */}
107 {mapsList.length > 0 ? (
108 <div className="flex flex-col gap-5 mb-8" id="mapsContainer">
109 {mapsList.map(map => (
110 <div
111 key={map.id}
112 onClick={() => onNavigateToBlockA()}
113 className="bg-white rounded-2xl border border-zinc-200 overflow-hidden flex flex-col group hover:shadow-md
114 transition-all cursor...
115 {/* Map Blueprint Preview */}
116 <div className="h-44 bg-zinc-50 relative overflow-hidden flex items-center justify-center">
117 <img
118 alt="Blueprint thumbnail"
119 className="w-full h-full object-cover opacity-35 transition-transform duration-300 group-hover:scale-[1.02]"
120 src={map.imageThumbnail}
121 referrerPolicy="no-referrer"
122 />
123 {/* Central Overlay Badge Circle */}
124 <div className="absolute inset-0 flex items-center justify-center bg-blue-900/5">
125 <div className="p-3 bg-white rounded-full shadow-md border-2 border-[#002f5c] group-hover:scale-105
126 transition-all cursor-pointer">
127 </div>
128 </div>
129 </div>
130
131 {/* Text Info Section */}
132 <div className="p-4 flex justify-between items-center bg-white border-t border-zinc-100">
133 <div className="flex-1 min-w-0 pr-2">
134 <h3 className="font-sans font-bold text-base text-[#002f5c] truncate">
135 {map.name}
136 </h3>
137 <p className="text-xs text-zinc-500 mt-1 font-medium">
138 {map.floorsCount} floors saved · {map.sizeMb} MB
139 </p>
140 </div>
141 <button
142 aria-label="Delete map"
143 onClick={e => handleDeleteMap(map.id, e)}
144 className="p-2.5 text-red-500 hover:bg-red-50 rounded-full transition-colors shrink-0"
145 >
146 <Trash2 className="w-5 h-5" />
147 </button>
148 </div>

```

```

149 </div>
150 ))}
151 </div>
152 ) : (
153 /* Empty State */
154 <div className="flex flex-col items-center justify-center text-center p-8 bg-zinc-50 rounded-2xl border border-zinc-200
155 border-zinc-200" style={{border: '2px solid #ccc'}}>
156 <QrCode className="w-10 h-10 text-[#002f5c]" />
157 <span className="absolute -bottom-1 -right-1 bg-white rounded-full p-2 border border-zinc-200 shadow-sm text-teal-500">
158 <Smartphone className="w-4 h-4" />
159 </span>
160 </div>
161 <h3 className="font-sans font-bold text-lg text-[#002f5c] mb-1">
162 No maps saved yet
163 </h3>
164 <p className="text-xs text-zinc-550 max-w-xs mb-5 leading-relaxed">
165 Scan a QR code at any building entrance to save its map offline, or search to download one manually.
166 </p>
167 <div className="flex flex-col gap-2 w-full max-w-[200px] justify-center">
168 <button
169 onClick={onScanClick}
170 className="bg-teal-500 hover:bg-teal-600 text-white font-bold py-2.5 rounded-full flex items-center justify-center
171 gap-2 shadow-..."
172 <QrCode className="w-4 h-4" />
173 Scan QR Code
174 </button>
175 <button
176 onClick={() => setDownloadModalOpen(true)}
177 className="border border-zinc-250 hover:bg-zinc-100 text-zinc-700 font-bold py-2.5 rounded-full flex items-center
178 justify-center..."
179 <DownloadCloud className="w-4 h-4" />
180 Download List
181 </button>
182 </div>
183 </div>
184 )}
185
186 /* Manual download modal */
187 {downloadModalOpen && (
188 <div className="fixed inset-0 bg-black/60 backdrop-blur-sm z-[999] flex items-center justify-center p-4">
189 <div className="bg-white rounded-2xl max-w-sm w-full border border-zinc-200 shadow-2xl overflow-hidden p-6 relative">
190 <h3 className="text-lg font-bold text-[#002f5c] mb-2">
191 Download Offline Map
192 </h3>
193 <p className="text-xs text-zinc-500 mb-4 leading-relaxed">
194 Enter a building name or research lab identifier to bundle & save its floor map directory locally.
195 </p>
196
197 <div className="space-y-3 mb-6">
198 <input
199 type="text"
200 placeholder="e.g. Life Sciences Center"
201 value={searchUrl}
202 onChange={(e) => setSearchUrl(e.target.value)}
203 className="w-full px-4 py-2.5 bg-zinc-50 border border-zinc-200 focus:outline-none focus:ring-1 focus:ring-[#002f5c]
204 rounded-x..."
205 <p className="text-[10px] text-zinc-450 flex items-center gap-1.5 leading-normal">
206 <Info className="w-3.5 h-3.5 text-blue-500 shrink-0" />
207 These maps persist completely offline to prevent navigation blackouts.
208 </p>
209 </div>
210
211 <div className="flex gap-2 justify-end text-xs">
212 <button
213 onClick={() => setDownloadModalOpen(false)}
214 className="px-4 py-2 border border-zinc-200 rounded-xl text-zinc-650 font-bold hover:bg-zinc-55"
215 >
216 Cancel
217 </button>
218 <button
219 onClick={handleDownloadMap}
220 disabled={downloadSuccess}
221 className="px-4 py-2 bg-teal-500 hover:bg-teal-600 disabled:bg-zinc-300 text-white font-bold rounded-xl"
222 >
223 {downloadSuccess ? 'Downloading bundle...' : 'Save Offline'}
224 </button>
225 </div>
226 </div>

```

```
227     </div>
228   )}
229 </div>
230 );
231 }
```

## TypeScript/React | frontend/src/components/BottomSheet.tsx

[illegible]

```

69   if (!dragging.current) return;
70   dragging.current = false;
71   snapTo(liveH.current);
72   }, [snapTo]);
73
74   // Attach global move/end listeners so dragging outside the element still works
75   useEffect(() => {
76     const onMouseMove = (e: MouseEvent) => handleDragMove(e.clientX);
77     const onTouchMove = (e: TouchEvent) => handleDragMove(e.touches[0].clientY);
78     const onMouseUp = () => handleDragEnd();
79     const onTouchEnd = () => handleDragEnd();
80
81     window.addEventListener('mousemove', onMouseMove);
82     window.addEventListener('touchmove', onTouchMove, { passive: true });
83     window.addEventListener('mouseup', onMouseUp);
84     window.addEventListener('touchend', onTouchEnd);
85
86     return () => {
87       window.removeEventListener('mousemove', onMouseMove);
88       window.removeEventListener('touchmove', onTouchMove);
89       window.removeEventListener('mouseup', onMouseUp);
90       window.removeEventListener('touchend', onTouchEnd);
91     };
92   }, [handleDragMove, handleDragEnd]);
93
94   // Recalculate on resize
95   useEffect(() => {
96     const onResize = () => snapTo(liveH.current);
97     window.addEventListener('resize', onResize);
98     return () => window.removeEventListener('resize', onResize);
99   }, [snapTo]);
100
101   return (
102     <div
103       style={{
104         height: heightPx,
105         transition: snapping ? 'height 0.35s cubic-bezier(0.32,0.72,0,1)' : 'none',
106       }}
107       className="w-full bg-[#f0f0f8] rounded-t-[28px] shadow-[0_-6px_32px_rgba(0,0,0,0.13)] border-t border-zinc-200 flex
108       flex-col overflow-hi...
109       {/* Drag handle  */}
110       <div
111         className="w-full flex justify-center items-center pt-3 pb-2 shrink-0 cursor-grab active:cursor-grabbing touch-none
112         onMouseDown={e => handleDragStart(e.clientX)}
113         onTouchStart={e => handleDragStart(e.touches[0].clientY)}
114         >
115         <div className="w-10 h-1.5 bg-zinc-300 rounded-full" />
116       </div>
117
118       {/* Scrollable content  */}
119       <div className="flex-1 overflow-y-auto overscroll-contain no-scrollbar px-4 pb-8">
120         {children}
121       </div>
122     </div>
123   );
124 }

```

## LaptopLanding.tsx

TypeScript/React | frontend/src/components/LaptopLanding.tsx

255 lines | 11.5 KB

```

1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   *
5   * LaptopLanding - shown when the app is opened on a desktop/laptop browser.
6   * Gives users a full product overview and a QR code to open the app on mobile.
7   */
8
9   import React from 'react';
10  import { QRCodeSVG } from 'qrcode.react';
11  import {
12    Compass,
13    MapPin,
14    Wifi,
15    ShieldAlert,
16    Accessibility,
17    Zap,

```



```

18   Navigation,
19   Volume2,
20   ArrowRight,
21   Github,
22 } from 'lucide-react';
23
24 const VERCEL_URL = 'https://campuspilot-ai.vercel.app/';
25
26 const FEATURES = [
27   {
28     icon: <Navigation className="w-6 h-6" />,
29     color: 'from-teal-400/20 to-teal-600/10',
30     border: 'border-teal-500/20',
31     accent: 'text-teal-400',
32     title: 'Smart Route Planning',
33     desc: 'Outdoor & indoor routes optimised in real-time around obstacles, closures, and your accessibility needs.',
34   },
35   {
36     icon: <MapPin className="w-6 h-6" />,
37     color: 'from-blue-400/20 to-blue-600/10',
38     border: 'border-blue-500/20',
39     accent: 'text-blue-400',
40     title: 'Real-Time Positioning',
41     desc: 'GPS + BLE beacon fusion gives floor-level accuracy inside any campus building.',
42   },
43   {
44     icon: <Accessibility className="w-6 h-6" />,
45     color: 'from-violet-400/20 to-violet-600/10',
46     border: 'border-violet-500/20',
47     accent: 'text-violet-400',
48     title: 'Accessibility First',
49     desc: 'Six navigation modes – wheelchair, vision, hearing, cognitive, chronic & standard – all customisable.',
50   },
51   {
52     icon: <ShieldAlert className="w-6 h-6" />,
53     color: 'from-red-400/20 to-red-600/10',
54     border: 'border-red-500/20',
55     accent: 'text-red-400',
56     title: 'Emergency SOS',
57     desc: 'One-tap SOS broadcasts your GPS location via cellular SMS and peer-to-peer mesh network simultaneously.',
58   },
59   {
60     icon: <Volume2 className="w-6 h-6" />,
61     color: 'from-amber-400/20 to-amber-600/10',
62     border: 'border-amber-500/20',
63     accent: 'text-amber-400',
64     title: 'Audio Beacon Guidance',
65     desc: 'Directional sound cues help visually impaired users navigate corridors, lifts and doorways hands-free.',
66   },
67   {
68     icon: <Wifi className="w-6 h-6" />,
69     color: 'from-emerald-400/20 to-emerald-600/10',
70     border: 'border-emerald-500/20',
71     accent: 'text-emerald-400',
72     title: 'Mesh Network',
73     desc: 'Device-to-device mesh keeps emergency alerts alive even when cellular coverage drops inside buildings.',
74   },
75 ];
76
77 const STATS = [
78   { value: '6', label: 'Accessibility Modes' },
79   { value: '<3m', label: 'Indoor Accuracy' },
80   { value: '100%', label: 'Privacy – No Login' },
81   { value: '24 / 7', label: 'Emergency Mesh' },
82 ];
83
84 export default function LaptopLanding() {
85   return (
86     <div className="min-h-screen bg-[#080c14] text-white font-sans antialiased overflow-x-hidden">
87
88       {/* ■■■ Ambient background blobs ■■■ */}
89       <div className="fixed inset-0 pointer-events-none overflow-hidden">
90         <div className="absolute -top-40 -left-40 w-[600px] h-[600px] rounded-full bg-teal-600/10 blur-[120px]" />
91         <div className="absolute top-1/3 -right-40 w-[500px] h-[500px] rounded-full bg-blue-600/10 blur-[120px]" />
92         <div className="absolute bottom-0 left-1/3 w-[400px] h-[400px] rounded-full bg-violet-600/8 blur-[100px]" />
93       </div>
94
95       {/* ■■■ Navbar ■■■ */}

```

```
96 <header className="sticky top-0 z-50 border-b border-white/5 bg-[#080c14]/80 backdrop-blur-xl">
97 <div className="max-w-7xl mx-auto px-6 h-16 flex items-center justify-between">
98 <div className="flex items-center gap-2.5">
99 <div className="w-8 h-8 rounded-lg bg-gradient-to-br from-teal-400 to-blue-500 flex items-center justify-center">
100 <Compass className="w-4 h-4 text-white" />
101 </div>
102 <span className="text-lg font-black tracking-tight">CampusPilot <span className="text-teal-400">AI</span></sp...
103 </div>
104 <nav className="hidden md:flex items-center gap-8 text-sm font-semibold text-zinc-400">
105 <a href="#features" className="hover:text-white transition-colors">Features</a>
106 <a href="#how-it-works" className="hover:text-white transition-colors">How It Works</a>
107 <a href="#scan" className="hover:text-white transition-colors">Get Started</a>
108 </nav>
109 </div>
110 </header>
111
112 {/ * ■ ■ Hero ■ ■ */}
113 <section className="relative max-w-7xl mx-auto px-6 pt-24 pb-20 flex flex-col lg:flex-row items-center gap-16">
114
115 {/ * Left copy */}
116 <div className="flex-1 text-center lg:text-left">
117 {/ * Badge */}
118 <div className="inline-flex items-center gap-2 px-3 py-1.5 rounded-full border border-teal-500/30 bg-teal-500/10
119 text-teal-500">
120 Accessibility-First Campus Navigation
121 </div>
122
123 <h1 className="text-5xl lg:text-6xl font-black tracking-tighter leading-[1.05] mb-6">
124 Navigate any campus
125 <br />
126 <span className="bg-gradient-to-r from-teal-400 via-blue-400 to-violet-400 bg-clip-text text-transparent">
127 smarter & safer
128 </span>
129 </h1>
130
131 <p className="text-lg text-zinc-400 leading-relaxed max-w-xl mx-auto lg:mx-0 mb-10">
132 CampusPilot AI delivers real-time indoor & outdoor routing with six accessibility modes,
133 emergency SOS mesh broadcast, and audio beacon guidance – no login required.
134 </p>
135
136 <div className="flex flex-col sm:flex-row items-center justify-center lg:justify-start gap-4">
137 <a
138 href="#features"
139 className="flex items-center gap-2 px-6 py-3.5 rounded-2xl border border-white/10 hover:border-white/20 text-zinc-300
140 font-semib...
141 Explore Features
142 </a>
143 </div>
144 </div>
145
146 {/ * Right – QR card */}
147 <div id="scan" className="flex-shrink-0 flex flex-col items-center gap-5">
148 <div className="relative p-6 rounded-3xl border border-white/10 bg-white/5 backdrop-blur-xl shadow-2xl flex flex-col
149 items-center">
150 <div className="absolute -inset-px rounded-3xl bg-gradient-to-br from-teal-500/20 via-transparent to-blue-500/20
151 pointer-events...">
152 <p className="text-xs font-bold text-zinc-400 tracking-widest uppercase">Scan to Open on Mobile</p>
153
154 <div className="bg-white p-4 rounded-2xl shadow-xl">
155 <QRCodeSVG
156 value={VERCEL_URL}
157 size={180}
158 bgColor="#ffffff"
159 fgColor="#0f172a"
160 level="H"
161 includeMargin={false}
162 />
163 </div>
164
165 <div className="flex flex-col items-center gap-1 text-center">
166 <span className="text-sm font-bold text-white">campuspilot-ai.vercel.app</span>
167 <span className="text-xs text-zinc-500">Point your phone camera at the QR code</span>
168 </div>
169 </div>
170
171 {/ * Platform pill */}
172 <div className="flex items-center gap-2 px-4 py-2 rounded-full bg-white/5 border border-white/10 text-xs text-zinc-400
173 font-semib...>
174 <span className="w-2 h-2 rounded-full bg-teal-400 animate-pulse" />
175 </div>
```

```
174 Works on iOS, Android & any mobile browser
175 </div>
176 </div>
177 </section>
178
179 {/ * ■■ Stats bar ■■ */}
180 <section className="border-y border-white/5 bg-white/[0.02]">
181 <div className="max-w-7xl mx-auto px-6 py-10 grid grid-cols-2 lg:grid-cols-4 gap-8">
182 {STATS.map((s) => (
183 <div key={s.label} className="flex flex-col items-center text-center gap-1">
184 <span className="text-3xl font-black bg-gradient-to-r from-teal-400 to-blue-400 bg-clip-text
185 text-transparent bg-clip-text" style={{writingMode: 'vertical-rl', transform: 'rotate(180deg)'}}>{s.value}</span>
186 <span className="text-sm font-sans text-zinc-500 uppercase tracking-wider">{s.label}</span>
187 ))}
188 </div>
189 </section>
190
191 {/ * ■■ Features ■■ */}
192 <section id="features" className="max-w-7xl mx-auto px-6 py-24">
193 <div className="text-center mb-14">
194 <p className="text-xs font-bold text-teal-400 uppercase tracking-widest mb-3">Platform Capabilities</p>
195 <h2 className="text-4xl font-black tracking-tight mb-4">Everything your campus needs</h2>
196 <p className="text-zinc-400 max-w-xl mx-auto">Designed from the ground up for inclusive, barrier-free campus movement –
197 for everyone.
198 <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-5">
199 {FEATURES.map((f) => (
200 <div
201 key={f.title}
202 className={`relative p-6 rounded-2xl border ${f.border} bg-gradient-to-br ${f.color} backdrop-blur-sm
203 hover:scale-[1.02] transition`}
204 <div className={`${f.accent} mb-4`}>{f.icon}</div>
205 <h3 className="font-bold text-base text-white mb-2">{f.title}</h3>
206 <p className="text-sm text-zinc-400 leading-relaxed">{f.desc}</p>
207 </div>
208 ))}
209 </div>
210 </section>
211
212 {/ * ■■ How it works ■■ */}
213 <section id="how-it-works" className="border-t border-white/5 bg-white/[0.015]">
214 <div className="max-w-5xl mx-auto px-6 py-24 text-center">
215 <p className="text-xs font-bold text-blue-400 uppercase tracking-widest mb-3">Get Started in Seconds</p>
216 <h2 className="text-4xl font-black tracking-tight mb-16">Three steps. Zero friction.</h2>
217 <div className="grid md:grid-cols-3 gap-10">
218 {[
219 { step: '01', title: 'Scan the QR code', desc: 'Use your phone camera to open CampusPilot AI instantly – no App Store
220 download. ', title: 'Pick your mode', desc: 'Choose from six accessibility profiles. Your preference is stored
221 locally. ', title: 'Start navigating', desc: 'Search your destination, get turn-by-turn directions, and tap SOS if
222 you need help. ' }
223 ].map((item) => (
224 <div key={item.step} className="flex flex-col items-center gap-4">
225 <div className="w-14 h-14 rounded-2xl bg-gradient-to-br from-teal-500/20 to-blue-500/20 border border-teal-500/20 flex
226 items-center justify-center">{item.step}</div>
227 <h3 className="font-bold text-white">{item.title}</h3>
228 <p className="text-sm text-zinc-400 leading-relaxed">{item.desc}</p>
229 </div>
230 ))}
231 </div>
232 </div>
233 </section>
234
235 {/ * ■■ Footer CTA ■■ */}
236 <section className="border-t border-white/5">
237 <div className="max-w-7xl mx-auto px-6 py-16 flex flex-col md:flex-row items-center justify-between gap-8">
238 <div className="flex items-center gap-3">
239 <div className="w-9 h-9 rounded-xl bg-gradient-to-br from-teal-400 to-blue-500 flex items-center justify-center">
240 <Compass className="w-5 h-5 text-white" />
241 </div>
242 <div>
243 <p className="font-black text-white">CampusPilot AI</p>
244 <p className="text-xs text-zinc-500">Accessibility-first campus navigation</p>
245 </div>
246 </div>
247 <p className="text-xs text-zinc-600 text-center md:text-right">
248 Built with React + Vite · Leaflet Maps · Deployed on Vercel<br />
249 No login · No data collection · 100% client-side
250 </p>
251 </div>
```

```
252 </section>
253 </div>
254 );
255 }
```

## ReportIssueView.tsx

TypeScript/React | frontend/src/components/ReportIssueView.tsx

459 lines | 18.4 KB

```
1  /**
2   * @license
3   * SPDX-License-Identifier: Apache-2.0
4   */
5
6  import React, { useState } from 'react';
7  import { ArrowLeft, Camera, Send, CheckCircle, MapPin, Droplet, MoreHorizontal, AlertCircle } from 'lucide-react';
8  import { HazardIssue } from '../types';
9  import { INITIAL_ISSUES } from '../data';
10 import { uploadToCloudinary } from '../services/api';
11
12 interface ReportIssueViewProps {
13   onBackClick?: () => void;
14   onSubmitSuccess?: (newIssue: HazardIssue) => void;
15   initialLocation?: string;
16 }
17
18 const CATEGORIES = [
19   { id: 'construction', label: 'Construction' },
20   { id: 'lift', label: 'Lift Not Working' },
21   { id: 'toilet', label: 'Accessible Toilet Closed' },
22   { id: 'wet_floor', label: 'Wet Floor' },
23   { id: 'door', label: 'Door Not Opening' },
24   { id: 'other', label: 'Other' }
25 ] as const;
26
27 // Custom custom SVG icons to match the high-fidelity screenshots
28 const ConstructionIcon = ({ className }: { className?: string }) => (
29   <svg className={className} fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24"
30     stroke="black" data-bbox="125 488 848 508" />
31   <path data-bbox="125 508 848 518" />
32   </svg>
33 );
34
35 const LiftIcon = ({ className }: { className?: string }) => (
36   <svg className={className} fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24"
37     stroke="black" data-bbox="125 553 848 573" />
38   <path data-bbox="125 573 848 583" />
39   <path data-bbox="125 583 848 593" />
40   <path data-bbox="125 593 848 603" />
41   <path data-bbox="125 603 848 613" />
42   </svg>
43 );
44
45 const WheelchairIcon = ({ className }: { className?: string }) => (
46   <svg className={className} fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24"
47     stroke="black" data-bbox="125 673 848 693" />
48   <path data-bbox="125 693 848 703" />
49   <path data-bbox="125 703 848 713" />
50   </svg>
51 );
52
53 const DoorIcon = ({ className }: { className?: string }) => (
54   <svg className={className} fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24"
55     stroke="black" data-bbox="125 763 848 783" />
56   <circle cx="15" cy="12" r="1.5" />
57   </svg>
58 );
59
60 export default function ReportIssueView({
61   onBackClick,
62   onSubmitSuccess,
63   initialLocation = 'Engineering Block A – Corridor B'
64 }: ReportIssueViewProps) {
65   const [selectedCategory, setSelectedCategory] = useState<string>('');
66   const [locationValue, setLocationValue] = useState<string>(initialLocation);
67   const [additionalDetails, setAdditionalDetails] = useState<string>('');
68   const [photoAttached, setPhotoAttached] = useState<boolean>(false);
69   const [photoPreviewUrl, setPhotoPreviewUrl] = useState<string | null>(null);
```

```

70 const [photoFileOrData, setPhotoFileOrData] = useState<File | string | null>(null);
71 const [isUploading, setIsUploading] = useState(false);
72 const [uploadError, setUploadError] = useState<string | null>(null);
73 const [showPhotoOptions, setShowPhotoOptions] = useState(false);
74 const [submissionFeedback, setSubmissionFeedback] = useState(false);
75 const [activeReports, setActiveReports] = useState<HazardIssue[]>(INITIAL_ISSUES);
76
77 const fileInputRef = React.useRef<HTMLInputElement>(null);
78 const [isCameraActive, setIsCameraActive] = useState(false);
79 const [videoStream, setVideoStream] = useState<MediaStream | null>(null);
80 const videoRef = React.useRef<HTMLVideoElement>(null);
81
82 const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
83   const file = e.target.files?.[0];
84   if (file) {
85     if (photoPreviewUrl) {
86       URL.revokeObjectURL(photoPreviewUrl);
87     }
88     const url = URL.createObjectURL(file);
89     setPhotoPreviewUrl(url);
90     setPhotoFileOrData(file);
91     setPhotoAttached(true);
92     setShowPhotoOptions(false);
93   }
94 };
95
96 const startCamera = async () => {
97   try {
98     if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) {
99       throw new Error('Camera access API not supported in this environment');
100     }
101
102     const stream = await navigator.mediaDevices.getUserMedia({
103       video: { facingMode: { ideal: 'environment' } }
104     });
105     setVideoStream(stream);
106     setIsCameraActive(true);
107     setShowPhotoOptions(false);
108     setTimeout(() => {
109       if (videoRef.current) {
110         videoRef.current.srcObject = stream;
111       }
112     }, 100);
113   } catch (err: any) {
114     let errorMessage = err.message || 'Camera access blocked or unsupported';
115     if (err.name === 'NotAllowedError' || err.name === 'PermissionDeniedError') {
116       errorMessage = 'Camera permission was blocked. Please tap the lock/info icon in the URL bar and select "Site Settings"
117       to enable camera access.';
118     } else {
119       fileInputRef.current?.click();
120     }
121     setShowPhotoOptions(false);
122   }
123 };
124
125 const capturePhoto = () => {
126   if (videoRef.current && videoStream) {
127     const canvas = document.createElement('canvas');
128     canvas.width = videoRef.current.videoWidth || 640;
129     canvas.height = videoRef.current.videoHeight || 480;
130     const ctx = canvas.getContext('2d');
131     if (ctx) {
132       ctx.drawImage(videoRef.current, 0, 0, canvas.width, canvas.height);
133       const dataUrl = canvas.toDataURL('image/jpeg');
134       setPhotoPreviewUrl(dataUrl);
135       setPhotoFileOrData(dataUrl);
136       setPhotoAttached(true);
137     }
138     stopCamera();
139   }
140 };
141
142 const stopCamera = () => {
143   if (videoStream) {
144     videoStream.getTracks().forEach(track => track.stop());
145     setVideoStream(null);
146   }
147   setIsCameraActive(false);

```

```

148   };
149
150   React.useEffect(() => {
151     return () => {
152       if (videoStream) {
153         videoStream.getTracks().forEach(track => track.stop());
154       }
155     };
156   }, [videoStream]);
157
158   const renderCategoryIcon = (id: string, className: string) => {
159     switch (id) {
160       case 'construction':
161         return <ConstructionIcon className={className} />;
162       case 'lift':
163         return <LiftIcon className={className} />;
164       case 'toilet':
165         return <WheelchairIcon className={className} />;
166       case 'wet_floor':
167         return <Droplet className={className} />;
168       case 'door':
169         return <DoorIcon className={className} />;
170       case 'other':
171         return <MoreHorizontal className={className} />;
172       default:
173         return <MoreHorizontal className={className} />;
174     }
175   };
176
177   const handleSubmit = async (e: React.FormEvent) => {
178     e.preventDefault();
179     setIsUploading(true);
180     setUploadError(null);
181
182     let imageUrl = undefined;
183     try {
184       if (photoAttached && photoFileOrData) {
185         imageUrl = await uploadToCloudinary(photoFileOrData);
186       }
187
188       const matchedCategory = CATEGORIES.find(c => c.id === selectedCategory);
189       const typeName = matchedCategory ? matchedCategory.label : 'Other issue';
190
191       const newIssue: HazardIssue = {
192         id: `issue-${Date.now()}`,
193         type: (selectedCategory as any) || 'other',
194         typeName,
195         location: locationValue || 'Campus Walkway',
196         details: additionalDetails || 'No details specified',
197         photoAttached,
198         photoUrl: imageUrl,
199         timestamp: new Date().toISOString(),
200         isCustom: true
201       };
202
203       setSubmissionFeedback(true);
204       setActiveReports(prev => [newIssue, ...prev]);
205
206       setTimeout(() => {
207         setSubmissionFeedback(false);
208         setSelectedCategory('');
209         setAdditionalDetails('');
210         setPhotoAttached(false);
211         setPhotoPreviewUrl(null);
212         setPhotoFileOrData(null);
213         if (onSubmitSuccess) {
214           onSubmitSuccess(newIssue);
215         }
216       }, 2500);
217     } catch (err: any) {
218       setUploadError(err.message || 'Failed to upload photo. Please check your configuration.');
```

```

226  { /* Top bar with back option if present */}
227  <div className="flex items-center gap-3 mb-6">
228    {onBackClick && (
229      <button
230        aria-label="Go back"
231        onClick={onBackClick}
232        className="p-1.5 text-zinc-500 hover:text-zinc-800 hover:bg-zinc-100 rounded-full transition-all shrink-0"
233        >
234        <ArrowLeft className="w-5 h-5" />
235      </button>
236    )}
237    <h1 className="text-2xl font-bold font-sans tracking-tight text-[#002f5c]">
238      Report an Issue
239    </h1>
240  </div>
241
242  {submissionFeedback && (
243    <div className="mb-6 bg-teal-50 border border-teal-500/20 rounded-2xl p-4 flex items-start gap-3 shadow-sm"
244      >
245      <div>
246        <h4 className="font-bold text-teal-800">Report Submitted</h4>
247        <p className="text-xs text-teal-700/80 mt-1">
248          Thank you! Facility staff has been alerted. This hazard issue is now integrated into real-time routing engines.
249        </p>
250      </div>
251    </div>
252  )}
253
254  <p className="font-sans text-sm font-medium text-zinc-500 mb-5 text-left">
255    What type of issue are you reporting?
256  </p>
257
258  { /* Grid of issues */}
259  <div className="grid grid-cols-2 gap-4 mb-6" id="issue-grid">
260    {CATEGORIES.map(category => {
261      const isSelected = selectedCategory === category.id;
262      return (
263        <button
264          key={category.id}
265          onClick={() => setSelectedCategory(category.id)}
266          className={`flex flex-col items-center justify-center p-4 bg-white border rounded-2xl min-h-[112px] transition-all
267            ${isSelected ? 'border-[#002f5c] bg-blue-50/20 shadow-md scale-[1.01]' : 'border-zinc-200'}
268          `}
269          type="button"
270        >
271        <span className="mb-2">
272          {renderCategoryIcon(category.id, "w-8 h-8 text-[#002f5c]}")
273        </span>
274        <span className="font-sans font-bold text-xs text-[#002f5c] text-center">
275          {category.label}
276        </span>
277      </button>
278    )};
279  )}
280  </div>
281
282  { /* Details Form Card */}
283  <form onSubmit={handleSubmit} className="space-y-5 bg-white p-5 rounded-2xl border border-zinc-200 shadow-sm text-left">
284    { /* Location Row */}
285    <div className="space-y-2">
286      <label className="font-sans font-bold text-sm text-[#002f5c] flex items-center gap-1.5">
287        <MapPin className="w-4 h-4 text-zinc-450" />
288        Location
289      </label>
290      <div className="relative bg-[#f8f5ff] rounded-xl px-1 border-b-2 border-indigo-100">
291        <input
292          type="text"
293          value={locationValue}
294          onChange={(e) => setLocationValue(e.target.value)}
295          className="w-full bg-transparent border-0 focus:ring-0 font-sans text-sm font-semibold text-zinc-800 px-3 py-2.5"
296        />
297      </div>
298    </div>
299    <div>
300      <div>
301        <div>
302          <div>
303            { /* Additional details */}

```

```

304 <div className="space-y-2">
305 <label htmlFor="details" className="font-sans font-bold text-sm text-[#002f5c]">
306 Additional Details
307 </label>
308 <textarea
309 id="details"
310 rows={3}
311 value={additionalDetails}
312 onChange={(e) => setAdditionalDetails(e.target.value)}
313 className="w-full bg-[#fcfcff] border border-zinc-200 focus:border-[#002f5c] focus:ring-1 focus:ring-[#002f5c]
314 rplAded0x0ef0HAddadetails..."
315 required
316 ></textarea>
317 </div>
318
319 {/* Hidden File input */}
320 <input
321 type="file"
322 accept="image/*"
323 ref={fileInputRef}
324 onChange={handleFileChange}
325 className="hidden"
326 />
327
328 {/* Action / Photo Attach */}
329 <div className="flex items-center gap-4 pt-3 border-t border-zinc-100">
330 {photoAttached && photoPreviewUrl ? (
331 <div className="relative h-12 w-12 rounded-xl overflow-hidden border border-zinc-200 shrink-0 group">
332 <img src={photoPreviewUrl} alt="Preview" className="h-full w-full object-cover" />
333 <button
334 type="button"
335 onClick={() => {
336 if (photoPreviewUrl) URL.revokeObjectURL(photoPreviewUrl);
337 setPhotoAttached(false);
338 setPhotoPreviewUrl(null);
339 setPhotoFileOrData(null);
340 }}
341 className="absolute inset-0 bg-black/60 hover:bg-black/80 flex items-center justify-center text-white text-[10px]
342 font-bold tr...
343 Delete
344 </button>
345 </div>
346 ) : (
347 <button
348 type="button"
349 onClick={() => setShowPhotoOptions(true)}
350 aria-label="Add photo"
351 className="flex-shrink-0 flex items-center justify-center w-12 h-12 rounded-full bg-[#e2dfff] text-[#002f5c]
352 hover:bg-indigo-100...
353 <Camera className="w-5 h-5" />
354 </button>
355 )}
356 <span className="font-sans text-xs text-zinc-500 font-medium text-left">
357 {photoAttached ? '✓ Photo attached' : 'Add a photo to help us locate the issue.'}
358 </span>
359 </div>
360
361 {/* Modal for Photo Options */}
362 {showPhotoOptions && (
363 <div className="fixed inset-0 bg-black/60 backdrop-blur-sm z-[9999] flex items-end justify-center">
364 <div className="w-full max-w-md bg-white rounded-t-3xl p-6 shadow-2xl animate-slide-up text-center">
365 <div className="w-12 h-1.5 bg-zinc-200 rounded-full mx-auto mb-6" />
366 <h3 className="text-base font-extrabold text-[#002f5c] mb-6">Attach Photo</h3>
367
368 <div className="grid grid-cols-2 gap-4">
369 <button
370 type="button"
371 onClick={startCamera}
372 className="flex flex-col items-center justify-center p-5 bg-zinc-50 border border-zinc-200 rounded-2xl
373 hover:bg-zinc-100 tra...
374 <Camera className="w-8 h-8 text-[#002f5c] mb-2" />
375 <span className="text-xs font-bold text-[#002f5c]">Take Photo</span>
376 </button>
377 <button
378 type="button"
379 onClick={() => fileInputRef.current?.click()}
380 className="flex flex-col items-center justify-center p-5 bg-zinc-50 border border-zinc-200 rounded-2xl
381 hover:bg-zinc-100 tra...

```



```

382 <svg className="w-8 h-8 text-[#002f5c] mb-2" fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 24 24">
383 <path strokeLinecap="round" strokeLinejoin="round" d="M2.25 15a4.5 4.5 0 0 4.5 4.5H18a3.75 3.75 0 0 0 1.332-7.257 3 3...
384 </svg>
385 <span className="text-xs font-bold text-[#002f5c]">Upload File</span>
386 </button>
387 </div>
388
389 <button
390 type="button"
391 onClick={() => setShowPhotoOptions(false)}
392 className="w-full mt-5 h-12 bg-zinc-100 hover:bg-zinc-200 text-zinc-650 font-bold rounded-xl text-xs uppercase
393 tracking-wider ...
394 Cancel
395 </button>
396 </div>
397 </div>
398 })
399
400 {/* Error Dialog Banner */}
401 {uploadError && (
402 <div className="bg-red-50 border border-red-200 text-red-600 rounded-2xl p-4 flex items-start gap-3 shadow-sm text-xs
403 font-sans font-medium className="w-5 h-5 text-red-600 shrink-0 mt-0.5" />
404 <span>{uploadError}</span>
405 </div>
406 )}
407
408 {/* Submit */}
409 <button
410 type="submit"
411 disabled={!selectedCategory || isUploading}
412 className="w-full h-14 bg-[#002f5c] disabled:bg-zinc-300 disabled:dark:bg-zinc-800 text-white disabled:text-zinc-500
413 font-sans font-...
414 <Send className="w-4 h-4 transform rotate-0" />
415 {isUploading ? 'Uploading photo & submitting...' : 'Submit Report'}
416 </button>
417 </form>
418
419 {/* Live Camera View Overlay */}
420 {isCameraActive && (
421 <div className="fixed inset-0 bg-black z-[9999] flex flex-col justify-between p-6 text-white text-center">
422 <div className="pt-4 flex justify-between items-center w-full max-w-md mx-auto">
423 <h3 className="text-xs font-extrabold tracking-widest uppercase">Camera Capture</h3>
424 <button
425 type="button"
426 onClick={stopCamera}
427 className="text-xs font-bold text-zinc-400 hover:text-white px-3 py-1.5 rounded-xl bg-zinc-800"
428 >
429 Cancel
430 </button>
431 </div>
432
433 <div className="relative flex-1 w-full max-w-md mx-auto rounded-3xl overflow-hidden bg-zinc-900 my-6 flex items-center
434 justify-ce...
435 ref={videoRef}
436 autoPlay
437 playsInline
438 muted
439 className="w-full h-full object-cover"
440 />
441 {/* Sight guide box overlay */}
442 <div className="absolute inset-10 border-2 border-dashed border-white/20 rounded-2xl pointer-events-none" />
443 </div>
444
445 <div className="w-full max-w-md mx-auto pb-8">
446 <button
447 type="button"
448 onClick={capturePhoto}
449 className="w-20 h-20 bg-white border-8 border-zinc-350 rounded-full cursor-pointer hover:scale-105 active:scale-95
450 transition-tr...
451 <div className="w-10 h-10 bg-[#002f5c] rounded-full" />
452 </button>
453 <span className="text-xs text-zinc-400 font-bold mt-3 block">Tap capture button to save image</span>
454 </div>
455 </div>
456 )}
457 </div>
458 );
459 }

```



## 08 Frontend — Config

**vite.config.ts** TypeScript | frontend/vite.config.ts

22 lines | 0.7 KB

```
1 import tailwindcss from '@tailwindcss/vite';
2 import react from '@vitejs/plugin-react';
3 import path from 'path';
4 import {defineConfig} from 'vite';
5
6 export default defineConfig(() => {
7   return {
8     plugins: [react(), tailwindcss()],
9     resolve: {
10      alias: {
11        '@': path.resolve(__dirname, '.'),
12      },
13    },
14    server: {
15      // HMR is disabled in AI Studio via DISABLE_HMR env var.
16      // Do not modify file watching is disabled to prevent flickering during agent edits.
17      hmr: process.env.DISABLE_HMR !== 'true',
18      // Disable file watching when DISABLE_HMR is true to save CPU during agent edits.
19      watch: process.env.DISABLE_HMR === 'true' ? null : {},
20    },
21  };
22 });
```

**tsconfig.json** JSON | frontend/tsconfig.json

26 lines | 0.5 KB

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "experimentalDecorators": true,
5     "useDefineForClassFields": false,
6     "module": "ESNext",
7     "lib": [
8       "ES2022",
9       "DOM",
10      "DOM.Iterable"
11    ],
12    "skipLibCheck": true,
13    "moduleResolution": "bundler",
14    "isolatedModules": true,
15    "moduleDetection": "force",
16    "allowJs": true,
17    "jsx": "react-jsx",
18    "paths": {
19      "@/*": [
20        "./*"
21      ]
22    },
23    "allowImportingTsExtensions": true,
24    "noEmit": true
25  }
26 }
```

**package.json** JSON | frontend/package.json

40 lines | 1.0 KB

```
1 {
2   "name": "react-example",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite --port=3000 --host=0.0.0.0",
8     "build": "vite build",
9     "preview": "vite preview",
10    "clean": "rm -rf dist server.js",
11    "lint": "tsc --noEmit"
12  },
```

```
13  "dependencies": {
14    "@tailwindcss/vite": "^4.1.14",
15    "@types/leaflet": "^1.9.21",
16    "@types/leaflet-routing-machine": "^3.2.9",
17    "@vitejs/plugin-react": "^5.0.4",
18    "leaflet": "^1.9.4",
19    "leaflet-routing-machine": "^3.2.12",
20    "lucide-react": "^0.546.0",
21    "motion": "^12.23.24",
22    "qrcode.react": "^4.2.0",
23    "react": "^19.0.1",
24    "react-dom": "^19.0.1",
25    "react-leaflet": "^5.0.0",
26    "vite": "^6.2.3"
27  },
28  "devDependencies": {
29    "@types/express": "^4.17.21",
30    "@types/node": "^22.14.0",
31    "autoprefixer": "^10.4.21",
32    "dotenv": "^17.2.3",
33    "esbuild": "^0.25.0",
34    "express": "^4.21.2",
35    "tailwindcss": "^4.1.14",
36    "tsx": "^4.21.0",
37    "typescript": "~5.8.2",
38    "vite": "^6.2.3"
39  }
40 }
```

## 09 Evaluations

run\_eval.py

Python | evals/run\_eval.py

77 lines | 2.4 KB

```

1  """
2  CampusPilot AI – Evaluation Runner
3  =====
4  Runs the 20 golden queries against the intent classifier and profile handler,
5  reporting pass/fail for each query and a final accuracy score.
6
7  Usage:
8  python3 evals/run_eval.py
9
10 Note:
11 This script validates the heuristic classification layer only. Full LLM
12 end-to-end evaluation requires a live GROQ_API_KEY and a running
13 Neo4j instance (see evals/e2e_results.md for those results).
14 """
15
16 import json
17 import os
18 import sys
19
20 # Ensure the project root is on the path for module resolution
21 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
22
23 from backend.agent.intent_classifier import classify_intent
24 from backend.agent.profile_handler import parse_profile
25
26
27 def run_evals(queries_path: str = None) -> dict:
28     """
29     Loads the golden query dataset and evaluates intent classification accuracy.
30
31     Args:
32     queries_path (str): Path to the golden_queries.json file. Defaults to
33     the file co-located with this script.
34
35     Returns:
36     dict: A summary containing 'passed', 'total', and 'accuracy' keys.
37     """
38     if queries_path is None:
39         queries_path = os.path.join(os.path.dirname(__file__), "golden_queries.json")
40
41     with open(queries_path, "r", encoding="utf-8") as f:
42         queries = json.load(f)
43
44     passed = 0
45     total = len(queries)
46
47     print("=" * 65)
48     print(" CampusPilot AI – Intent Classification Evaluation")
49     print("=" * 65)
50
51     for q in queries:
52         predicted_intent = classify_intent(q["query"])
53         profile = parse_profile(q["profile"]).value
54         expected = q["expected_intent"]
55
56         success = predicted_intent == expected
57         if success:
58             passed += 1
59             status = "■ PASS"
60         else:
61             status = f"■ FAIL (got='{predicted_intent}', expected='{expected}')"
62
63         print(f"[{q['id']:>3}] ({profile:<18}) {status}")
64         print(f" Query: \"{q['query']}\")")
65
66     accuracy = passed / total * 100
67     print("=" * 65)
68     print(f" Result: {passed}/{total} passed | Accuracy: {accuracy:.1f}%")
69     print("=" * 65)
70

```

```

71     return {"passed": passed, "total": total, "accuracy": accuracy}
72
73
74     if __name__ == "__main__":
75         results = run_evals()
76         # Exit with a non-zero code if accuracy drops below 80% (CI gate)
77         sys.exit(0 if results["accuracy"] >= 80.0 else 1)

```

## golden\_queries.json

JSON | evals/golden\_queries.json

162 lines | 5.1 KB

```

1  [
2  {
3      "id": "q1",
4      "query": "Take me to the library.",
5      "profile": "Wheelchair",
6      "expected_intent": "navigate",
7      "expected_tool": "route_query",
8      "notes": "Standard navigation; wheelchair profile must trigger avoid_stairs=true"
9  },
10 {
11     "id": "q2",
12     "query": "Where is the nearest restroom?",
13     "profile": "Vision-impaired",
14     "expected_intent": "find_facility",
15     "expected_tool": "route_query",
16     "notes": "Facility lookup; vision-impaired profile must trigger detailed_landmarks=true"
17 },
18 {
19     "id": "q3",
20     "query": "I fell down the stairs, I need help!",
21     "profile": "Invisible",
22     "expected_intent": "emergency",
23     "expected_tool": "flag_obstacle",
24     "notes": "Emergency override – must take strict priority over all other intents"
25 },
26 {
27     "id": "q4",
28     "query": "Is there a ramp to the main building?",
29     "profile": "Wheelchair",
30     "expected_intent": "accessibility_query",
31     "expected_tool": "route_query",
32     "notes": "Accessibility-specific query, not a navigation request"
33 },
34 {
35     "id": "q5",
36     "query": "Go to the cafeteria.",
37     "profile": "Cognitive",
38     "expected_intent": "navigate",
39     "expected_tool": "route_query",
40     "notes": "Simple navigation; cognitive profile must trigger simplified_steps=true"
41 },
42 {
43     "id": "q6",
44     "query": "Where can I find a quiet room?",
45     "profile": "Cognitive",
46     "expected_intent": "find_facility",
47     "expected_tool": "route_query",
48     "notes": "Facility-specific query with cognitive profile"
49 },
50 {
51     "id": "q7",
52     "query": "Route to the CS department.",
53     "profile": "Hearing-impaired",
54     "expected_intent": "navigate",
55     "expected_tool": "route_query",
56     "notes": "Navigation with hearing-impaired profile; must trigger visual_cues_only=true"
57 },
58 {
59     "id": "q8",
60     "query": "I'm lost.",
61     "profile": "Invisible",
62     "expected_intent": "emergency",
63     "expected_tool": "flag_obstacle",
64     "notes": "Implicit distress signal; should be classified as emergency"
65 },
66 {

```

```
67   "id": "q9",
68   "query": "Does the engineering building have an elevator?",
69   "profile": "Wheelchair",
70   "expected_intent": "accessibility_query",
71   "expected_tool": "route_query",
72   "notes": "Verifies accessibility query detection for elevator keywords"
73 },
74 {
75   "id": "q10",
76   "query": "Directions to room 101.",
77   "profile": "Vision-impaired",
78   "expected_intent": "navigate",
79   "expected_tool": "route_query",
80   "notes": "Specific room navigation with vision-impaired landmark enrichment"
81 },
82 {
83   "id": "q11",
84   "query": "Fire alarm is going off near the gym.",
85   "profile": "Hearing-impaired",
86   "expected_intent": "emergency",
87   "expected_tool": "flag_obstacle",
88   "notes": "Fire keyword triggers emergency; hearing-impaired adds visual alert context"
89 },
90 {
91   "id": "q12",
92   "query": "Find the nearest accessible parking.",
93   "profile": "Wheelchair",
94   "expected_intent": "find_facility",
95   "expected_tool": "route_query",
96   "notes": "Accessible parking is a find_facility intent, not navigate"
97 },
98 {
99   "id": "q13",
100  "query": "Navigate to the student health centre.",
101  "profile": "Cognitive",
102  "expected_intent": "navigate",
103  "expected_tool": "route_query",
104  "notes": "navigate keyword explicit; simplified steps required for cognitive profile"
105 },
106 {
107   "id": "q14",
108   "query": "I need to get to the admin office urgently.",
109   "profile": "Invisible",
110   "expected_intent": "navigate",
111   "expected_tool": "route_query",
112   "notes": "Urgency does not override to emergency without a distress keyword"
113 },
114 {
115   "id": "q15",
116   "query": "Is the main auditorium wheelchair accessible?",
117   "profile": "Wheelchair",
118   "expected_intent": "accessibility_query",
119   "expected_tool": "route_query",
120   "notes": "accessibility_query: wheelchair keyword + 'accessible' keyword"
121 },
122 {
123   "id": "q16",
124   "query": "I need police. Someone stole my bag.",
125   "profile": "Invisible",
126   "expected_intent": "emergency",
127   "expected_tool": "flag_obstacle",
128   "notes": "police keyword must trigger emergency classification immediately"
129 },
130 {
131   "id": "q17",
132   "query": "Where is the science block?",
133   "profile": "Vision-impaired",
134   "expected_intent": "find_facility",
135   "expected_tool": "route_query",
136   "notes": "where is pattern triggers find_facility; detailed landmarks expected"
137 },
138 {
139   "id": "q18",
140   "query": "Go to the principal's office.",
141   "profile": "Cognitive",
142   "expected_intent": "navigate",
143   "expected_tool": "route_query",
144   "notes": "Go to pattern triggers navigate; simplified steps for cognitive profile"
```

```
145 },
146 {
147   "id": "q19",
148   "query": "Are there tactile paths near the library entrance?",
149   "profile": "Vision-impaired",
150   "expected_intent": "accessibility_query",
151   "expected_tool": "route_query",
152   "notes": "Accessibility-specific query focused on tactile navigation infrastructure"
153 },
154 {
155   "id": "q20",
156   "query": "I am hurt and cannot move.",
157   "profile": "Invisible",
158   "expected_intent": "emergency",
159   "expected_tool": "flag_obstacle",
160   "notes": "hurt keyword must trigger emergency; Invisible profile as fallback"
161 }
162 ]
```



10

Documentation

README.md

Markdown | README.md

60 lines | 2.4 KB

```
1  # CampusPilot AI
2
3  Agentic AI-powered accessible campus navigation – LLM agent, QR indoor positioning, 5 disability profiles, offline-first
4  PWA. Built for Capgem...
5  ## Team & Roles
6  - **Anshul** – LangChain agent, tools, memory, prompts, golden queries, eval scripts
7  - **Dhruv** – FastAPI app, routers, Neo4j connection, database schema/seed, Architecture Decision Records (ADRs)
8  - **Rishabh** – pytest unit + integration tests, polished README
9  - **Hrishikesh** – React PWA (Vite + TypeScript), cost models, rollout plan, demo videos
10 - **Aatharva** – Primary documentation, competitive analysis, citations, WCAG
11
12 ## Links
13 - **Live Demo (Frontend):** [TBD]
14 - **API Base URL:** [TBD]
15 - **Demo Video:** [TBD - Link to video or /demo folder]
16 - **Documentation:** [/docs](./docs)
17
18 ## Tech Stack
19 ![Generative AI](https://img.shields.io/badge/Generative_AI-121011?style=flat&logo=openai&logoColor=white)
20 ![LangChain](https://img.shields.io/badge/LangChain-121011?style=flat&logo=langchain&logoColor=white)
21 ![Groq](https://img.shields.io/badge/Groq-121011?style=flat&logo=groq&logoColor=white)
22 ![Neo4j](https://img.shields.io/badge/Neo4j-121011?style=flat&logo=neo4j&logoColor=white)
23 ![FastAPI](https://img.shields.io/badge/FastAPI-121011?style=flat&logo=fastapi&logoColor=white)
24 ![React PWA](https://img.shields.io/badge/React_PWA-121011?style=flat&logo=react&logoColor=white)
25
26 ## AI Agent Architecture (Backend)
27 The backend utilizes an **Agentic Flow** driven by Llama 3.3 70B via Groq and LangChain.
28 - **Tools**: The agent dynamically calls tools (`route_query`, `qr_lookup`, `profile_detect`, `flag_obstacle`) to
29     catch up on the current state and maintain contextual state across conversations (e.g. your last scanned QR code, active disability
30     profile, etc.). Queries are pre-classified (navigation vs. emergency) to ensure lightning-fast responses and strict
31     fallbacks.
32 ## Setup Instructions
33
34 ### Prerequisites
35 - Node.js (for frontend)
36 - Python 3.9+ (for backend)
37 - Neo4j Database
38
39 ### Environment Variables
40 1. Copy `.env.example` to `.env`
41 2. Fill in the required values (e.g., `GROQ_API_KEY`). **DO NOT commit your `.env` file.**
42
43 ### How to run Backend
44 ```bash
45 # 1. Move into the backend directory
46 cd backend
47
48 # 2. Install the required Python packages
49 pip3 install -r requirements.txt
50
51 # 3. Start the FastAPI server (coming soon)
52 # uvicorn main:app --reload
53 ```
54
55 ### How to run Frontend
56 ```bash
57 cd frontend
58 npm install
59 npm run dev
60 ```
```

architecture.md

Markdown | docs/architecture.md

57 lines | 2.7 KB

```
1  # CampusPilot AI – Core Solution Architecture
2
3  **Author:** Dhruv Sarda (Core Solution Architecture Lead)
4  **Version:** 1.0 | June 9, 2026
5
```

```
6 ---
7
8 ## 1. System Components Overview
9
10 CampusPilot AI consists of four major layers interacting in real-time to provide context-aware, offline-resilient, and
11 disability-accessible w...
12 1. Vite React PWA Frontend: Serves as the user interface, incorporating accessibility-compliant viewport settings,
13 FastAPI Backend: Orchestrates API requests, translates client profiles/locations, calls the AI agent
14 LangChain Agent (Llama 3.3 70B via Groq): Classifies query intents, extracts destinations, detects disability
15 Neo4j Graph Database: Stores the digital twin of the campus layout (rooms, halls, corridors, entrances, stairs,
16 lifts, and QR points).
17 ```mermaid
18 graph TD
19   Client[React PWA Client] -->|Scan QR / Send Msg| API[FastAPI Backend]
20   API -->|Intent / Context enrichment| Agent[LangChain Agent - Groq/Llama]
21   Agent -->|Resolve query & metadata| API
22   API -->|Cypher Route Query| DB[(Neo4j Graph DB)]
23   DB -->|Shortest path graph results| API
24   API -->|Format Route + Instructions| Client
25   ```
26
27 ---
28
29 ## 2. Graph Database Modeling
30
31 We represent campus infrastructure as a mathematical graph  $G = (V, E)$ , enabling efficient multi-constraint pathfinding
32 queries.
33 ### Nodes (V)
34 - `QRPoint`: Physical QR codes serving as offline positioning landmarks.
35 - `Room`: Classrooms, offices, laboratories, auditoriums.
36 - `Corridor`: Junctions and segments linking campus nodes.
37 - `Lift` / `Stairs`: Vertical transition shafts.
38 - `Entrance`: Entry gates connecting outdoor routes to indoor structures.
39
40 ### Relationships (E)
41 - `CONNECTS_TO`: Direct walking pathways with properties:
42   - `distance_meters` (float)
43   - `wheelchair_accessible` (boolean)
44   - `tactile_paving` (boolean)
45   - `stairs` (boolean)
46   - `lift` (boolean)
47
48 ---
49
50 ## 3. End-to-End Navigation Flow
51
52 The path calculation runs as follows:
53 1. The user scans a QR code at their starting location, which sets the starting `QRPoint` node.
54 2. The user types a request (e.g., "Take me to Seminar Hall 101").
55 3. The FastAPI router parses the input, queries the intent classifier, resolves the destination room name from the text
56 query badcode Neo4j using Cypher pathfinding filtered by the user's disability profile constraints.
57 5. The computed node path and edges are formatted into directional instructions and displayed to the user.
```

## ai\_architecture.md

Markdown | docs/ai\_architecture.md

89 lines | 4.8 KB

```
1 # CampusPilot AI – Architecture Document
2
3 **Author:** Anshul Kumaria (AI & Intelligence Lead)
4 **Version:** 1.0 | June 9, 2026
5
6 ---
7
8 ## 1. Why Agentic AI and Not RAG-Only?
9
10 Most AI navigation assistants are built on a Retrieval-Augmented Generation (RAG) architecture: the user asks a
11 question, the system retrie...
12 When a wheelchair user says "take me to the library", the system does not need to retrieve a paragraph about the
13 library. It needs to:
14 1. Know where the user currently is (via QR scan).
15 2. Know what routing constraints apply (wheelchair → avoid stairs, prefer elevators).
16 3. Query a live graph database (Neo4j) for the shortest accessible path.
17 4. Format a step-by-step response using tactile, auditory, or simplified language depending on the profile.
18
19 A RAG system cannot do steps 1-3 because it cannot call external tools or reason about what information it needs next.
20 An agent can. This ...
21 ---
```

```
22
23 ## 2. The ReAct Pattern
24
25 CampusPilot AI implements the ReAct (Reason + Act) loop, as formalised by Yao et al. (2022). The LLM alternates
26 between two phases:
27 - Thought: The model introspects what information it is missing and decides what action to take next.
28 - Action: The model invokes a tool and waits for the observation (the tool's return value).
29
30 This continues in a loop until the model has enough information to generate its final response.
31
32 ### Example Execution Trace
33
34 User: "Take me to the library." | Profile: Wheelchair | QR Scanned: #102
35
36 ```
37 Thought: I need the user's current location. I will look up QR #102.
38 Action: qr_lookup("102")
39 Observation: "Main Entrance"
40
41 Thought: I now know start=Main Entrance. The profile is Wheelchair,
42 so I must avoid stairs and prefer elevators.
43 Action: route_query(start="Main Entrance", end="Library", profile="Wheelchair")
44 Observation: "Go straight → Elevator B → Floor 2 → Left → Library"
45
46 Thought: I have the route. I will format a clear, step-by-step response.
47 Final Response: "You are at the Main Entrance. Walk straight to Elevator B
48 (10 metres ahead). Take it to Floor 2. Turn left – the library
49 is 15 metres ahead on your right."
50 ```
51
52 ---
53
54 ## 3. Tool Registry
55
56 The agent has access to four tools, each backed by a LangChain `@tool` decorator:
57
58 | Tool | Purpose | Key Argument |
59 |---|---|---|
60 | `qr_lookup` | Resolves a QR code ID to a campus node | `qr_id: str` |
61 | `route_query` | Queries Neo4j for the optimal route given a profile | `start_node, end_node, profile` |
62 | `profile_detect` | Infers the user's disability profile from text if not explicitly set | `user_input: str` |
63 | `flag_obstacle` | Records a temporary obstacle on the shadow map | `location, description` |
64
65 The LLM autonomously decides which tools to call and in what order. This is not a hard-coded flow – it is emergent from
66 the system prompt and ...
67
68 ---
69
70 ## 4. Memory and State Management
71
72 Navigation is a multi-turn problem. A user might scan a QR code in message 1, ask a follow-up question in message 3,
73 and report an obstacle...
74 - Conversation history: every user/assistant turn is stored and prepended to each LLM call.
75 - Last known QR location: persisted between turns so the agent knows where the user is even without rescanning.
76 - Active disability profile: set at the start of a session and propagated to every `route_query` call automatically.
77
78 ---
79
80 ## 5. Emergency Handling Architecture
81
82 Emergency responses bypass the LLM entirely. The `classify_intent()` function runs before the agent executor is
83 invoked. If the intent is ...
84 This design decision is safety-critical: we cannot allow network latency or LLM inference time to delay an SOS response.
85
86 ---
87
88 ## 6. Disability Profile Integration
89
90 The five profiles (Wheelchair, Vision-impaired, Hearing-impaired, Cognitive, Invisible) are not described in the system
91 prompt. They are opera...
```

problem\_analysis.md

Markdown | docs/problem\_analysis.md

85 lines | 2.2 KB

```
1 # Campus Navigation Failure: Root Cause Analysis
2
3 ## Problem Statement
4
5 Students, faculty members, visitors, and individuals with accessibility needs frequently experience difficulty
6 navigating large campuses due to...
```

```
6
7  ## 5 Whys Analysis
8
9  ### Problem
10 Users struggle to navigate large campuses efficiently.
11
12 ### Why 1?
13 Indoor navigation is often inaccurate.
14
15 ### Why 2?
16 Most navigation systems rely heavily on GPS.
17
18 ### Why 3?
19 GPS signals become unreliable inside buildings.
20
21 ### Why 4?
22 Existing navigation solutions were primarily designed for outdoor environments.
23
24 ### Why 5?
25 Campus navigation has traditionally been treated as a mapping problem rather than an accessibility and wayfinding
26 challenge.
27 ## Root Cause
28
29 Existing campus navigation systems lack four critical capabilities:
30
31 - Accurate indoor positioning
32 - Accessibility-aware routing
33 - Real-time environmental awareness
34 - Reliable offline operation
35
36 ## Secondary Root Causes
37
38 ### Accessibility Gap
39
40 Most solutions only support basic wheelchair routing and fail to address the needs of visually impaired,
41 hearing-impaired, cognitively diverse...
42 ### Real-Time Information Gap
43
44 Route disruptions such as blocked corridors, broken elevators, maintenance activities, and temporary closures are not
45 reflected in navigation ...
46 ### Connectivity Gap
47
48 Many solutions become ineffective during network outages, emergencies, or periods of high congestion.
49
50 ### Adoption Gap
51
52 Users are often reluctant to install dedicated navigation applications for infrequent use.
53
54 ## Impact
55
56 - Lost time
57 - Increased stress
58 - Reduced accessibility
59 - Delayed arrival
60 - Poor visitor experience
61 - Lower emergency preparedness
62
63 ## Why Existing Solutions Fall Short
64
65 ### Google Maps
66 Limited indoor navigation support.
67
68 ### Static Campus Maps
69 No real-time updates.
70
71 ### Physical Signage
72 Requires manual interpretation and local familiarity.
73
74 ### Accessibility Apps
75 Typically focus on only one disability category.
76
77 ## Why CampusPilot AI
78
79 CampusPilot AI addresses these challenges through:
80
81 - Conversational navigation
82 - Accessibility-first routing
83 - QR-based indoor positioning
```

```
84 - Shadow Mapping for real-time adaptation
85 - Offline Bluetooth mesh communication
```

limitations\_and\_risks.m

Markdown | docs/limitations\_and\_risks.md

56 lines | 3.8 KB

```
1 # Risks & Limitations
2
3 > An honest risk section *strengthens* a submission – it shows the team
4 > understands the problem. Each risk below has a mitigation.
5
6 ## 1. Technical risks
7
8 | Risk | Why it matters | Mitigation |
9 |-----|-----|-----|
10 | **QR positioning is point-based** | Location is known only at scan points; between QR codes the position goes stale;
11 | **Model dependency** | An agent (Llama 3.3 70B via Groq) that invents a route or facility could
12 | **Out-of-date status data** | An out-of-date "elevator working" record can strand a wheelchair user | "Last verified"
13 | **Campus dead-zones** | Campus dead-zones; users may lose signal mid-route | Cache active routes; degrade
14 | **Slow responses** | Slow responses hurt usability; per-call cost adds up | Cache common queries/routes;
15 | keep the agent's to...
16
17 ## 2. Data & privacy risks
18 - **Sensitive disability data.** Storing disability profiles is high-risk
19   personal data → collect the minimum, get consent, allow deletion, anonymise
20   location history (see `data_strategy.md` §4).
21 - **Bias in routing/recommendations.** A model could systematically favour some
22   users → test across profiles; lean on the shared-preference research to design
23   fairly.[INsite]
24 - **Security.** Real-time location + identity is a target → standard auth,
25   encryption in transit and at rest, least-privilege access.
26
27 ## 3. Accessibility & UX risks
28 - **Over-reliance.** Users may trust the app over their own judgement → present
29   guidance as assistance, encourage verification, never imply certainty the data
30   doesn't support.
31 - **Coverage gaps.** Buildings without verified accessibility data → be explicit
32   in the UI about unverified areas rather than guessing.
33 - **Edge cases per disability.** One design can't cover everyone → continue
34   testing with real assistive tech and, ideally, disabled users (automated WCAG
35   scans catch only part of the problem – see `wcag_compliance.md`).
36
37 ## 4. Operational & compliance risks
38 - **QR maintenance.** Physical QR codes can be removed, defaced, or covered →
39   a periodic check/replacement plan, and a way for users to report a missing code
40   (see `tech_stack_rationale.md`). (Still far lighter than beacon battery upkeep.)
41 - **Regulatory scope.** Public universities face an April 2026 ADA deadline; a
42   non-conformant tool is a compliance liability → treat WCAG 2.1/2.2 AA as a
43   hard requirement, not a stretch goal.[ADA2026]
44 - **Scale beyond one campus.** Accessibility data is campus-specific and
45   labour-intensive to collect → a clear data-onboarding process if expanding.
46
47 ## 5. Known limitations (state these plainly)
48 - Accuracy is bounded by the quality and freshness of campus accessibility data.
49 - Indoor positioning is approximate, not exact.
50 - The system assists navigation; it does not replace a user's own judgement,
51   mobility aids, or human help where needed.
52
53 > **[CONFIRM]** Add any risks specific to your actual implementation (e.g. a
54 > particular model's limits, a dataset gap you know about, demo-only shortcuts).
55
56 ---
57 *Sources: see `citations.md` – [Springer-Mapping], [Manduchi], [INsite], [ADA2026].*
```

acceptance\_criteria.md

Markdown | docs/acceptance\_criteria/acceptance\_criteria.md

86 lines | 2.2 KB

```
1 # CampusPilot AI – Acceptance Criteria
2
3 ## User Story 1: First-Year Student
4
5 ### Given
6 - The user is a new student unfamiliar with the campus.
7 - The user has entered a destination.
8
9 ### When
```

```
10 - The user requests navigation assistance.
11
12 ### Then
13 - The system provides a valid route to the destination.
14 - The route includes clear step-by-step instructions.
15 - Estimated travel time is displayed.
16 - Indoor and outdoor navigation are supported.
17 - The route is generated within acceptable response time.
18
19 ---
20
21 ## User Story 2: Wheelchair User
22
23 ### Given
24 - The user's accessibility profile is set to wheelchair user.
25
26 ### When
27 - The user requests a route.
28
29 ### Then
30 - The system automatically excludes stair-only paths.
31 - Elevators and ramps are prioritized.
32 - Accessible entrances are preferred.
33 - Users are notified about elevator outages or blocked accessible routes.
34 - The route remains compliant with accessibility constraints.
35
36 ---
37
38 ## User Story 3: Visually Impaired Professor
39
40 ### Given
41 - The user profile indicates visual impairment.
42
43 ### When
44 - The user requests navigation assistance.
45
46 ### Then
47 - Navigation instructions are provided through audio output.
48 - The interface remains compatible with screen readers.
49 - Landmark-based guidance is included.
50 - Critical alerts are delivered through audio and haptic feedback.
51 - Visual interaction is not required to complete navigation.
52
53 ---
54
55 ## User Story 4: Campus Administrator
56
57 ### Given
58 - The administrator has authorized system access.
59
60 ### When
61 - Route information is updated.
62
63 ### Then
64 - Changes are reflected in the navigation graph.
65 - Affected routes are recalculated automatically.
66 - Users receive updated route recommendations.
67 - Historical updates are logged for auditing purposes.
68 - Route modifications become available without requiring application reinstallation.
69
70 ---
71
72 ## User Story 5: Emergency User
73
74 ### Given
75 - An emergency situation exists.
76 - Internet connectivity may be unavailable.
77
78 ### When
79 - The user activates emergency mode or SOS.
80
81 ### Then
82 - The system provides the nearest safe evacuation route.
83 - Emergency alerts are transmitted through the mesh network.
84 - Last known location is attached to the alert.
85 - Emergency routing remains available offline.
86 - The user receives confirmation that the alert has been transmitted.
```

## tech\_stack\_rationale.md

Markdown | docs/research/tech\_stack\_rationale.md

44 lines | 4.1 KB

```
1 # Tech Stack Rationale
2
3 > Updated to match the project README. The pattern below – *decision → why →
4 > trade-off* – is what reviewers want for each component.
5
6 ## Stack at a glance
7
8 | Layer | Choice | Why this choice | Trade-off |
9 |-----|-----|-----|-----|
10 | **LLM** | **Llama 3.3 70B via Groq** | Groq's LPU inference is extremely low-latency – critical for the "
11 | lightweight framework" | **GolangChama** | Mature framework for the agentic loop: tool calling, memory, and prompt
12 | orchestration | **oneAgentFlow** with dynamic tool calls | The model decides *which* tool to call per query (vs. a
13 | fixed workflow) | **one_query**, 'qr_lookup', 'profile_detect', 'flag_obstacle' | Each maps a user need to a concrete action
14 | over memory graph | **oneAgentFlow** | Lets the agent personalise without
15 | **oneAgentFlow** | **oneAgentFlow** | Fast paths + strict fallbacks; emergency queries must
16 | **oneAgentFlow** | **oneAgentFlow** | A property graph is the natural model for routing: locations as nodes, *accessible* paths
17 | **oneAgentFlow** | **oneAgentFlow** | **QR codes** ('qr_lookup') | **No special hardware, no beacon batteries, cheap to deploy and
18 | **oneAgentFlow** | **oneAgentFlow** | (Python 3.9+) | Async, typed request models, auto OpenAPI docs; pairs well with the Python
19 | **oneAgentFlow** | **oneAgentFlow** | (Vite + TypeScript)** | Installable + **offline-first** via service worker (no signal in
20 | **oneAgentFlow** | **oneAgentFlow** | (unit + integration) | Standard, readable; integration tests catch agent/tool/DB wiring
21 | issues | Needs a labelled s...
22 ## Why the two headline choices are smart
23
24 - **QR over BLE beacons.** Beacon systems (ClickAndGo, Lazarillo) need hardware
25 that costs money and *batteries that die* – a maintenance liability we flagged
26 in 'limitations_and_risks.md'. QR anchors are nearly free and trivially
27 replaceable, which suits a campus and a hackathon budget.[NLS][Lazarillo]
28 - **Neo4j over a relational DB for routing.** Accessible routing is fundamentally
29 a graph problem (find the best *step-free* path); modelling accessibility
30 attributes directly on relationships is cleaner in a graph DB than joining
31 rows.
32
33 ## Accessibility-driven constraints
34 - Frontend framework choice is also an **accessibility** decision – components
35 must ship correct ARIA and pass VoiceOver/TalkBack testing (see
36 'wcag_compliance.md').[Perkins]
37 - QR positioning must have an accessible scan flow for blind users (audio
38 guidance to find/scan a code).
39
40 > **[CONFIRM]** Add exact versions, where it's hosted, and any service the demo
41 > depends on. Link the official docs (now listed in 'citations.md').
42
43 ---
44 *Sources: see 'citations.md' – [NLS], [Lazarillo], [Perkins].*
```

## adr-001-neo4j.md

Markdown | docs/adr/adr-001-neo4j.md

21 lines | 1.3 KB

```
1 # ADR-001: Selection of Neo4j for Campus Navigation Graph
2
3 ## Status
4 Approved
5
6 ## Context
7 Campus environments consist of complex indoor layouts spanning multiple floors, buildings, corridors, lifts, stairs,
8 and need to provide seamless routes filtered by accessibility constraints. For example, a wheelchair user needs paths
9 that are wheelchair accessible. Relational databases (like PostgreSQL) require recursive CTEs or complicated join queries to calculate paths across many
10 hops in an indoor mod...
11 ## Decision
12 We chose Neo4j (a graph database) as the primary data store for the campus map representation.
13
14 ## Consequences
15 - **Pros**:
16 - Cypher query language provides native shortest path algorithms (e.g., 'shortestPath') with inline relationship
17 attributes for easy integration of physical topologies where nodes represent locations and edges represent walkable pathways.
18 - Easy filtering of paths by properties (e.g., 'WHERE ALL(r IN relationships(p) WHERE r.wheelchair_accessible = true)').
19 - **Cons**:
20 - Requires running a separate Neo4j database instance.
21 - Less common for general-purpose transactional storage.
```

## adr-002-fastapi.md

Markdown | docs/adr/adr-002-fastapi.md

21 lines | 1.3 KB

```
1 # ADR-002: FastAPI over Flask/Django and REST over GraphQL
2
3 ## Status
4 Approved
5
6 ## Context
7 We require a lightweight, high-performance web service to power our backend navigation APIs. The API needs to receive
8 requests asynchronously and schema validation out of the box, whereas Django is too heavy and
9 Python-based, we need to decide between REST and GraphQL for communication between our frontend PWA and backend APIs.
10
11 ## Decision
12 We chose FastAPI as our python web framework and REST APIs for client-server communication.
13
14 ## Consequences
15 - **Pros**:
16 - Asynchronous capability (`async/await`) allows handling multiple parallel connections efficiently, especially during
17 media streaming. OpenAPI documentation generation (Swagger UI) allows implementation team to quickly test endpoints.
18 - Native Pydantic integration for strict request/response validation.
19 - REST is simple to cache, works with standard proxies/service workers, and has less client-side footprint than GraphQL
20 client libraries, su...
21 - Requires explicit code setup for dependency injection.
```



## End of Source Code

|            |                                                                                                   |
|------------|---------------------------------------------------------------------------------------------------|
| Repository | <a href="https://github.com/AK-1612/campuspilot-ai">https://github.com/AK-1612/campuspilot-ai</a> |
| Team       | Anshul Kumaria · Dhruv Sarda · Hrishikesh Kali · Atharva Deshmukh · Rishabh Alekar                |
| Event      | CapGemini Build-a-thon 2026                                                                       |
| Date       | June 14, 2026                                                                                     |